

IA353 – Redes Neurais

Tópico 9 – Deep Learning

Parte 2 – Redes Convolucionais + Dropout + Comitê de Máquinas

Índice Geral

1	Redes Neurais Convolucionais.....	2
1.1	Inspiração no campo de receptores.....	3
1.2	Operação de convolução	11
1.3	Filtros extratores de atributos.....	29
1.4	A operação de convolução na prática	41
1.5	A operação de convolução transposta na prática	46
1.6	Camadas convolucionais × camadas densas	55
1.7	Operadores max pooling e average pooling.....	61
1.8	Redes convolucionais históricas	63
1.9	Como projetar redes convolucionais?.....	67
2	Convoluções separáveis	77
2.1	Convolução separável no espaço.....	77
2.2	Convolução separável em profundidade	81
3	Dropout	95
4	Comitê de máquinas – Ensemble e Mistura de Especialistas	101
5	Referências bibliográficas	113

1 Redes Neurais Convolucionais

- Parte das figuras desta seção foi extraída de:
 - ✓ < <https://medium.freecodecamp.org/an-intuitive-guide-to-convolutional-neural-networks-260c2de0a050> >
 - ✓ < <https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/> >
 - ✓ < <http://neuralnetworksanddeeplearning.com/chap6.html> >
- Parte das figuras desta seção foi extraída de vídeos de Leo Isikdogan:
 - ✓ < <https://www.youtube.com/watch?v=-I0lry5ceDs> >
 - ✓ < <https://www.youtube.com/watch?v=fTw3K8D5xDs> >
- Parte das figuras desta seção foi extraída de vídeos de Siraj Raval:
 - ✓ < <https://www.youtube.com/watch?v=FTr3n7uBIuE&t=1339s> >
- Para a extração de atributos relevantes de imagens, camadas convolucionais, seguidas de operações de subamostragem, como *max pooling*, têm se mostrado capazes de conduzir a modelos de aprendizado de elevado desempenho.
- Rede Neural Convolucional (*Convolutional Neural Network*) = ConvNet = CNN

1.1 Inspiração no campo de receptores

- Os operadores convolucionais foram inspirados na forma como os sensores visuais cerebrais de mamíferos operam, usando campos de receptores: Robertson, B. “*A celebration of the 50th anniversary of David Hubel and Torsten Wiesel’s Receptive fields of single neurones in the cat’s striate cortex*”, The Journal of Physiology, vol. 587, no. 12, pp. 2721-2732, 2009.

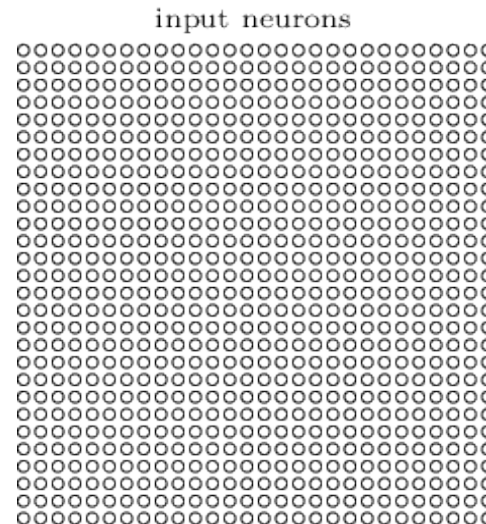


Figura 1 – Campo de receptores em um sensor visual

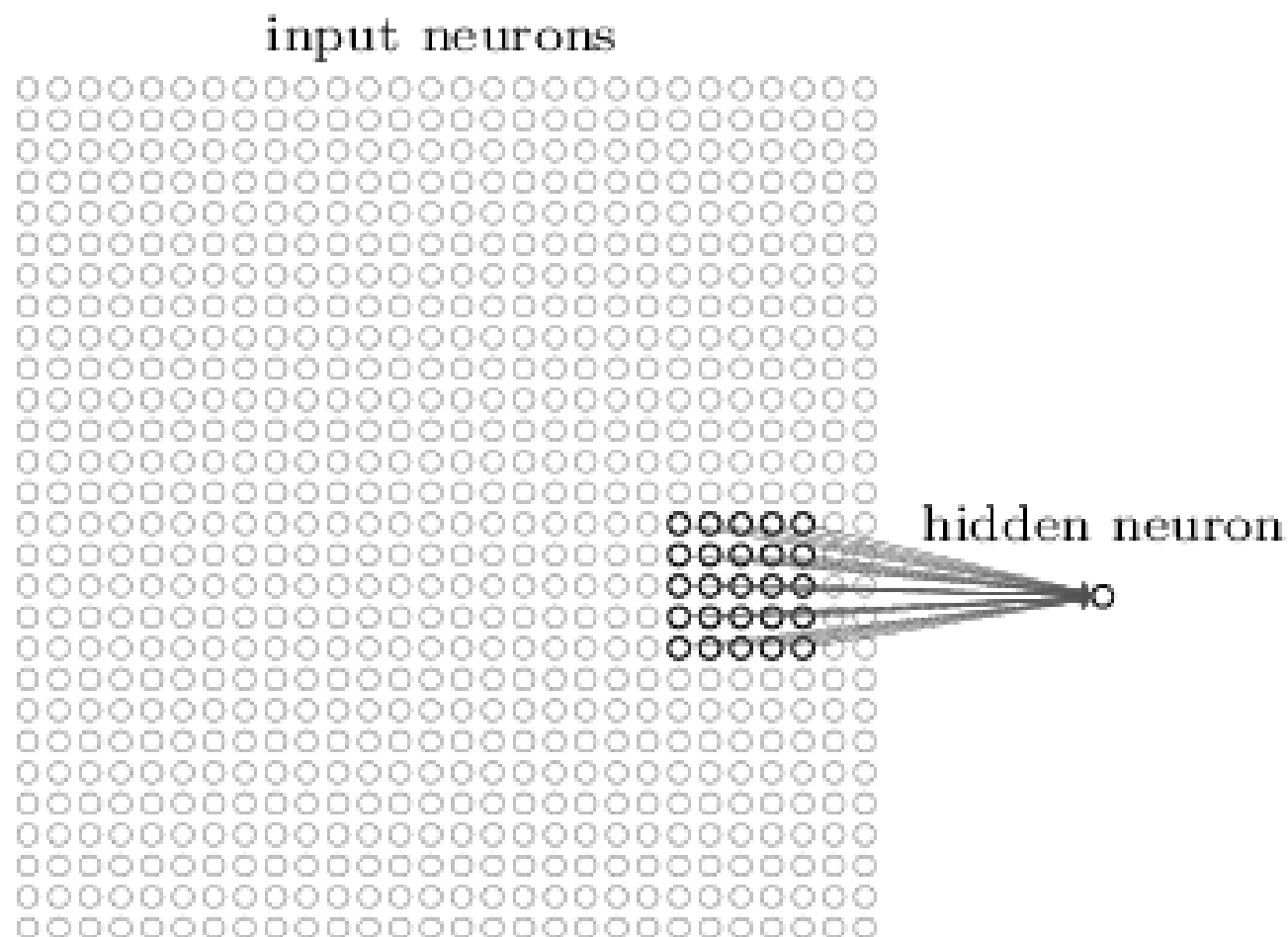


Figura 2 – Campo de receptores como entrada para filtros locais

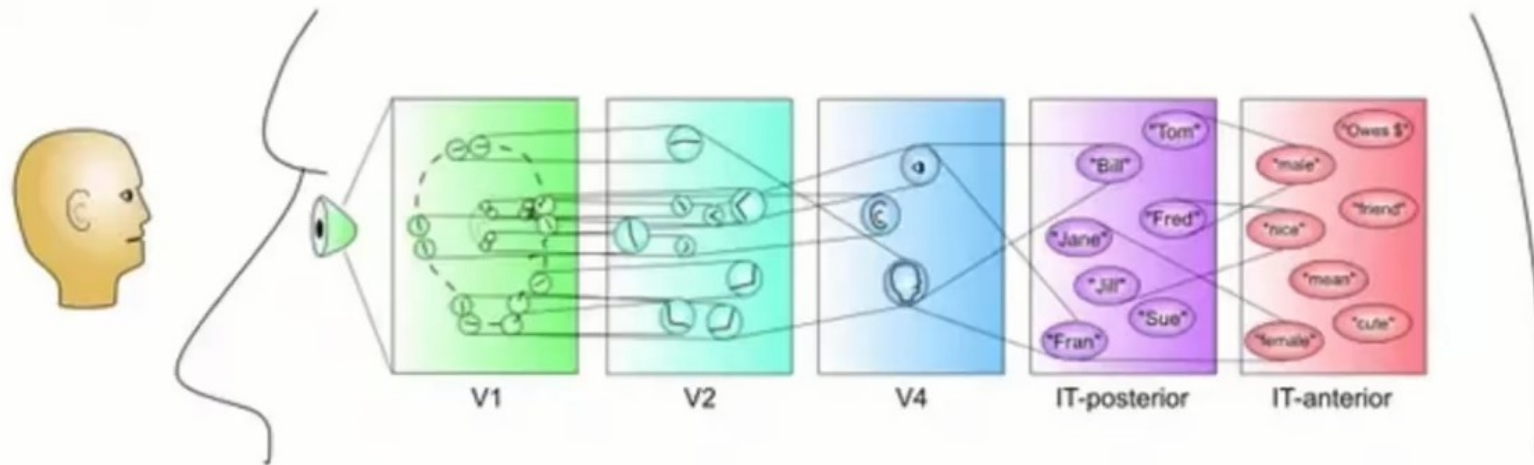


Figura 3 – Uma arquitetura em camadas que implementa uma hierarquia de campos de receptores, caracterizados por grupos de neurônios que reagem a conceitos específicos

- A inspiração levou a modelos de visão computacional caracterizados por:
 - ✓ Conexões locais;
 - ✓ Estrutura em camadas;
 - ✓ Operadores com algum grau de invariância espacial, ou seja, invariância à posição, ao tamanho, ao contraste, à rotação e à orientação.

- Mas tenha em mente que computadores e cérebros não “enxergam” as coisas do mesmo jeito.

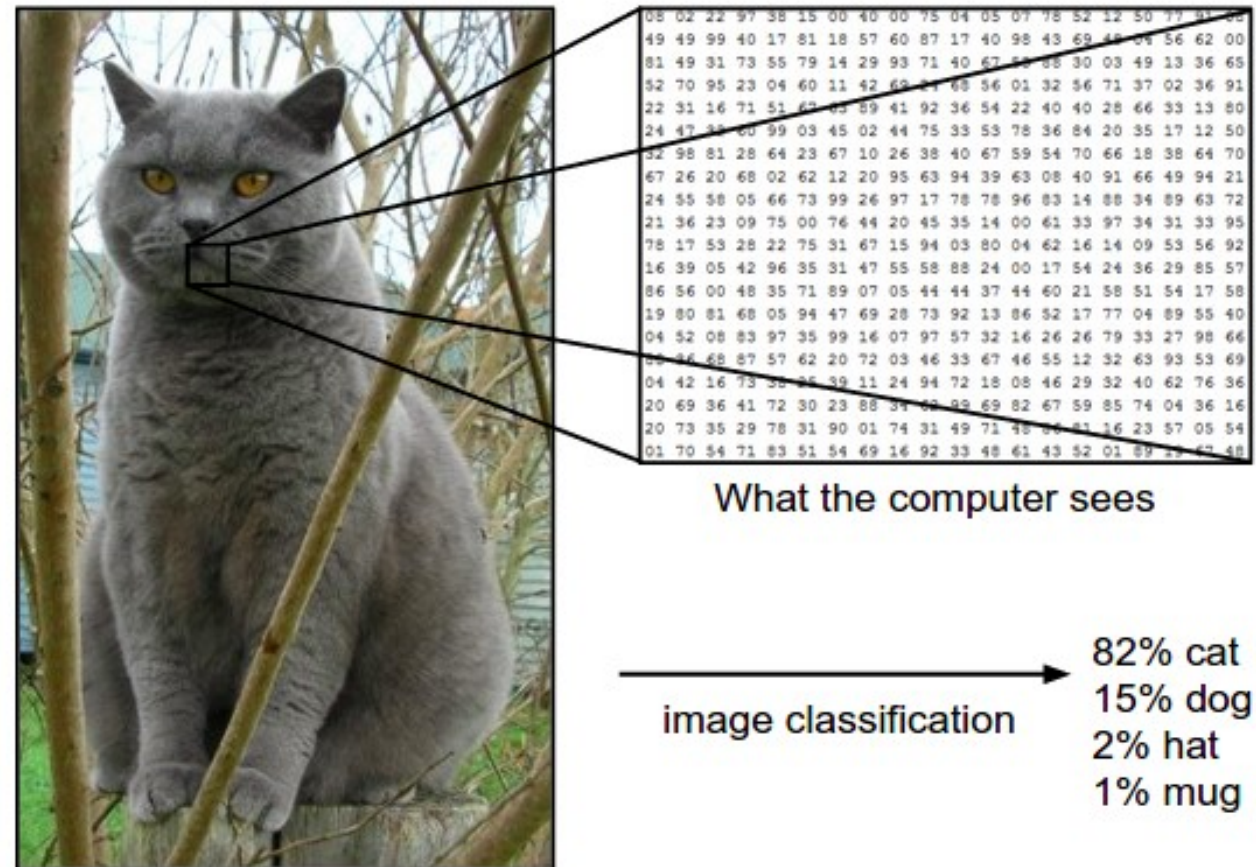
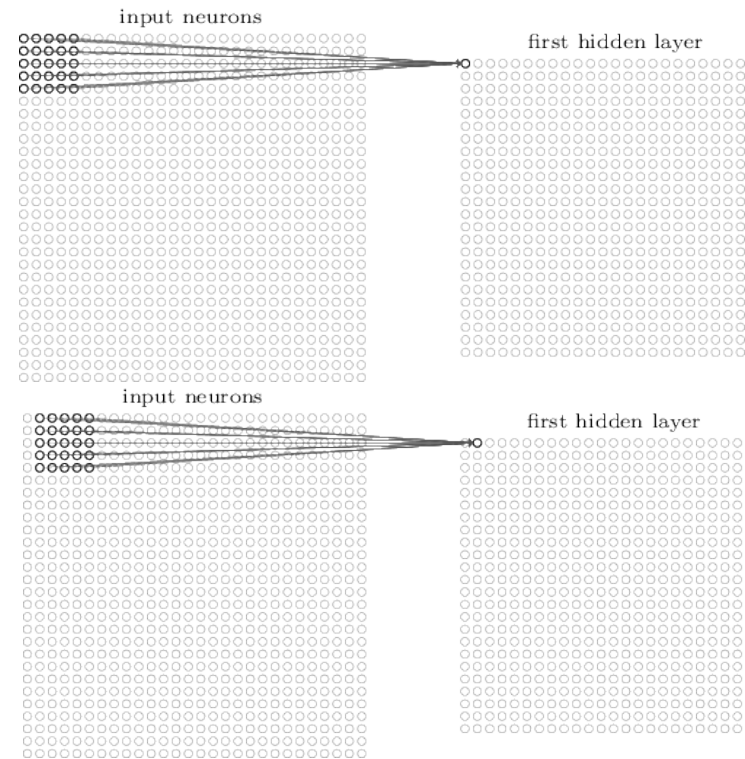


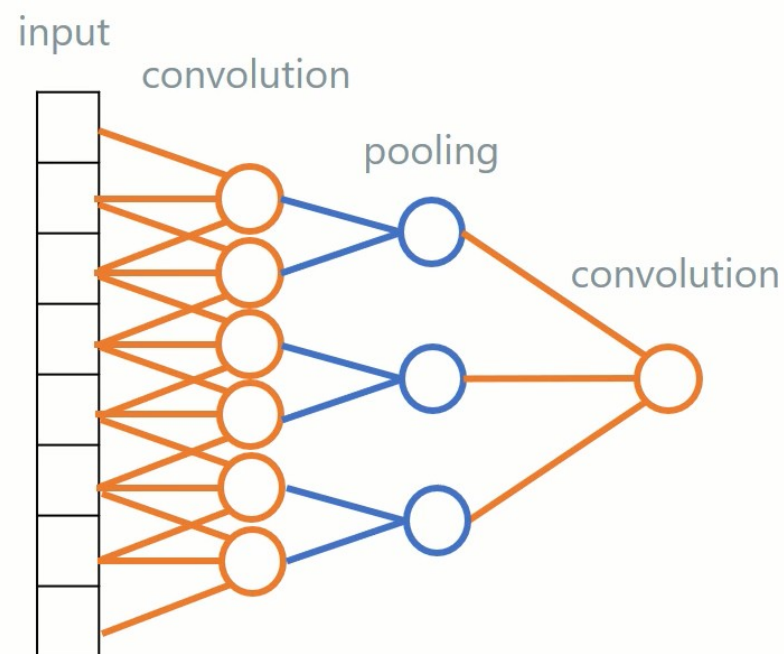
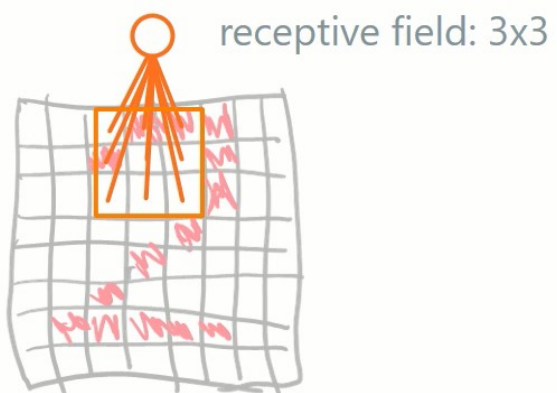
Figura 4 – Entrada e saída de um processo computacional para classificação de imagens

- Cada filtro local obtém uma saída a partir de *matching* com uma região do campo de receptores, passando o grau de *matching* por uma função não-linear.
- Imagens coloridas vão apresentar 3 campos de receptores (padrão RGB).
- É possível reposicionar cada filtro, varrendo o campo (os 3 campos quando há cor, também chamados de canais) de receptores segundo uma regra específica. O resultado desta operação de varredura é uma outra imagem, produzida pela ativação não-linear do filtro, ao varrer todo o campo de receptores.
- O passo de varredura (*stride*) é um hiperparâmetro, assim como as dimensões do filtro, o número de filtros, a função de ativação e a presença ou não de *padding*.
- Para que o filtro sempre esteja centrado no pixel corrente, mesmo sendo este de borda ou canto, pode-se recorrer ao conceito de preenchimento (*padding*).
- Não há um procedimento sistemático para a especificação desses hiperparâmetros, mas uma sugestão é partir de configurações que já levaram a redes neurais de alto desempenho na literatura.

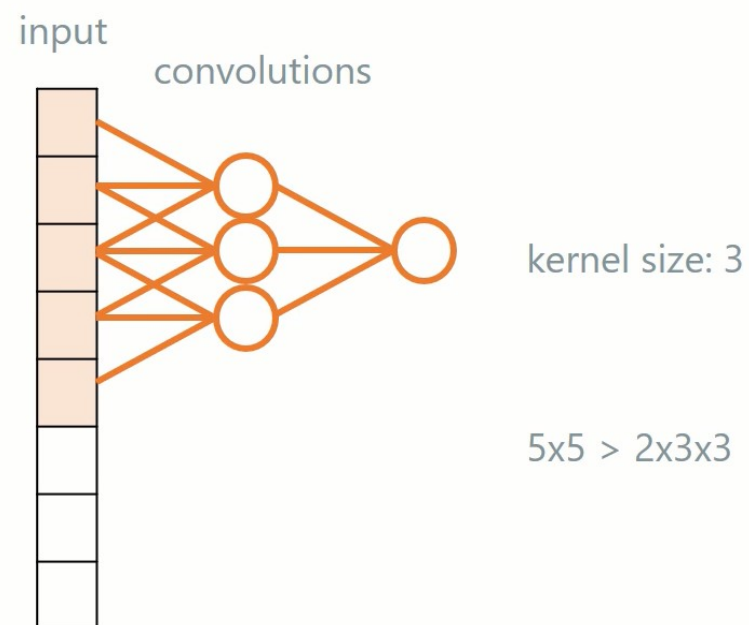
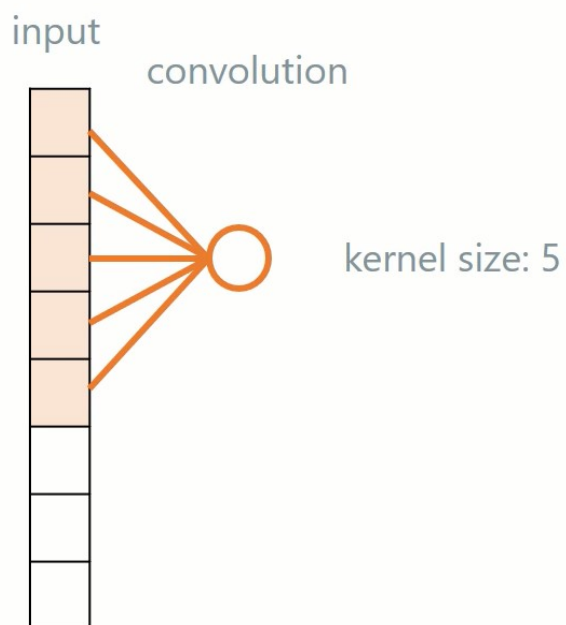


- Em termos fisiológicos, não é um mesmo e único filtro que varre o campo de receptores, mas sim um conjunto de filtros que operam em paralelo.
- Também é importante observar que o próprio campo de receptores está em permanente deslocamento, realizando uma varredura local da imagem que se encontra no campo de visão.

Receptive field

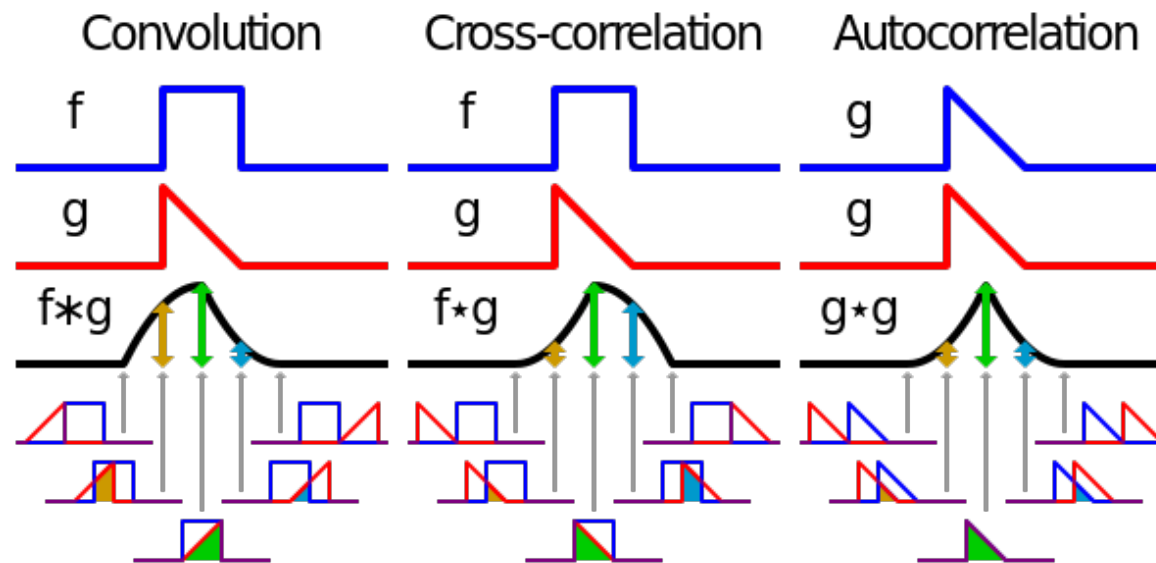


Kernel size



1.2 Operação de convolução

- Em aprendizado de máquina, as operações de convolução e de correlação cruzada são tomadas como sinônimos, embora elas só sejam equivalentes quando os sinais envolvidos são simétricos. O que se faz em camadas convolucionais de redes neurais profundas é correlação cruzada (*cross-correlation*). Em processamento digital de sinais, a correlação cruzada é uma medida de similaridade entre duas séries temporais, como uma função do atraso de uma série relativa à outra.



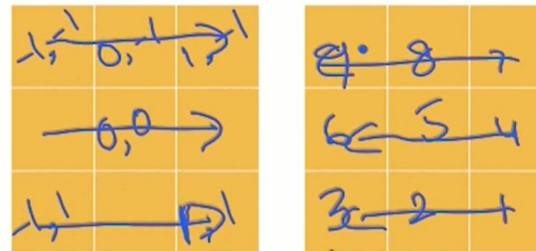
Convolution vs. Cross-Correlation

Cross-Correlation:

$$G = h \otimes F$$

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] F[i + u, j + v]$$

a	b	c
d	e	f
g	h	i



Convolution:

$$G = h * F$$

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] F[i - u, j - v]$$

- A convolução é uma operação comutativa ($h * F = F * h$), enquanto a correlação cruzada não ($h \otimes F \neq F \otimes h$), a menos que os sinais sejam simétricos.
- De qualquer modo, vamos empregar o termo convolução a partir de agora, embora a operação realizada seja, de fato, uma correlação cruzada.

$$a * b$$

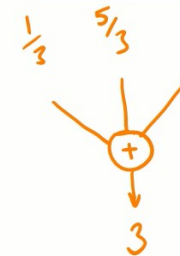
$$a = [1, 5, 3, 7, 5, 9, 7, 14]$$

$$b = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$$

$$a * b = [3]$$

$$a = [1, 5, 3, 7, 5, 9, 7, 14]$$

$$b = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$$



$$a * b = [3, 5]$$

$$a = [1, 5, 3, 7, 5, 9, 7, 14]$$

$$b = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$$

$$5 \times \frac{1}{3} + 3 \times \frac{1}{3} + 7 \times \frac{1}{3} = 5$$

$$a * b = [3, 5, 5, 7, 7, 10]$$

$$a = [1, 5, 3, 7, 5, 9, 7, 14]$$

$$b = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$$

- O filtro também pode ser chamado de *kernel*. No exemplo considerado, o papel do filtro é calcular médias locais do sinal, produzindo uma versão suavizada na saída.

Convolution

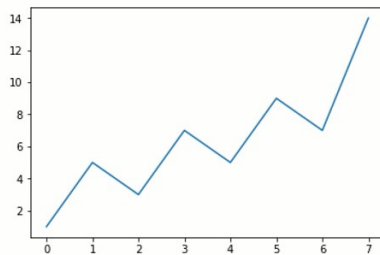
$$a * b = [3, 5, 5, 7, 7, 10]$$

Signal

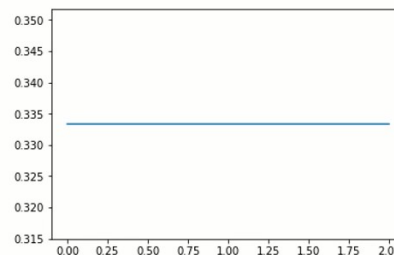
$$a = [1, 5, 3, 7, 5, 9, 7, 14]$$

Kernel (Filter)

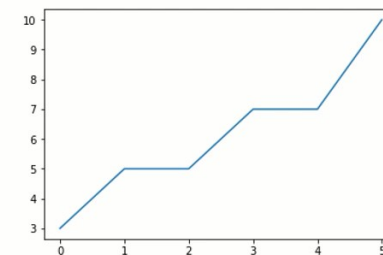
$$b = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$$



*



=



- O que faz o filtro a seguir, numa operação de convolução?

Convolution

Image

0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1

Filter

-1	0	1
-2	0	2
-1	0	1

0					

Convolution

Image

0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1

Filter

-1	0	1
-2	0	2
-1	0	1

0	0	4	4	0	0
0	0	4	4	0	0
0	0	4			

Convolution

Image

0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1

Filter

-1	0	1
-2	0	2
-1	0	1

0	0	4	4	0	0
0	0	4	4	0	0
0	0	4	4	0	0
0	0	4	4	0	0
0	0	4	4	0	0
0	0	4	4	0	0

Convolution

Image



Filter

-1	0	1
-2	0	2
-1	0	1

Convolution

Image



Filter

-1	0	1
-2	0	2
-1	0	1



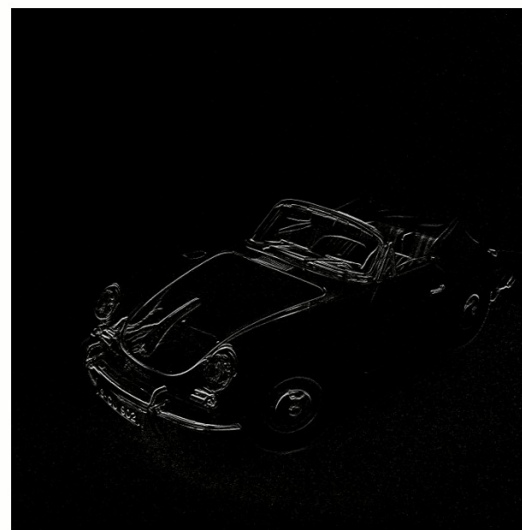
Convolution

Image



Filter

-1	-2	-1
0	0	0
1	2	1



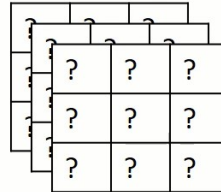
- Para imagens coloridas, pode-se definir um campo de filtro para cada canal (R, G e B), embora haja variações que serão investigadas mais adiante.
- Repare que, em qualquer camada, cada matriz é chamada de canal ou mapa de ativação.

Convolution

Image



Filter



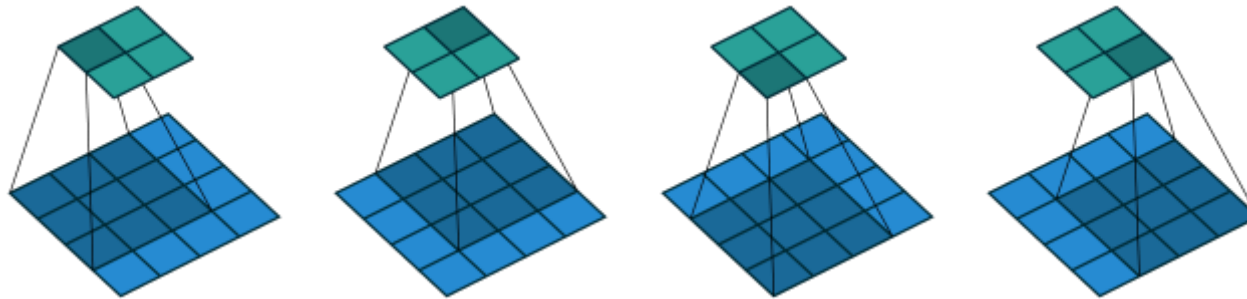


Figura 5 – Campo de receptores (em azul) sem contar com preenchimento (*padding*) e operação do filtro com $stride = 1$, formando um novo campo de receptores (em verde)

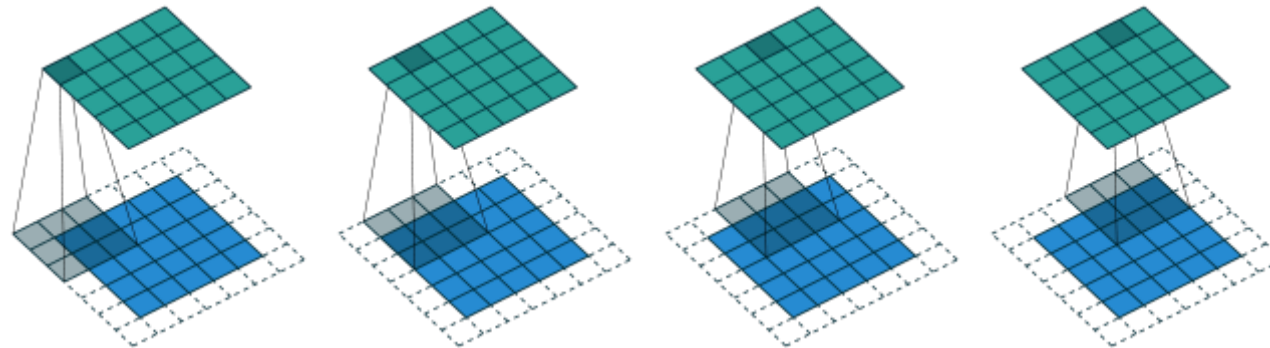


Figura 6 – Mesmo cenário da Figura 5, mas agora contando com preenchimento de 1 (*padding*, em branco)

Fonte das Figuras 5 a 8: < <https://arxiv.org/abs/1603.07285> >

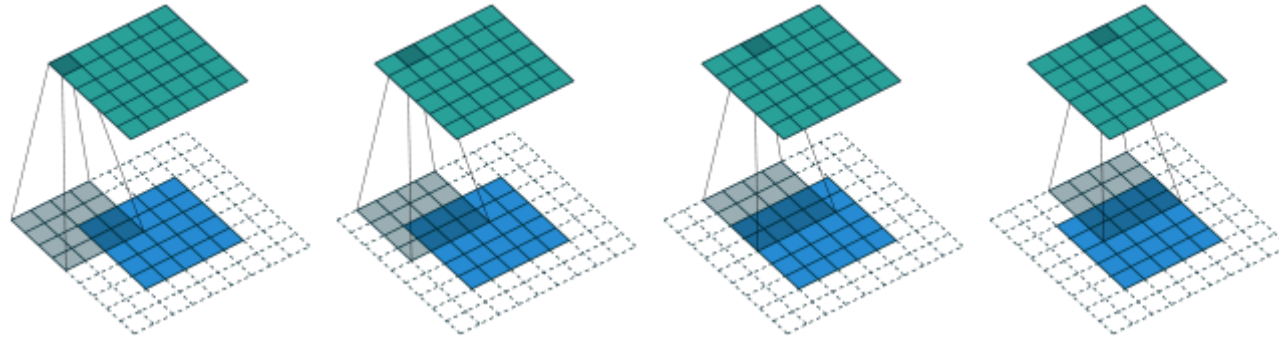


Figura 7 – Mesmo cenário da Figura 5, mas agora contando com preenchimento de 2 (*padding*, em branco)

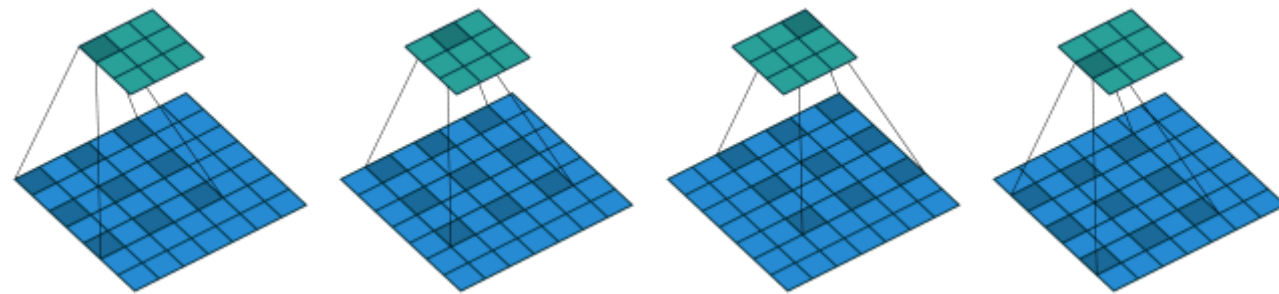


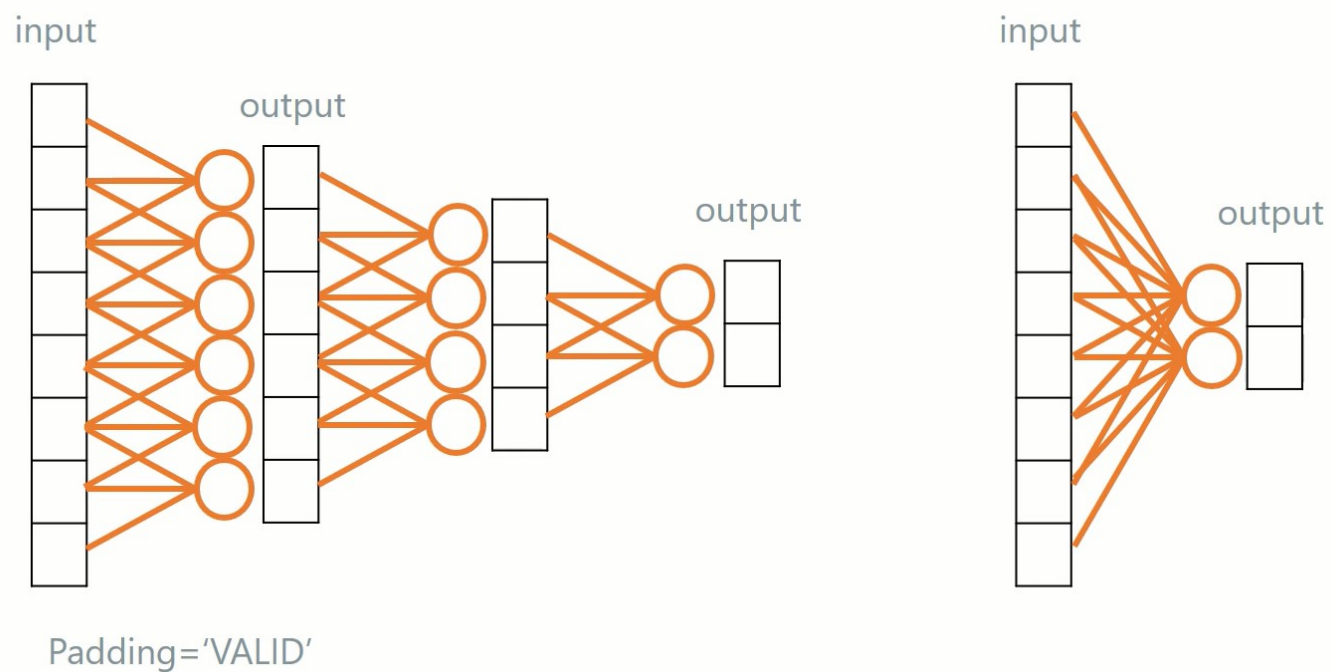
Figura 8 – Campo de receptores (em azul) e filtro numa operação de convolução dilatada (*dilated convolution*)

- A convolução dilatada pode ser usada como uma alternativa às operações de *pooling*.

- Segue um exemplo numérico (num cenário de pixels de entrada e pesos binários) da aplicação de convolução com $stride = 1$ e sem *padding*.

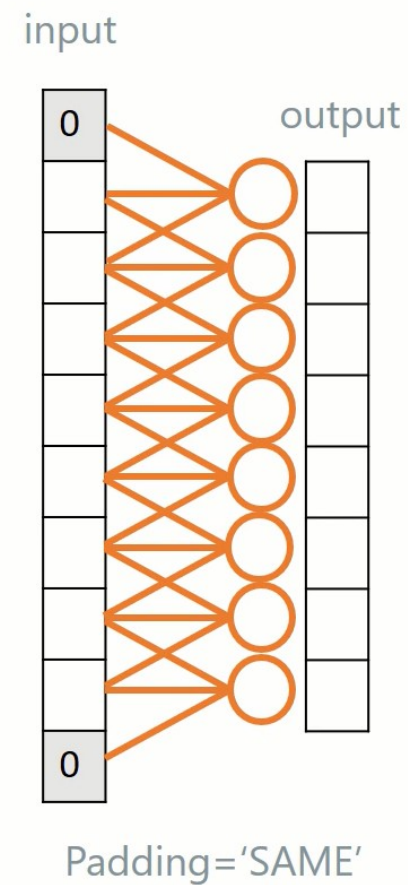
<table><tr><td>1x1</td><td>1x0</td><td>1x1</td><td>0</td><td>0</td></tr><tr><td>0x0</td><td>1x1</td><td>1x0</td><td>1</td><td>0</td></tr><tr><td>0x1</td><td>0x0</td><td>1x1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1x1	1x0	1x1	0	0	0x0	1x1	1x0	1	0	0x1	0x0	1x1	1	1	0	0	1	1	0	0	1	1	0	0	<table><tr><td>4</td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	4									<table><tr><td>1</td><td>1x1</td><td>1x0</td><td>0x1</td><td>0</td></tr><tr><td>0</td><td>1x0</td><td>1x1</td><td>1x0</td><td>0</td></tr><tr><td>0</td><td>0x1</td><td>1x0</td><td>1x1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1x1	1x0	0x1	0	0	1x0	1x1	1x0	0	0	0x1	1x0	1x1	1	0	0	1	1	0	0	1	1	0	0	<table><tr><td>4</td><td>3</td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	4	3								<table><tr><td>1</td><td>1</td><td>1x1</td><td>0x0</td><td>0x1</td></tr><tr><td>0</td><td>1</td><td>1x0</td><td>1x1</td><td>0x0</td></tr><tr><td>0</td><td>0</td><td>1x1</td><td>1x0</td><td>1x1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	1x1	0x0	0x1	0	1	1x0	1x1	0x0	0	0	1x1	1x0	1x1	0	0	1	1	0	0	1	1	0	0	<table><tr><td>4</td><td>3</td><td>4</td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	4	3	4						
1x1	1x0	1x1	0	0																																																																																																							
0x0	1x1	1x0	1	0																																																																																																							
0x1	0x0	1x1	1	1																																																																																																							
0	0	1	1	0																																																																																																							
0	1	1	0	0																																																																																																							
4																																																																																																											
1	1x1	1x0	0x1	0																																																																																																							
0	1x0	1x1	1x0	0																																																																																																							
0	0x1	1x0	1x1	1																																																																																																							
0	0	1	1	0																																																																																																							
0	1	1	0	0																																																																																																							
4	3																																																																																																										
1	1	1x1	0x0	0x1																																																																																																							
0	1	1x0	1x1	0x0																																																																																																							
0	0	1x1	1x0	1x1																																																																																																							
0	0	1	1	0																																																																																																							
0	1	1	0	0																																																																																																							
4	3	4																																																																																																									
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0x1</td><td>1x0</td><td>1x1</td><td>1</td><td>0</td></tr><tr><td>0x0</td><td>0x1</td><td>1x0</td><td>1</td><td>1</td></tr><tr><td>0x1</td><td>0x0</td><td>1x1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	1	0	0	0x1	1x0	1x1	1	0	0x0	0x1	1x0	1	1	0x1	0x0	1x1	1	0	0	1	1	0	0	<table><tr><td>4</td><td>3</td><td>4</td></tr><tr><td>2</td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	4	3	4	2						<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1x1</td><td>1x0</td><td>1x1</td><td>0</td></tr><tr><td>0</td><td>0x0</td><td>1x1</td><td>1x0</td><td>1</td></tr><tr><td>0</td><td>0x1</td><td>1x0</td><td>1x1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	1	0	0	0	1x1	1x0	1x1	0	0	0x0	1x1	1x0	1	0	0x1	1x0	1x1	0	0	1	1	0	0	<table><tr><td>4</td><td>3</td><td>4</td></tr><tr><td>2</td><td>4</td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	4	3	4	2	4					<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1x1</td><td>1x0</td><td>0x1</td></tr><tr><td>0</td><td>0</td><td>1x0</td><td>1x1</td><td>1x0</td></tr><tr><td>0</td><td>0</td><td>1x1</td><td>1x0</td><td>0x1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	1	0	0	0	1	1x1	1x0	0x1	0	0	1x0	1x1	1x0	0	0	1x1	1x0	0x1	0	1	1	0	0	<table><tr><td>4</td><td>3</td><td>4</td></tr><tr><td>2</td><td>4</td><td>3</td></tr><tr><td></td><td></td><td></td></tr></table>	4	3	4	2	4	3			
1	1	1	0	0																																																																																																							
0x1	1x0	1x1	1	0																																																																																																							
0x0	0x1	1x0	1	1																																																																																																							
0x1	0x0	1x1	1	0																																																																																																							
0	1	1	0	0																																																																																																							
4	3	4																																																																																																									
2																																																																																																											
1	1	1	0	0																																																																																																							
0	1x1	1x0	1x1	0																																																																																																							
0	0x0	1x1	1x0	1																																																																																																							
0	0x1	1x0	1x1	0																																																																																																							
0	1	1	0	0																																																																																																							
4	3	4																																																																																																									
2	4																																																																																																										
1	1	1	0	0																																																																																																							
0	1	1x1	1x0	0x1																																																																																																							
0	0	1x0	1x1	1x0																																																																																																							
0	0	1x1	1x0	0x1																																																																																																							
0	1	1	0	0																																																																																																							
4	3	4																																																																																																									
2	4	3																																																																																																									
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0x1</td><td>0x0</td><td>1x1</td><td>1</td><td>1</td></tr><tr><td>0x0</td><td>0x1</td><td>1x0</td><td>1</td><td>0</td></tr><tr><td>0x1</td><td>1x0</td><td>1x1</td><td>0</td><td>0</td></tr></table>	1	1	1	0	0	0	1	1	1	0	0x1	0x0	1x1	1	1	0x0	0x1	1x0	1	0	0x1	1x0	1x1	0	0	<table><tr><td>4</td><td>3</td><td>4</td></tr><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td></td><td></td></tr></table>	4	3	4	2	4	3	2			<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0x1</td><td>1x0</td><td>1x1</td><td>1</td></tr><tr><td>0</td><td>0x0</td><td>1x1</td><td>1x0</td><td>0</td></tr><tr><td>0</td><td>1x1</td><td>1x0</td><td>0x1</td><td>0</td></tr></table>	1	1	1	0	0	0	1	1	1	0	0	0x1	1x0	1x1	1	0	0x0	1x1	1x0	0	0	1x1	1x0	0x1	0	<table><tr><td>4</td><td>3</td><td>4</td></tr><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td></td></tr></table>	4	3	4	2	4	3	2	3		<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1x1</td><td>1x0</td><td>1x1</td></tr><tr><td>0</td><td>0</td><td>1x0</td><td>1x1</td><td>0x0</td></tr><tr><td>0</td><td>1</td><td>1x1</td><td>0x0</td><td>0x1</td></tr></table>	1	1	1	0	0	0	1	1	1	0	0	0	1x1	1x0	1x1	0	0	1x0	1x1	0x0	0	1	1x1	0x0	0x1	<table><tr><td>4</td><td>3</td><td>4</td></tr><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>4</td></tr></table>	4	3	4	2	4	3	2	3	4
1	1	1	0	0																																																																																																							
0	1	1	1	0																																																																																																							
0x1	0x0	1x1	1	1																																																																																																							
0x0	0x1	1x0	1	0																																																																																																							
0x1	1x0	1x1	0	0																																																																																																							
4	3	4																																																																																																									
2	4	3																																																																																																									
2																																																																																																											
1	1	1	0	0																																																																																																							
0	1	1	1	0																																																																																																							
0	0x1	1x0	1x1	1																																																																																																							
0	0x0	1x1	1x0	0																																																																																																							
0	1x1	1x0	0x1	0																																																																																																							
4	3	4																																																																																																									
2	4	3																																																																																																									
2	3																																																																																																										
1	1	1	0	0																																																																																																							
0	1	1	1	0																																																																																																							
0	0	1x1	1x0	1x1																																																																																																							
0	0	1x0	1x1	0x0																																																																																																							
0	1	1x1	0x0	0x1																																																																																																							
4	3	4																																																																																																									
2	4	3																																																																																																									
2	3	4																																																																																																									

Padding

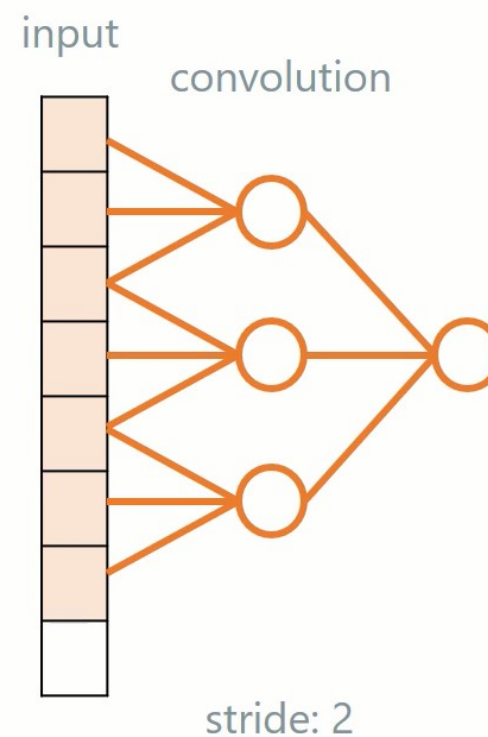
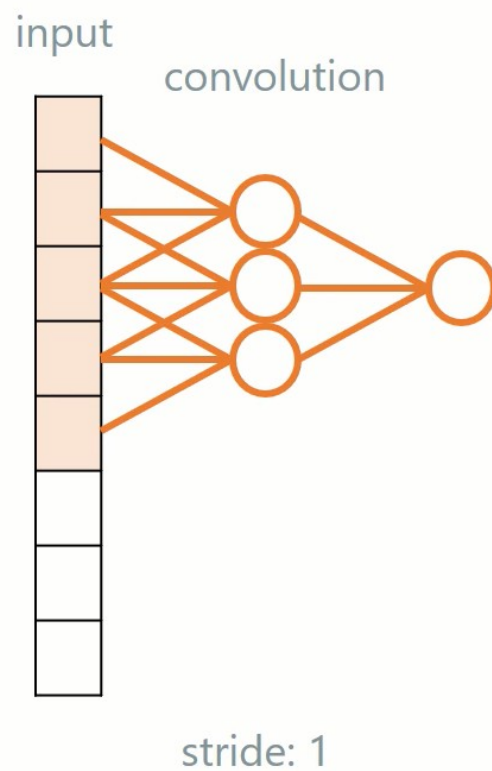


Padding

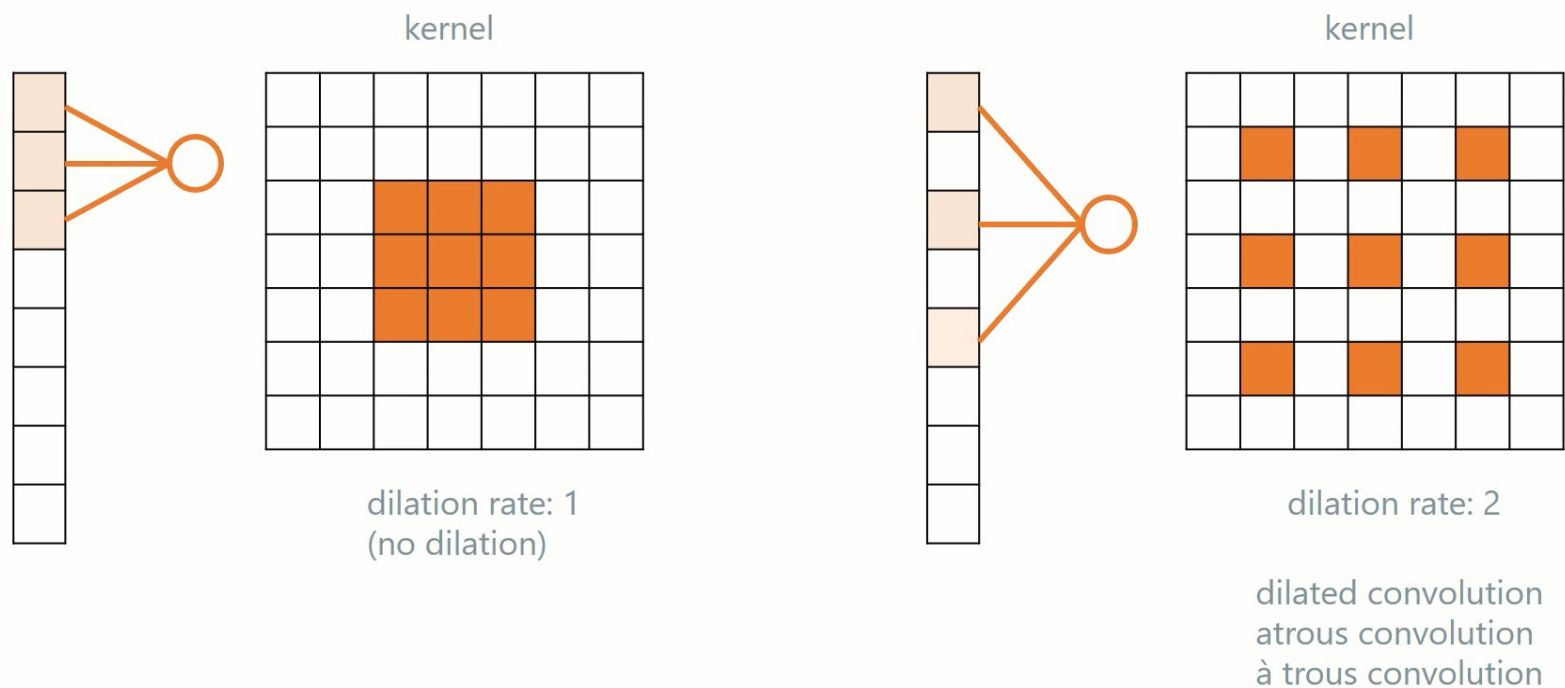
0	0	0	0	0	0	0	0	0	0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0	0	0	0	0	0	0	0	0	0



Stride



Dilation rate



1.3 Filtros extratores de atributos

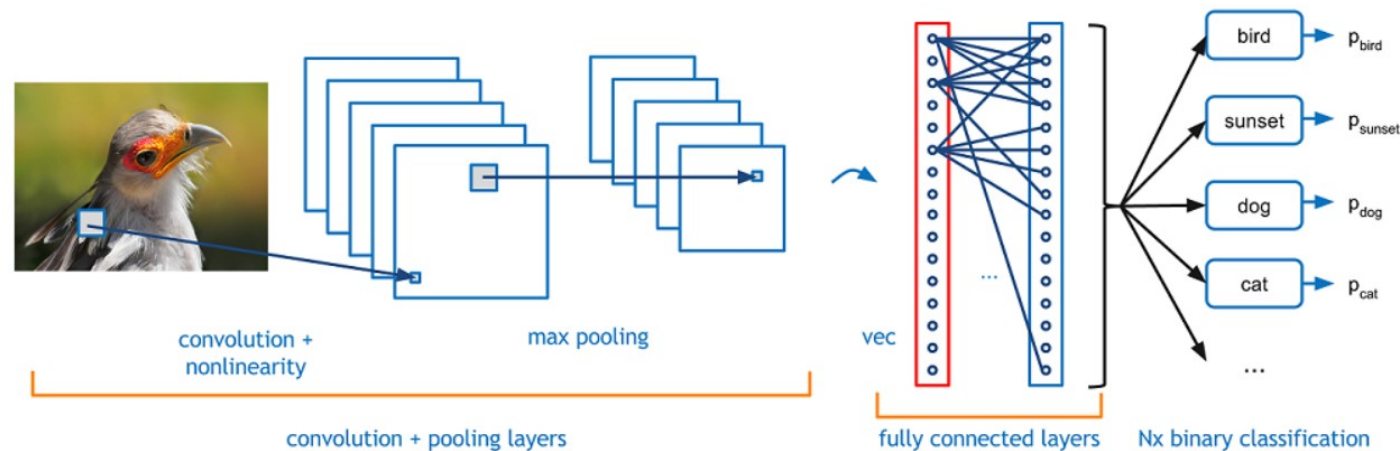


Figura 9 – Rede neural convolucional que recebe a imagem bruta em formato de pixels.

- Existe um padrão bem característico: múltiplas camadas convolucionais, que operam como uma cascata de filtros não-lineares extratores de atributos relevantes da imagem de entrada, seguidas por camadas *fully connected*, que sintetizam o mapeamento linear ou não-linear dos [atributos disponíveis no final da cascata de filtros] para um [vetor de probabilidades], indicando, assim, a probabilidade da imagem de entrada pertencer a cada uma das classes do problema de classificação.

- Apesar de vigorar um certo padrão na construção de redes neurais convolucionais, as possibilidades de configuração da topologia da rede como um todo ainda são enormes. Partir de topologias que já conduziram a bons resultados em aplicações afins e/ou ir inserindo módulos adicionais aos poucos são recomendações válidas.
- Todas as camadas (exceto, possivelmente, a última) realizam operações não-lineares, e o ajuste dos pesos sinápticos deve ocorrer por meio de técnicas de otimização não-linear, empregando a retropropagação do erro.
- Como já enfatizado em outros momentos do curso, o objetivo do treinamento é explicitamente minimizar a função-custo (função-perda) na saída da rede neural. Entretanto, para conquistar esta redução de custo (da perda), a rede acaba, indiretamente, por conceber uma cascata de filtros competentes e coerentes entre si, capazes de extrair automaticamente atributos relevantes dos dados de entrada que compõem o conjunto de treinamento.

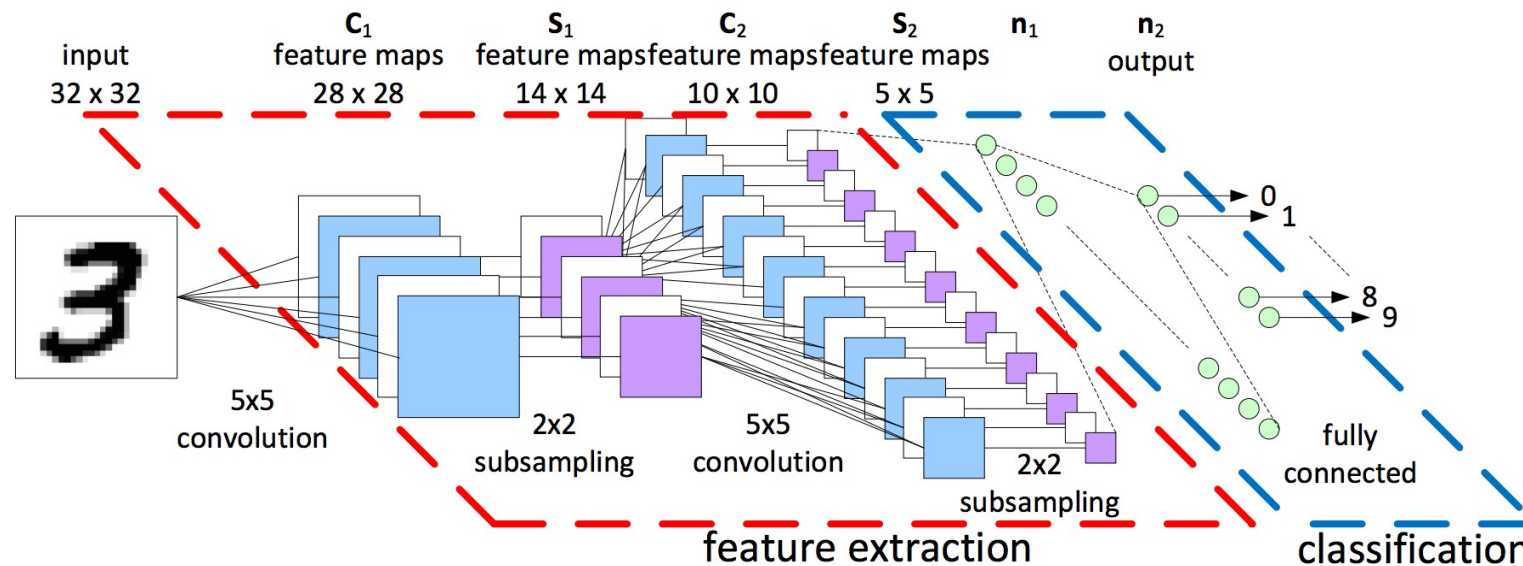
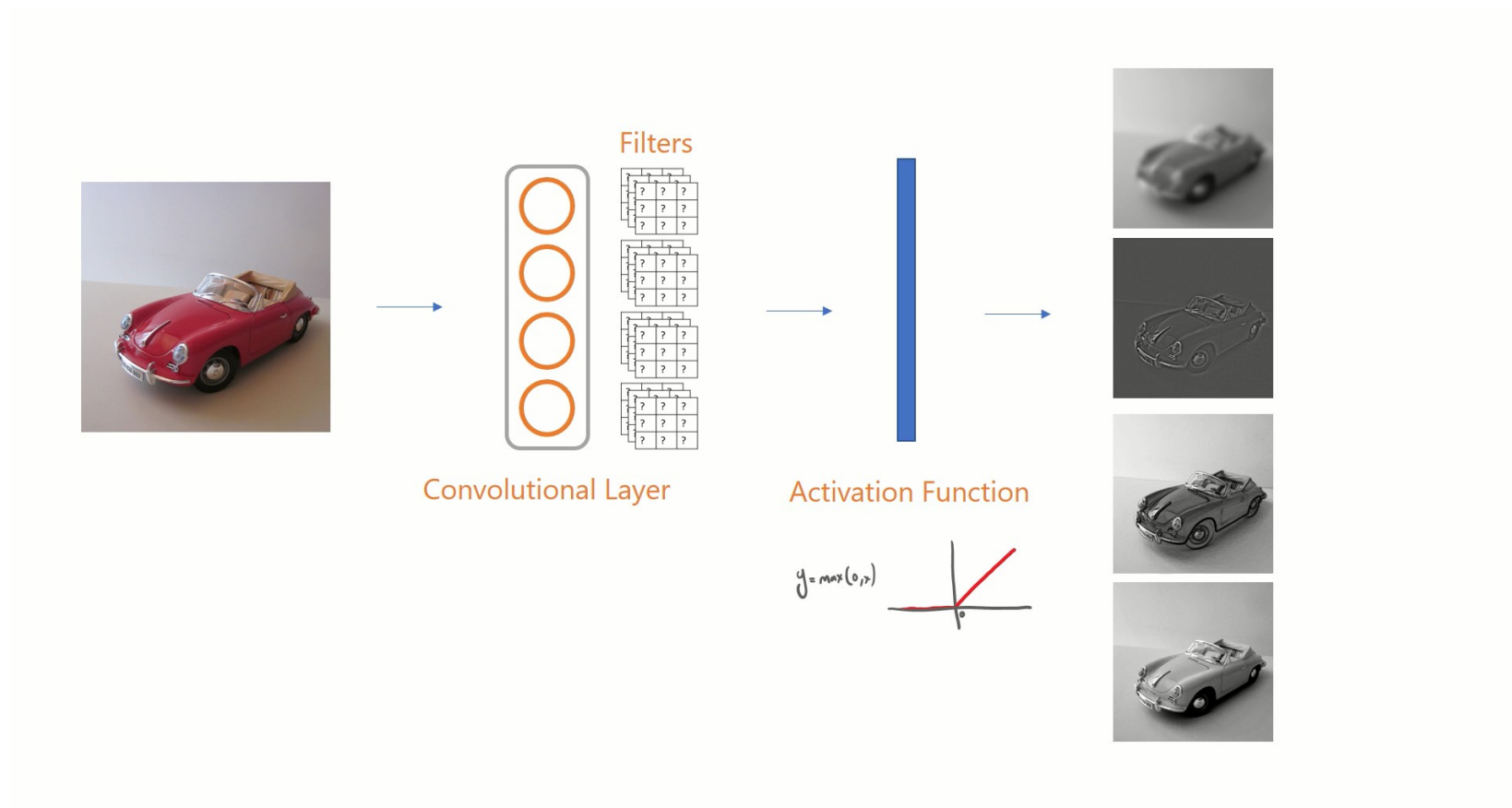
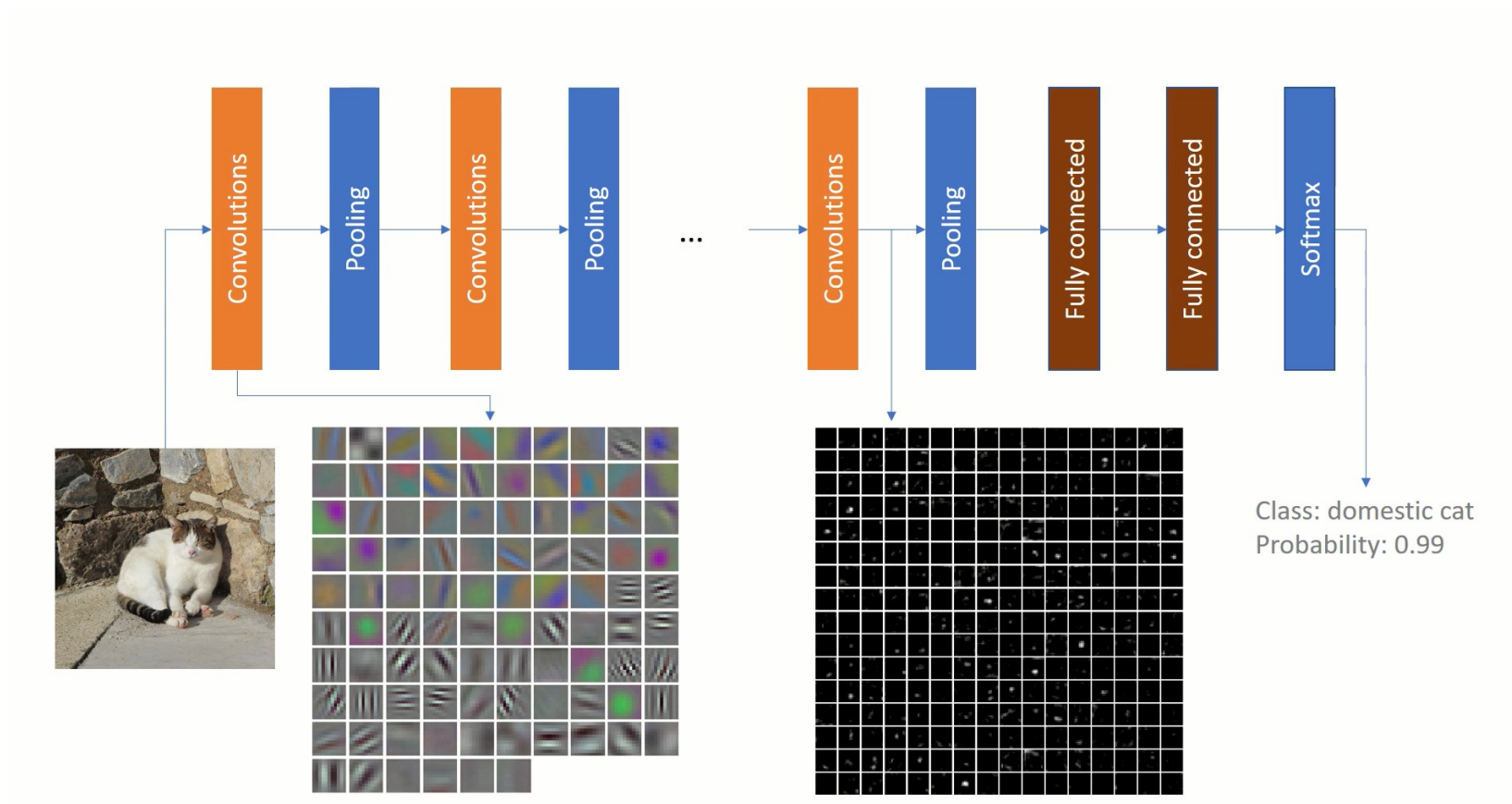


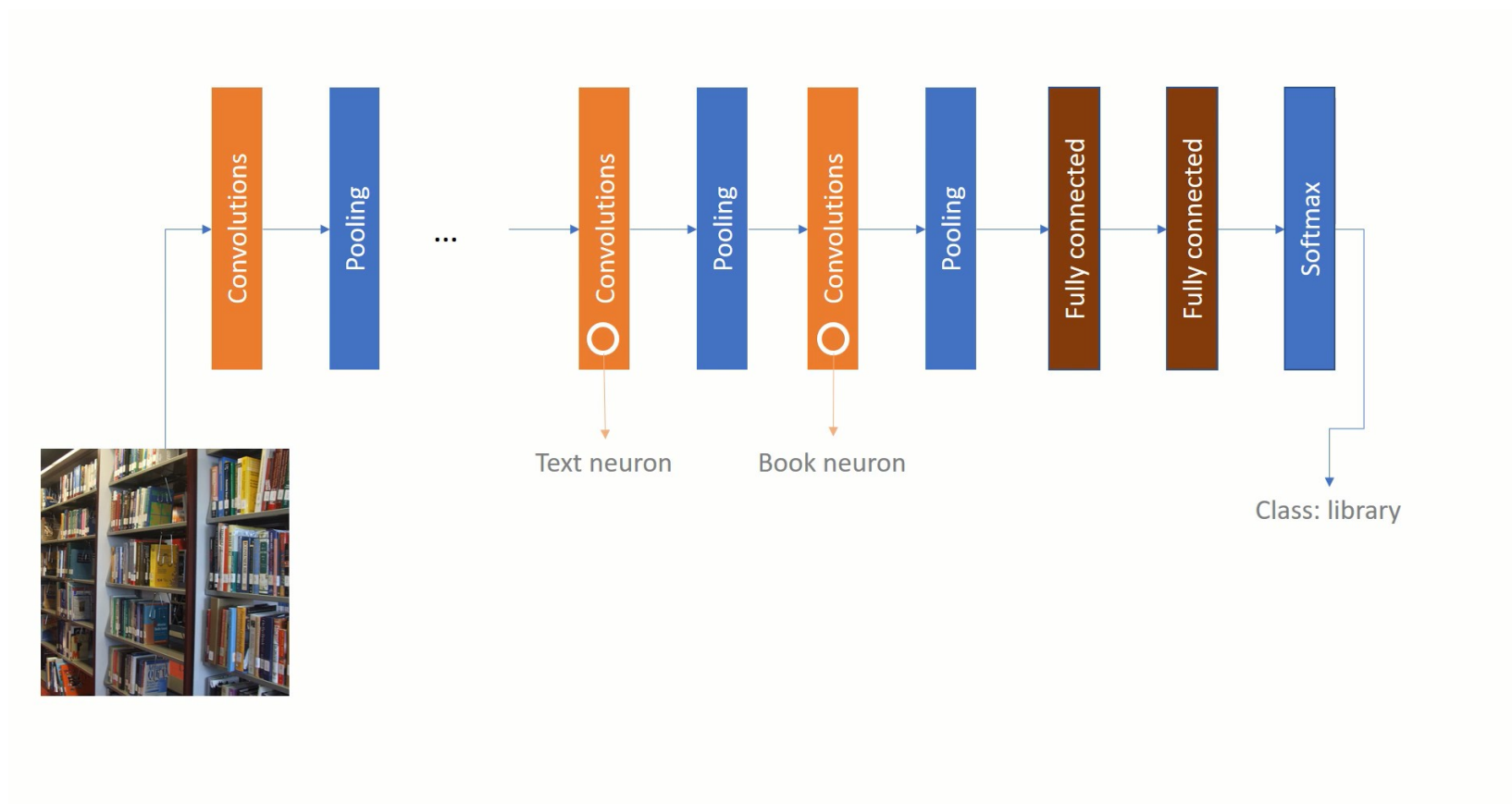
Figura 10 – Exemplo de arquitetura de rede convolucional para classificar imagens de dígitos manuscritos, com resolução inicial 32×32 , nas dez classes possíveis

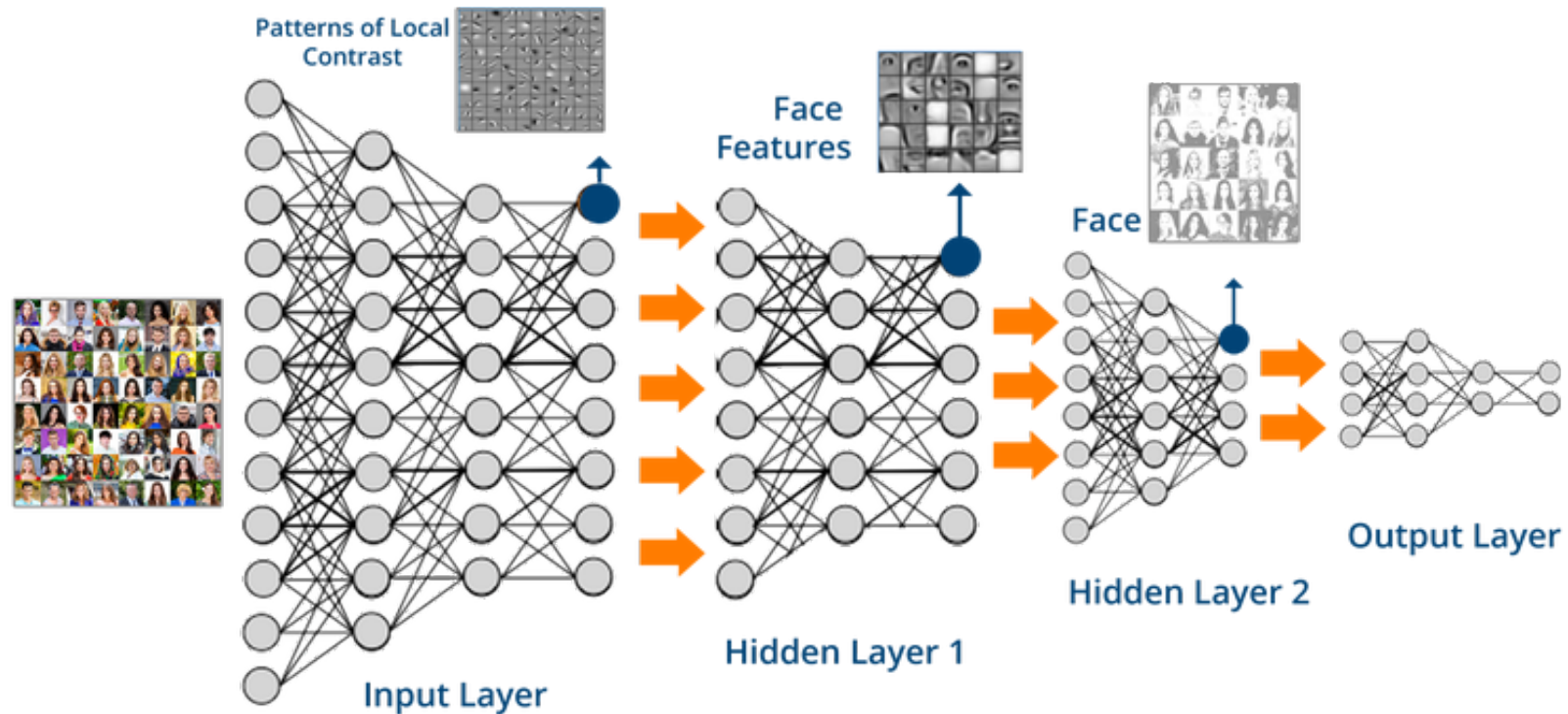
- As “fichas” em paralelo nas camadas convolucionais da Figura 10 representam o resultado da operação de convolução que os filtros convolucionais fazem nas imagens da camada anterior. A mesma imagem pode ser convoluída com múltiplos filtros, gerando múltiplas “fichas” na camada seguinte. Essas “fichas” são chamadas de **mapas de ativação ou canais**, um para cada filtro local. Foram definidos 6 filtros convolucionais/kernels para a primeira camada, com 1 campo cada, e 16 filtros convolucionais/kernels para a segunda camada, com 6 campos cada.

Fonte da Figura 10 acima: Peemen, M.; Mesman, B.; Corporaal, H. “*Speed sign detection and recognition by convolutional neural networks*”, Proceedings of the 8th International Automotive Congress (IAC’2011), pp. 162-170, 2011.



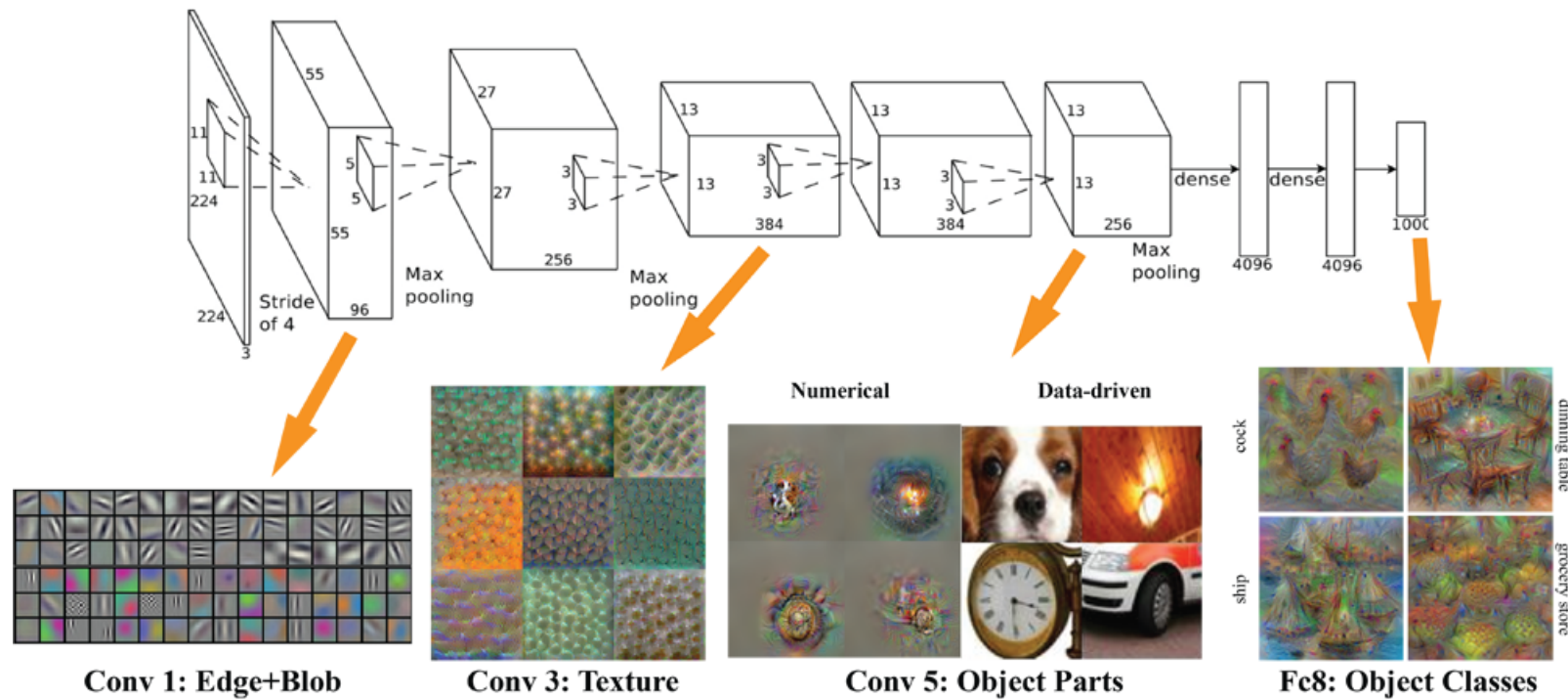






Fonte: < <https://medium.com/data-science/creating-a-movie-recommender-using-convolutional-neural-networks-be93e66464a7> >

ZEILER, M.D. & FERGUS, R. “Visualizing and Understanding Convolutional Networks”, arXiv:1311.2901v3, 2013.



Fontes: < <https://arxiv.org/abs/2009.13395> e <https://link.springer.com/article/10.1007/s12652-020-02236-z>>

$$96*(11*11*3+1)+256*(5*5*96+1)+384*(3*3*256+1)+384*(3*3*384+1)+256*(3*3*384+1)+4096*(6*6*256+1)+4096*(4096+1)+1000*(4096+1) = 62.378.344 \text{ parâmetros ajustáveis}$$

- Observação: Esses números são influenciados pelos valores de padding, stride e pela operação de max pooling, os quais não foram todos apresentados. Nas camadas convolucionais, o número de pesos não depende da resolução da imagem de entrada.

- O número de operações de multiplicação depende da resolução da imagem de entrada. As camadas convolucionais são capazes de processar imagens de qualquer resolução.
- O número de pesos total só vai depender da resolução da imagem de entrada na hora de obter os pesos da primeira camada densa.

Programa em Python que implementa a rede convolucional do slide anterior

```
import tensorflow as tf

model = tf.keras.models.Sequential()
model.add(tf.keras.Input(shape=(224, 224, 3)))
model.add(tf.keras.layers.Conv2D(96, kernel_size=(11, 11), activation='relu',
                                   padding='valid', strides=(4, 4)))
model.add(tf.keras.layers.Conv2D(256, kernel_size=(5, 5), activation='relu', padding='same'))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2)))
model.add(tf.keras.layers.Conv2D(384, kernel_size=(3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2)))
model.add(tf.keras.layers.Conv2D(384, kernel_size=(3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.Conv2D(256, kernel_size=(3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2)))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(4096, activation='relu'))
model.add(tf.keras.layers.Dense(4096, activation='relu'))
model.add(tf.keras.layers.Dense(1000, activation='softmax'))

model.summary()
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 54, 54, 96)	34,944
conv2d_1 (Conv2D)	(None, 54, 54, 256)	614,656
max_pooling2d (MaxPooling2D)	(None, 27, 27, 256)	0
conv2d_2 (Conv2D)	(None, 27, 27, 384)	885,120
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 384)	0
conv2d_3 (Conv2D)	(None, 13, 13, 384)	1,327,488
conv2d_4 (Conv2D)	(None, 13, 13, 256)	884,992
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 256)	0
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 4096)	37,752,832
dense_1 (Dense)	(None, 4096)	16,781,312
dense_2 (Dense)	(None, 1000)	4,097,000

Total params: 62,378,344 (237.95 MB)
Trainable params: 62,378,344 (237.95 MB)
Non-trainable params: 0 (0.00 B)

- É relevante ficar atento às camadas que mais “consomem” pesos sinápticos. Alterações pontuais na arquitetura podem resultar em uma expressiva redução no número de pesos sinápticos.

Common settings:

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters K ,
 - their spatial extent F ,
 - the stride S ,
 - the amount of zero padding P .
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
 - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

$K =$ (powers of 2, e.g. 32, 64, 128, 512)

- $F = 3, S = 1, P = 1$
- $F = 5, S = 1, P = 2$
- $F = 5, S = 2, P = ?$ (whatever fits)
- $F = 1, S = 1, P = 0$

- Para entradas 3D, é possível aplicar uma convolução 3D, como ilustrado na rede convolucional abaixo.

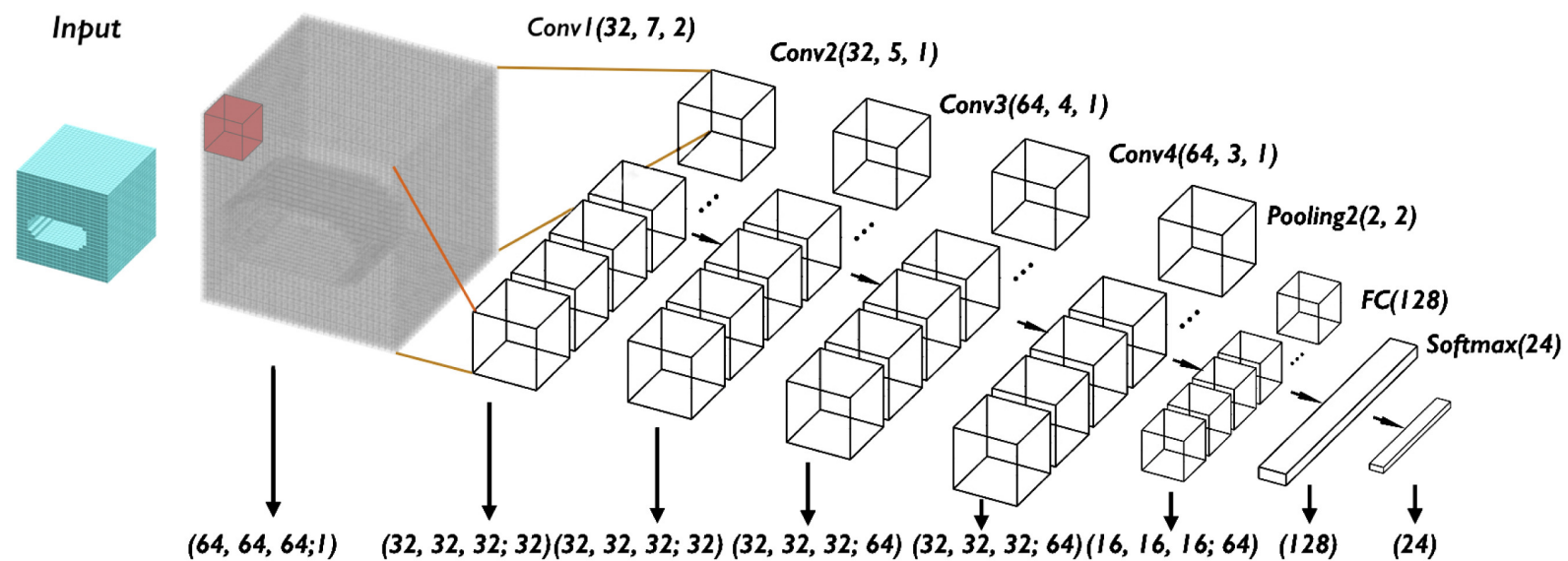
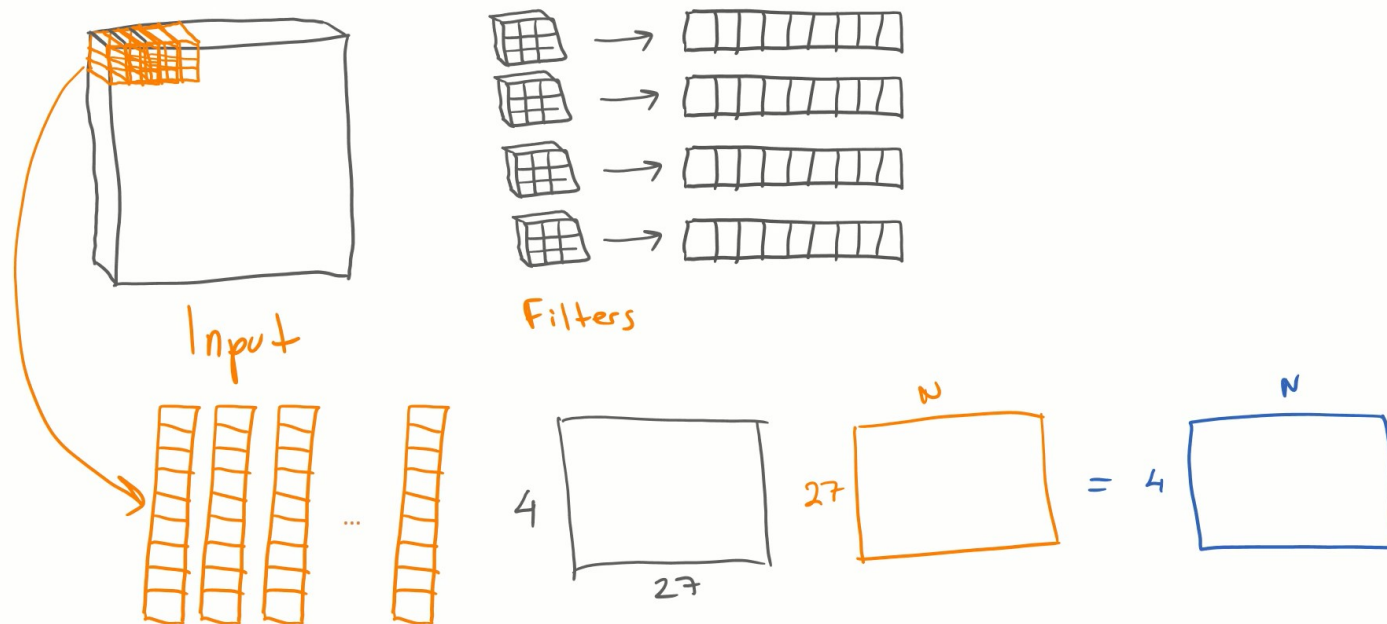


Figura extraída de: Zhang, Z.; Jaiswal, P.; Rai, R. “FeatureNet: Machining feature recognition based on 3D Convolution Neural Network”, Computer-Aided Design, vol. 101, pp. 12-22, 2018.

1.4 A operação de convolução na prática

- Voltando ao caso 2D, a convolução é implementada computacionalmente como uma operação matricial, após uma etapa de pré-processamento ilustrada a seguir.
- Confira também em: <https://arxiv.org/abs/1603.07285>

Implementation as a matrix multiplication

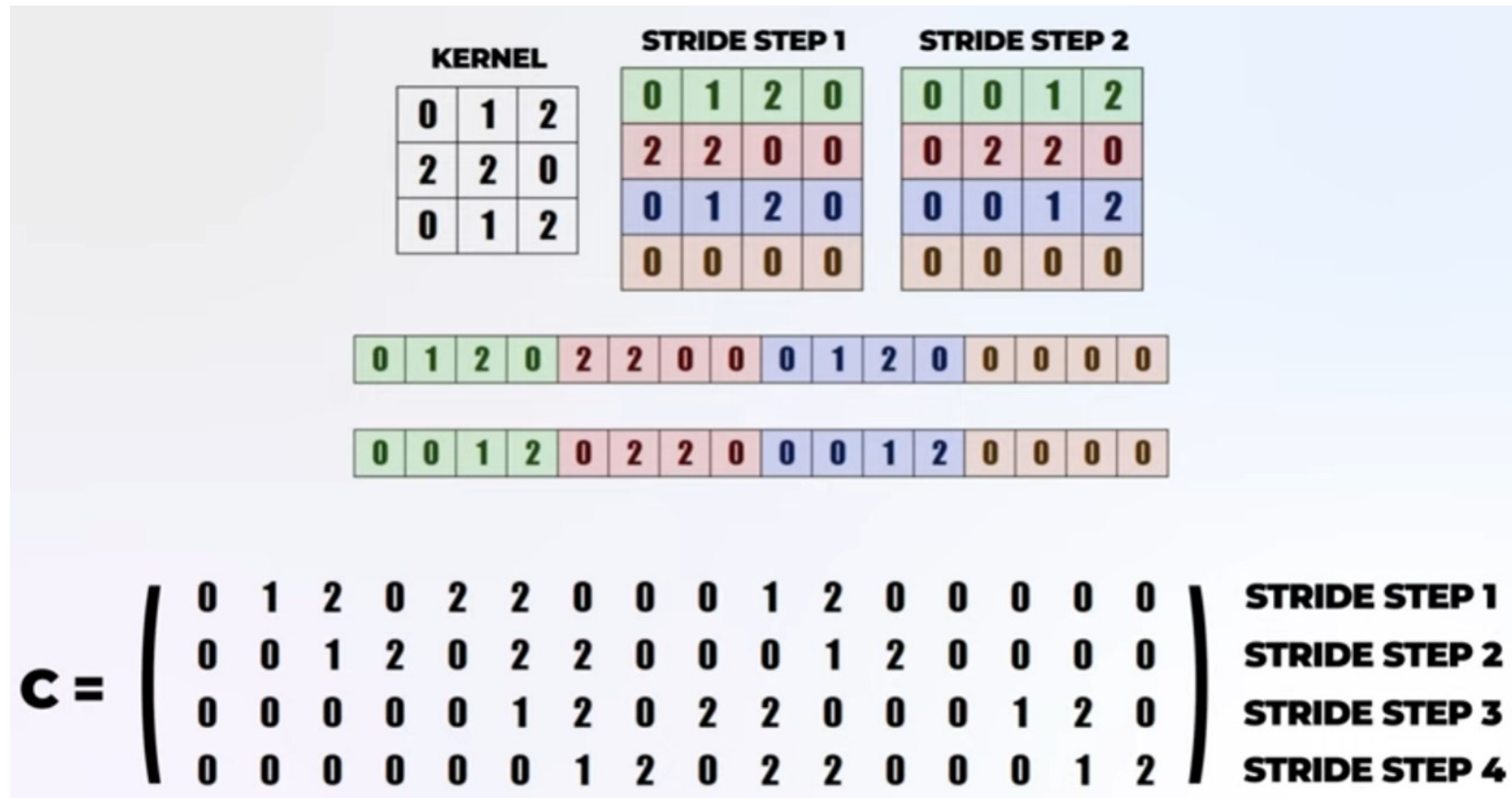


- Há variantes, como na formulação a seguir.
- As demais figuras desta seção foram extraídas de:

<https://www.youtube.com/watch?v=xoAv6D05j7g>

KERNEL			INPUT			
0	1	2	3	3	2	1
2	2	0	0	0	1	3
0	1	2	3	1	2	2
			2	0	0	2

KERNEL			STRIDE STEP 1				STRIDE STEP 2			
0	1	2	0	1	2			0	1	2
2	2	0	2	2	0			2	2	0
0	1	2	0	1	2			0	1	2



$$\mathbf{C} = \begin{pmatrix} 0 & 1 & 2 & 0 & 2 & 2 & 0 & 0 & 0 & 1 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 0 & 2 & 2 & 0 & 0 & 0 & 1 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 2 & 0 & 2 & 2 & 0 & 0 & 0 & 1 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 2 & 0 & 2 & 2 & 0 & 0 & 0 & 1 & 2 \end{pmatrix}$$

INPUT

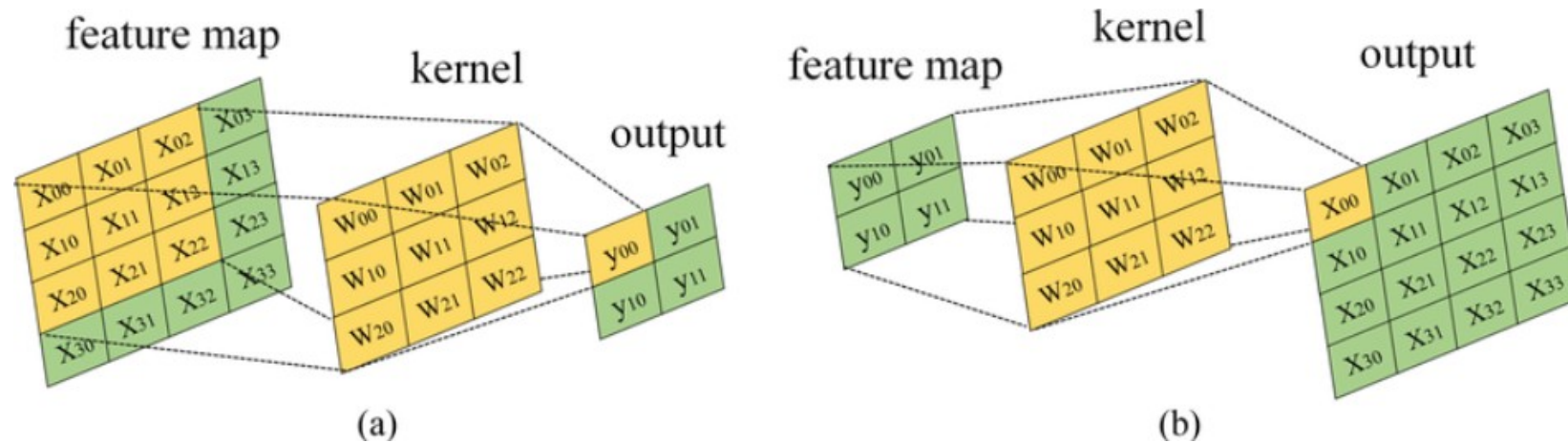
3	3	2	1
0	0	1	3
3	1	2	2
2	0	0	2

$$\mathbf{I} = \begin{pmatrix} 3 \\ 3 \\ 2 \\ 1 \\ 0 \\ 0 \\ 1 \\ 3 \\ 3 \\ 1 \\ 2 \\ 2 \\ 2 \\ 2 \\ 0 \\ 0 \\ 2 \end{pmatrix}$$

$$\mathbf{R}_{\text{CONV}} = \mathbf{C} \cdot \mathbf{I}$$
$$\mathbf{R}_{\text{CONV}} = \begin{pmatrix} 12 \\ 12 \\ 10 \\ 17 \end{pmatrix} \longrightarrow \begin{array}{|c|c|} \hline 12 & 12 \\ \hline 10 & 17 \\ \hline \end{array}$$

1.5 A operação de convolução transposta na prática

- Em cenários em que os mapas de ativação devem aumentar de resolução ao longo das camadas, recorre-se à operação de convolução transposta, também denominada *upsampling convolution*.

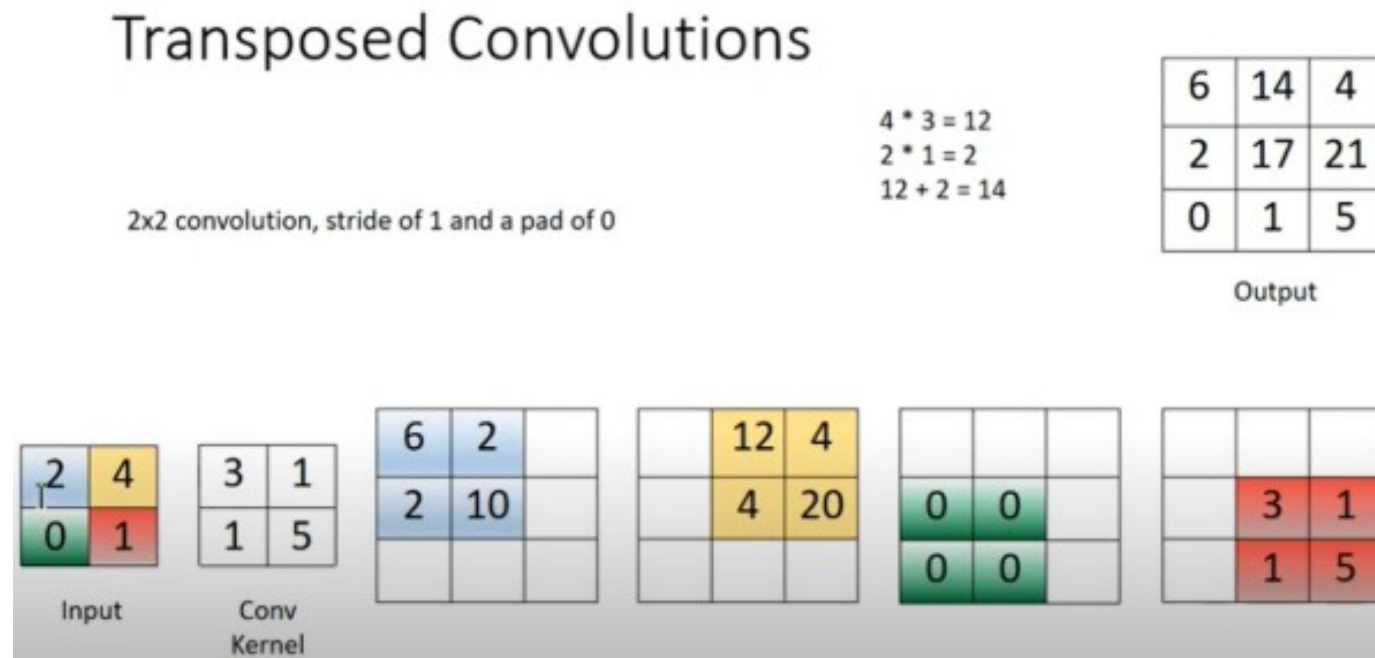


Convolução convencional

Convolução transposta

Fonte: <https://journals.aps.org/pre/abstract/10.1103/PhysRevE.100.033308>

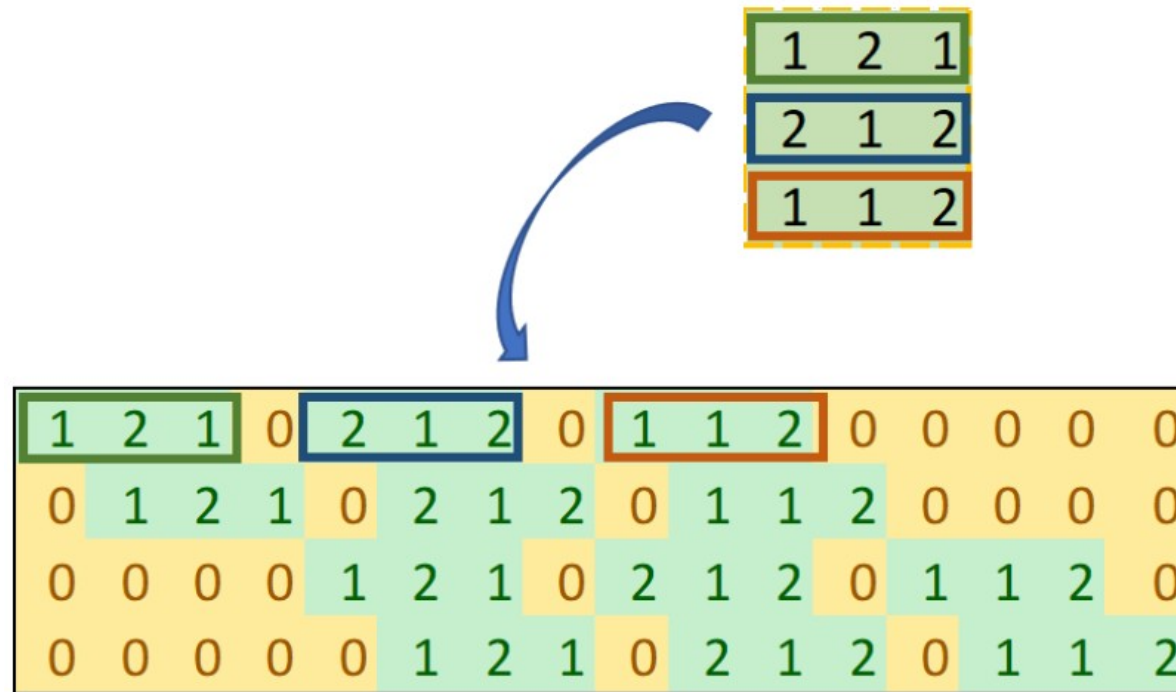
- Numa convolução convencional, se eu usar um kernel 2×2 para convoluir uma imagem de entrada 3×3 , eu obtenho um mapa de ativação 2×2 na saída. Como chegar a um mapa de ativação 3×3 , partindo de um kernel 2×2 que vai convoluir uma imagem de entrada 2×2 ?



Fonte: <https://stats.stackexchange.com/questions/587634/how-to-calculate-the-transposed-convolution>

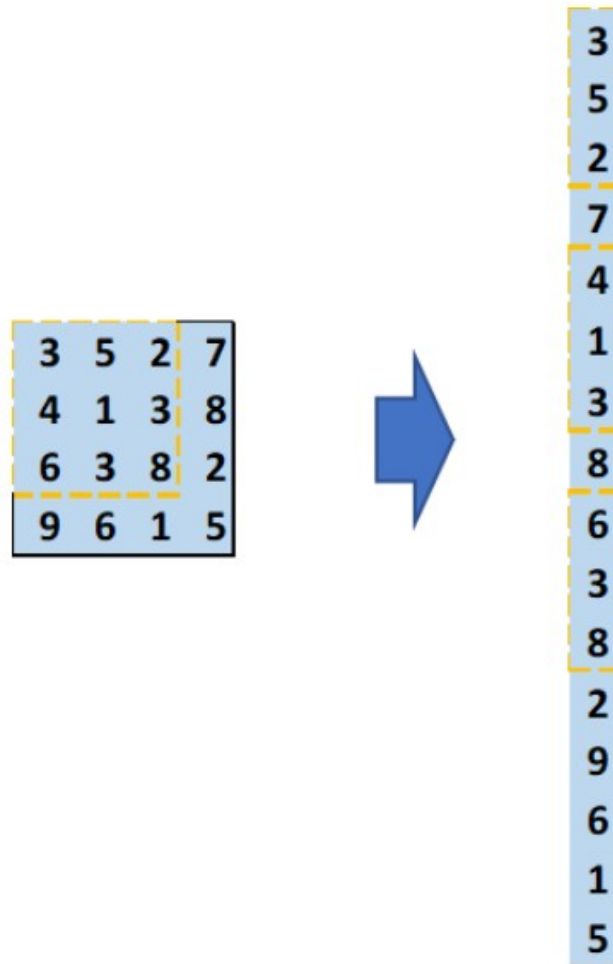
- Por que convolução transposta? As figuras a seguir foram extraídas de:

Fonte: <https://towardsdatascience.com/understand-transposed-convolutions-and-build-your-own-transposed-convolution-layer-from-scratch-4f5d97b2967/>

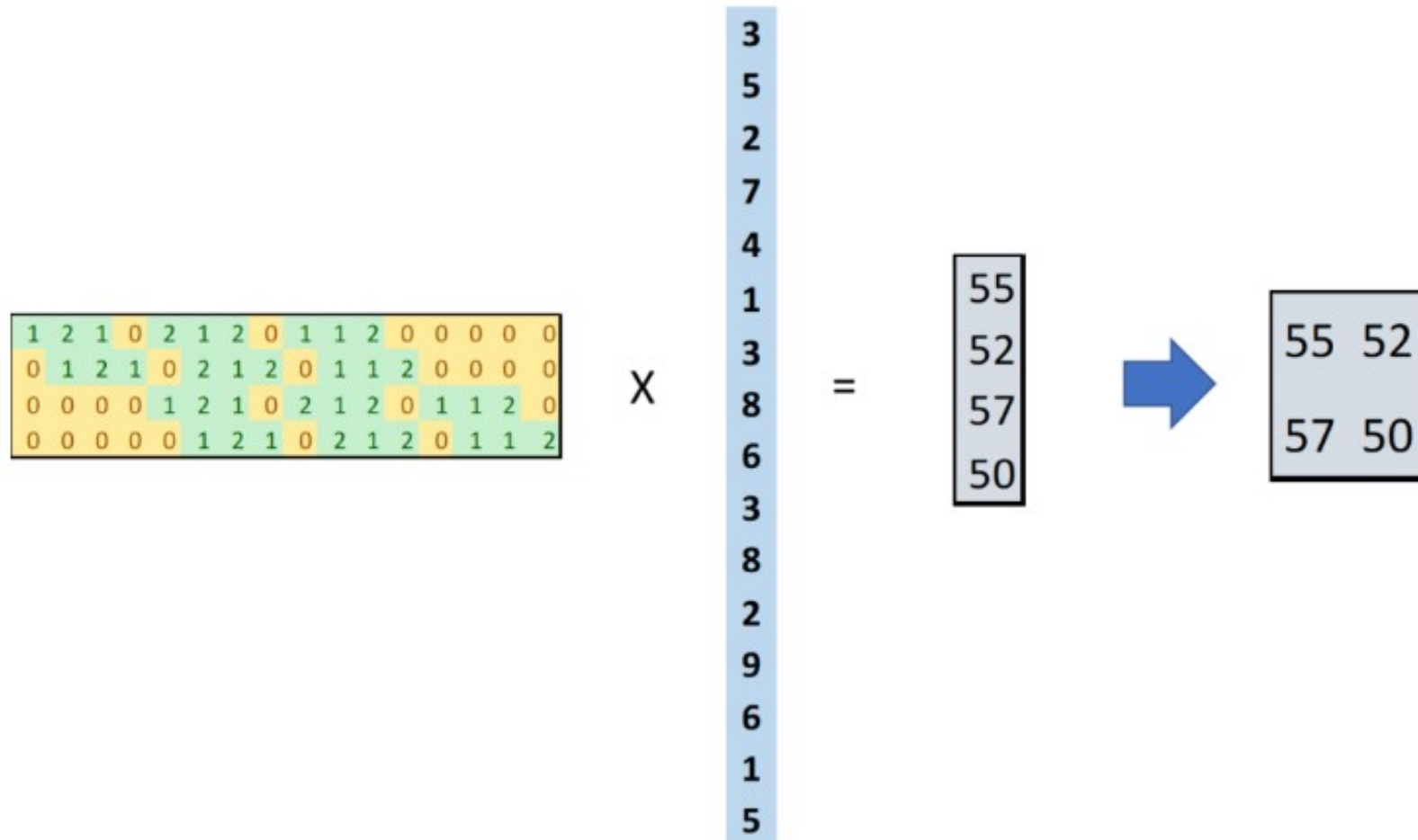


- Considere o mesmo esquema para montar a matriz de convolução, dispondo de um kernel 3×3 e tendo uma imagem de entrada 4×4 , de forma equivalente ao que foi feito nos slides 42 e 43.

- Vamos aplicar inicialmente a convolução convencional, já apresentada.
- Empilham-se as linhas da imagem de entrada.



- O resultado da convolução convencional vai ser o produto da matriz de convolução pela imagem empilhada.



- Agora, se eu transpuser a matriz de convolução (o que implica o uso do mesmo kernel) e considerar que a saída 2×2 obtida acima é a imagem de entrada, e que a nova saída a ser obtida deve ter a dimensão da entrada anterior (4×4), obtém-se:

The diagram illustrates a 2D convolution operation. A 16x16 input matrix is multiplied by a 4x4 kernel (55, 52, 57, 50). The result is a 16x16 output matrix, calculated as the sum of four 16x16 matrices, each being the input matrix multiplied by one of the kernel elements. The final output matrix is shown as a 4x4 grid of values.

Input Matrix (16x16):

1	0	0	0
2	1	0	0
1	2	0	0
0	1	0	0
2	0	1	0
1	2	2	1
2	1	1	2
0	2	0	1
1	0	2	0
1	1	1	2
2	1	2	1
0	2	0	2
0	0	1	0
0	0	1	1
0	0	2	1
0	0	0	2

Kernel (4x4):

55	52	57	50
----	----	----	----

Resulting Matrices (16x16 each):

Matrix 1 (55):

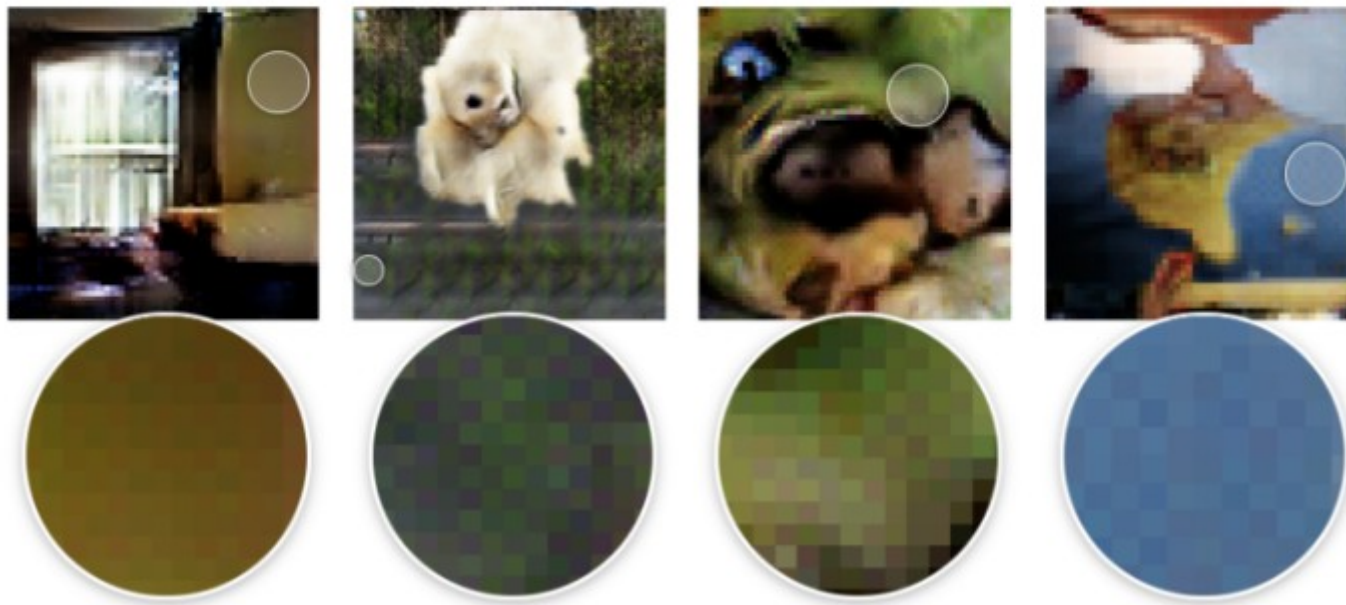
55	110	55	0	110	55	110	55	0	55	110	55	110	0	55	110
110	220	110	0	220	110	220	110	0	110	220	110	220	0	110	220
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0	55	110	55	0	55	110	55	0	55	110	55	0
110	220	110	0	110	220	110	0	110	220	110	0	110	220	110	0
0	110	0	0	0	110	0	0	0	0	0	0	0	0	0	0
55	110	55	0												

- Repare que o resultado não é a mesma imagem original de entrada, de dimensão 4×4 , pois não estamos falando de convolução inversa, mas de um operador de convolução que troca as dimensões dos mapas de ativação na entrada e na saída, quando comparado com o operador de convolução convencional, mantendo o mesmo kernel 3×3 .
- Para tanto, bastou transpor a matriz de convolução associada ao kernel 3×3 . Evidentemente, não há razão para usarmos o mesmo kernel (nos referimos, agora, aos seus parâmetros internos, já que a dimensão se mantém) nos dois tipos de convolução. Mas se as camadas forem espelhadas, eles devem ter as mesmas dimensões e seguir regras bem definidas para stride e padding.
- É possível, dependendo da aplicação, como no caso de autoencoders, que as camadas espelhadas que realizam convolução transposta implementem, de fato, o mapeamento inverso das camadas correspondentes que realizam convolução convencional, devendo encontrar o kernel que viabiliza a operação de inversão.



Problems with Transposed Convolutions:

Transposed convolutions suffer from chequered board effects as shown below.



Fonte: <https://towardsdatascience.com/transposed-convolution-demystified-84ca81b4baba/>

1.6 Camadas convolucionais × camadas densas

- A primeira motivação para o emprego de camadas convolucionais é a economia de conexões sinápticas, quando comparado com a demanda por conexões sinápticas de uma rede neural de camadas densas (*fully connected*).

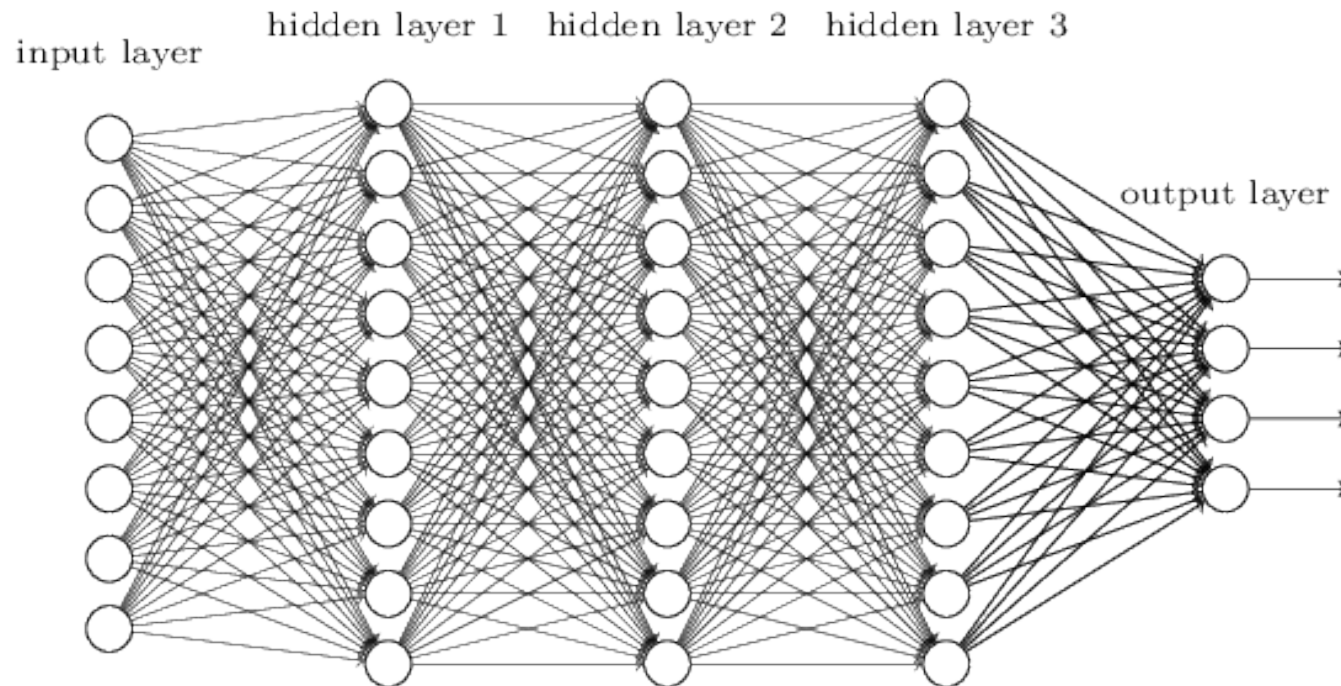


Figura 11 – Exemplo de uma rede neural com todas as camadas *fully connected*.

Fonte: < <http://neuralnetworksanddeeplearning.com/chap6.html> >

- Múltiplas camadas *fully connected* também são capazes de extrair atributos relevantes de imagens, mas requerem um número muito elevado de conexões sinápticas e de neurônios. Aqui, cada neurônio é um filtro não-linear que faz *matching* (produto interno) com toda a imagem produzida pela camada anterior.
- Além desse número elevado de conexões sinápticas, como o número de neurônios é finito, há ao menos outro grande desafio associado a esses filtros de *matching* global, ilustrado na figura a seguir.

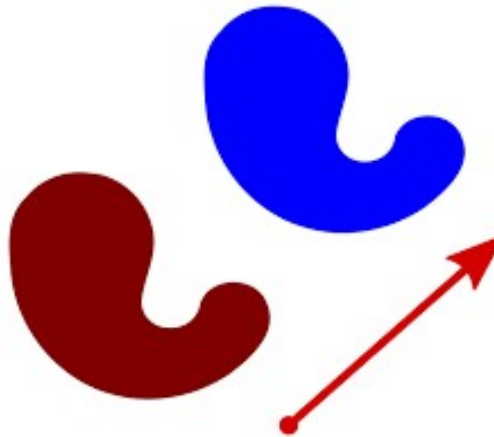
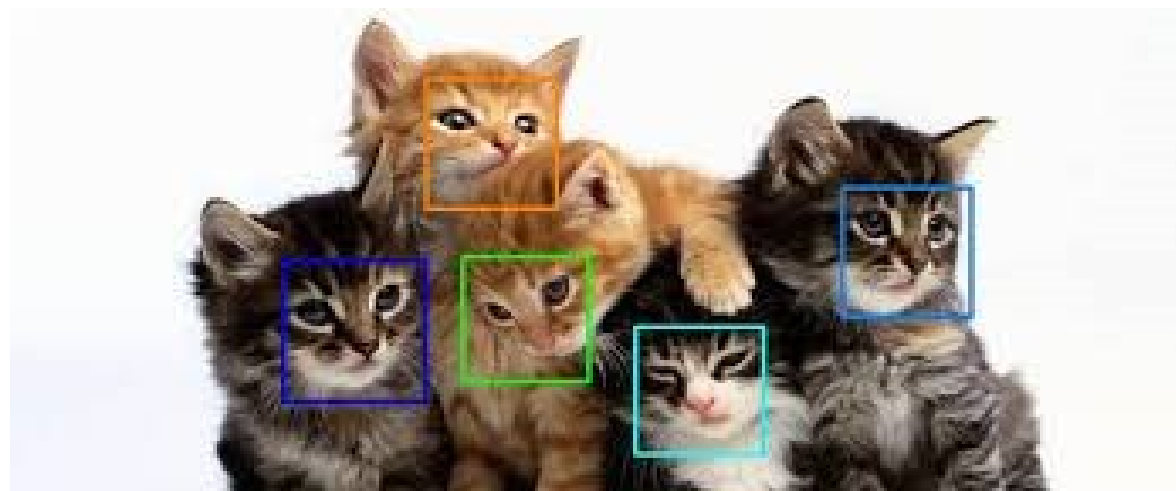


Figura 12 – Translação / Deslocamento de um mesmo objeto numa imagem

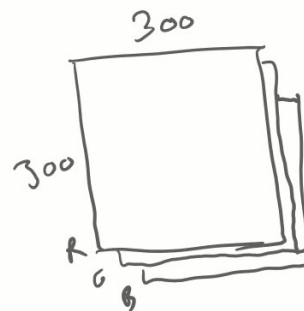
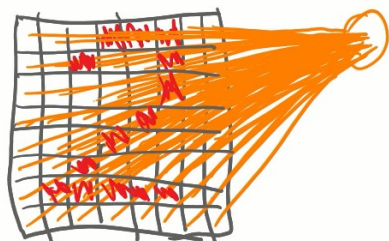
- É evidente que existem “infinitos” deslocamentos possíveis que podem ser aplicados a um mesmo objeto numa imagem. Logo, no caso de filtros globais, para se detectar o referido objeto sempre que ele estiver presente, independentemente de sua posição relativa na imagem, é necessário um número muito elevado de filtros globais, mesmo admitindo a possibilidade de *matching* parcial.
- O uso de convolução e de filtros locais permite atenuar bem esses dois problemas detectados, pois requer-se um único filtro local por objeto a ser detectado (ou uma hierarquia de filtros locais), e cada filtro local possui um número bem reduzido de conexões sinápticas, quando comparado a filtros globais.
- Neste sentido, quando se comparam diferentes imagens, se os objetos não sofressem rotações e deformações não-lineares, apenas deslocamentos, uma única camada convolucional, com filtros de dimensões apropriadas, seria suficiente.
- Esta questão é abordada na comparação entre os cenários de classificação ilustrados a seguir.



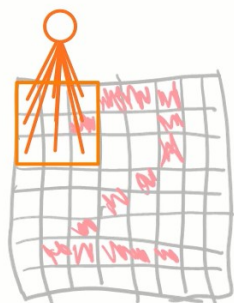
- Você consegue perceber em que diferem as duas tarefas de reconhecimento de padrões (ambas devem detectar gato) ilustradas acima?

- Na presença de transformações afins e deformações, recomenda-se o emprego de múltiplas camadas convolucionais, que representam uma cascata de filtros não-lineares locais extratores de atributos, os quais serão representados hierarquicamente ao longo das camadas.



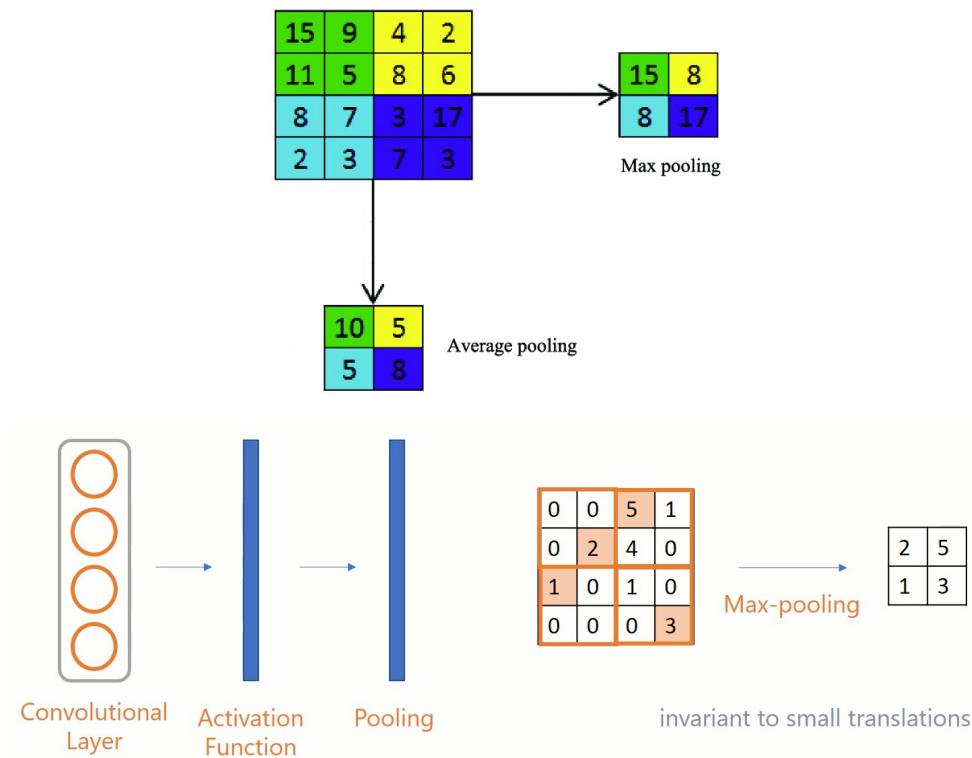


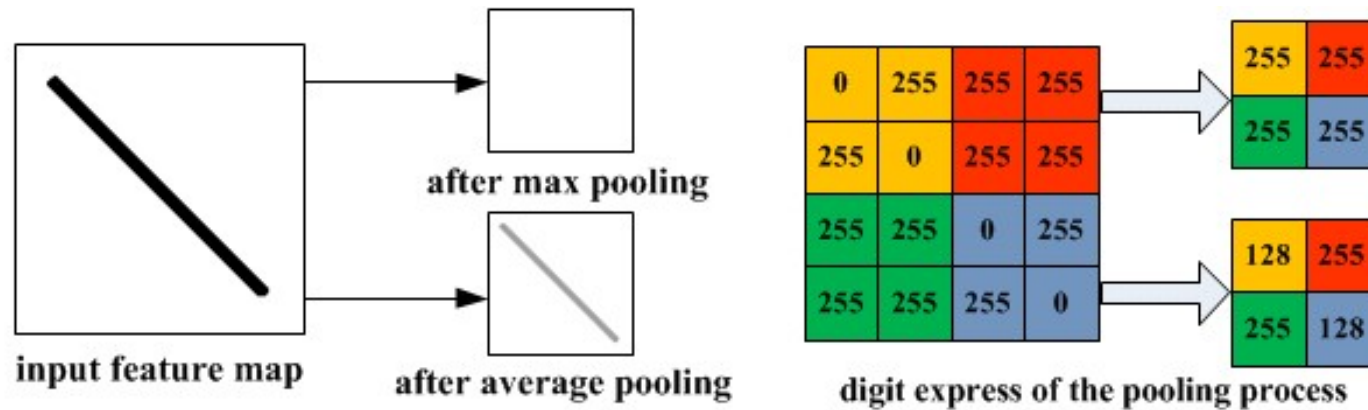
$$300 \times 300 \times 3$$



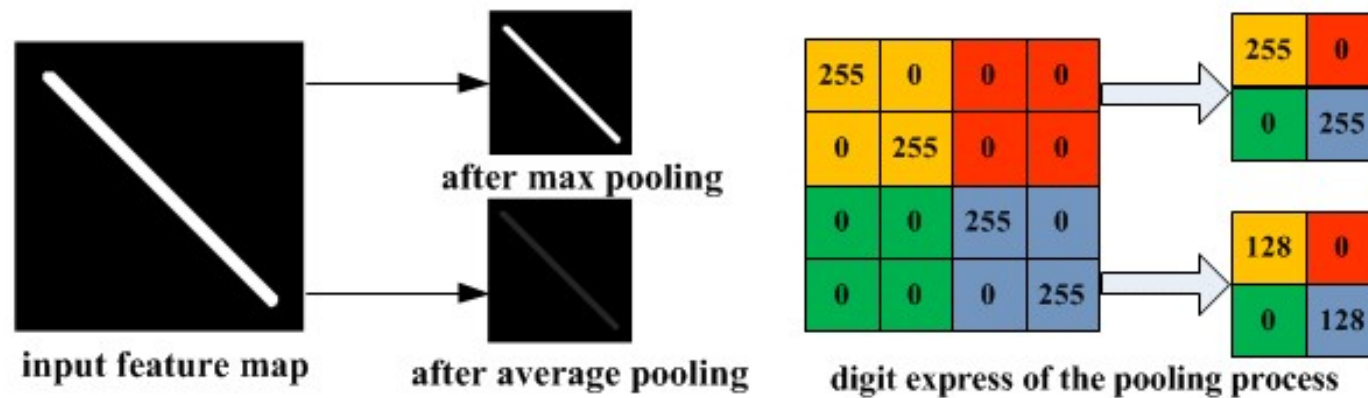
1.7 Operadores max pooling e average pooling

- *Pooling* se refere a uma técnica de sub-amostragem após uma operação de convolução sobre uma imagem. Preserva-se apenas a máxima ativação (*max pooling*) ou a ativação média (*average pooling*) do filtro em cada sub-região do mapa de ativação.





(a) Illustration of max pooling drawback



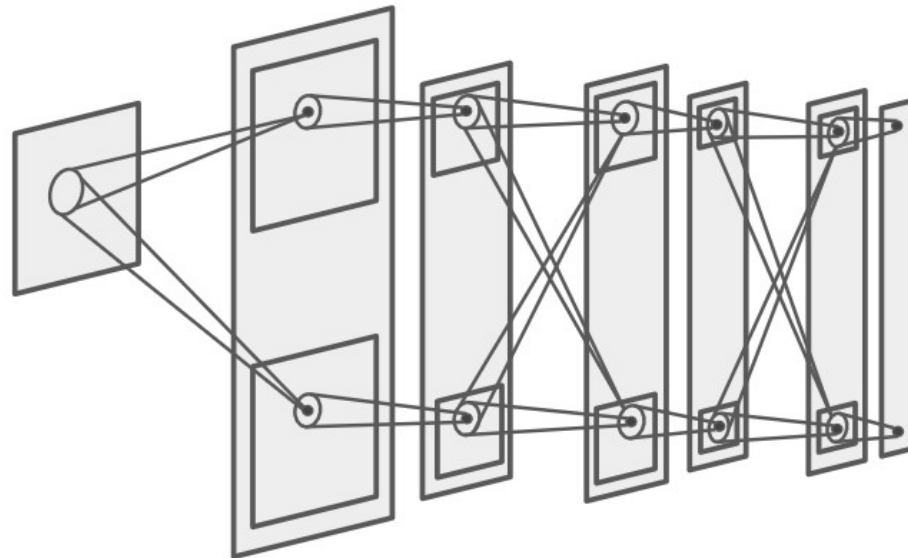
(b) Illustration of average pooling drawback

Figura 13 – Perda de informação com *max* e *average pooling*

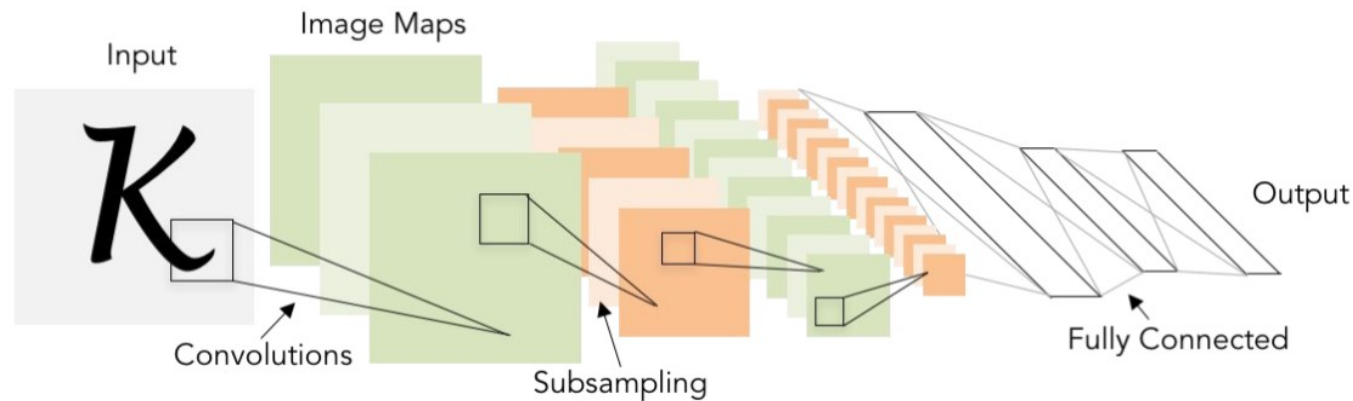
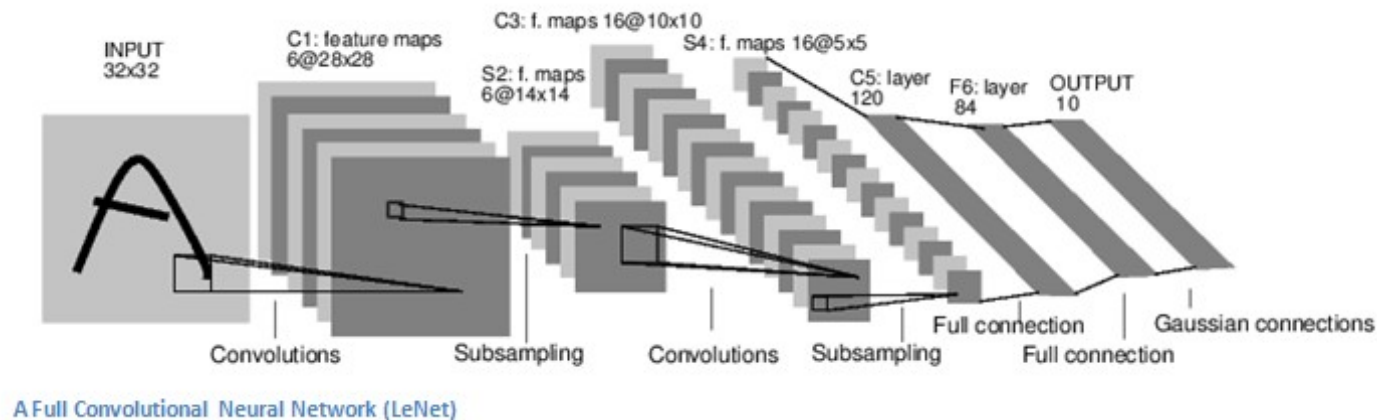
1.8 Redes convolucionais históricas

Neocognitron [Fukushima 1980]

“sandwich” architecture (SCSCSC...)
simple cells: modifiable parameters
complex cells: perform pooling



- **LeNet:** LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. “Gradient-Based Learning Applied to Document Recognition”, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, 1998.



- **AlexNet:** KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. “ImageNet Classification with Deep Convolutional Neural Networks”, Proceedings of the 25th International Conference on Neural Information Processing Systems (NIPS’2012), pp. 1097-1105, 2012.
- ILSVRC (ImageNet Large-Scale Visual Recognition Challenge): top-5 classification error rate in ImageNet dataset goes from 26,2% to 15,3%.

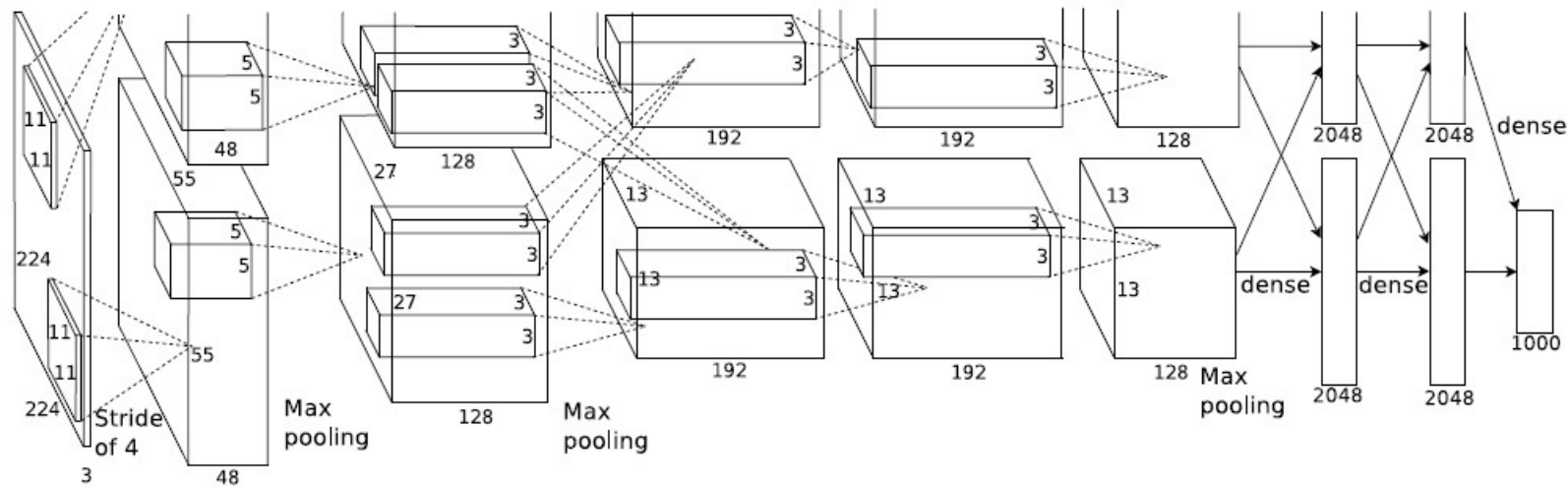


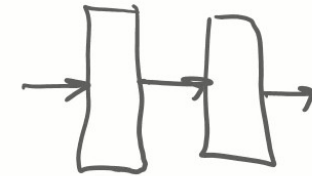
Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network’s input is 150,528-dimensional, and the number of neurons in the network’s remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

- 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax.
- To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective.
- ImageNet consists of over 15 million labeled high-resolution images in over 22,000 categories.
- DENG, J.; DONG, W.; SOCHER, R.; LI, L.-J.; LI, K.; FEI-FEI, L. “ImageNet: A Large-Scale Hierarchical Image Database”, Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’09), 2009.

1.9 Como projetar redes convolucionais?

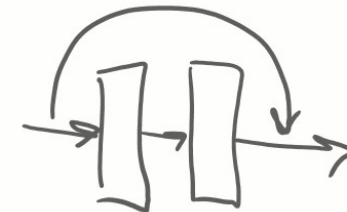
No human intervention in deep learning?

- humans are still in the loop!



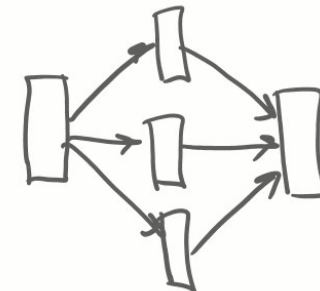
Grid search on all hyperparameters?

- too many hyperparameters
- infinitely many ways to design a network



Designing a deep neural network involves

- human expertise
- trial and error



Building blocks of CNNs

- Convolution
- Pooling

Hyperparameters

- Kernel size, stride, dilation rate
- Number of filters, number of layers
- Pooling size & type
- Padding type
- The way we arrange the layers

How to design a ConvNet?

TL;DR

- you don't design
- pick something that works and use it

Working on novel problems?

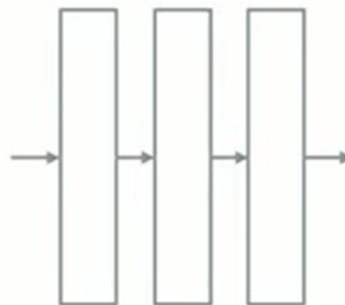
- borrow ideas from successful models
- design your own model

How do choose the number of layers and units?

- start small
- gradually increase model size
- smallest model: linear regression
- deeper:



- wider:

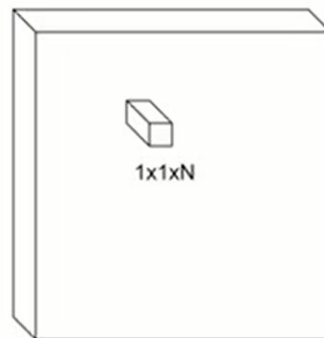


How to choose kernel size?

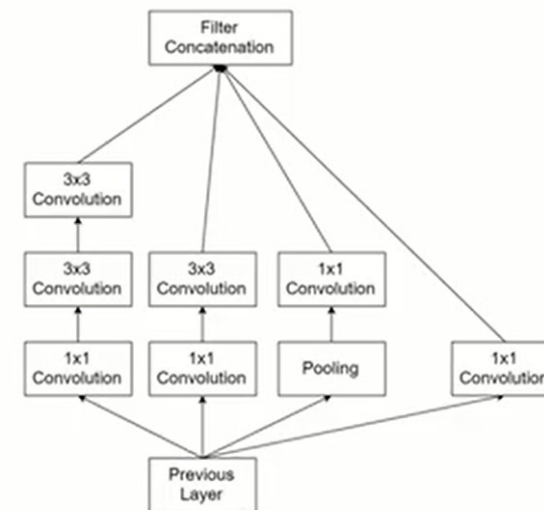
- 3x3 and 1x1 kernels usually work the best

Pointwise (1x1) filters

- channel-wise dense layers
- learn cross-channel features

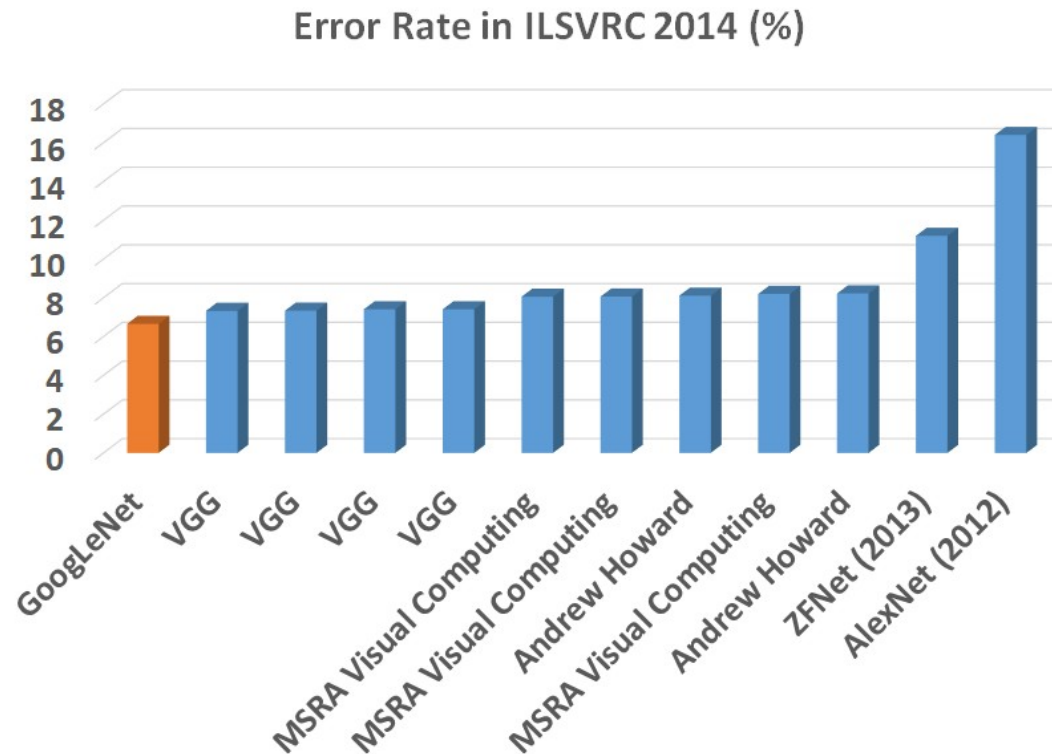


Inception module



- Inception module: GoogLeNet (<https://arxiv.org/abs/1409.4842>).

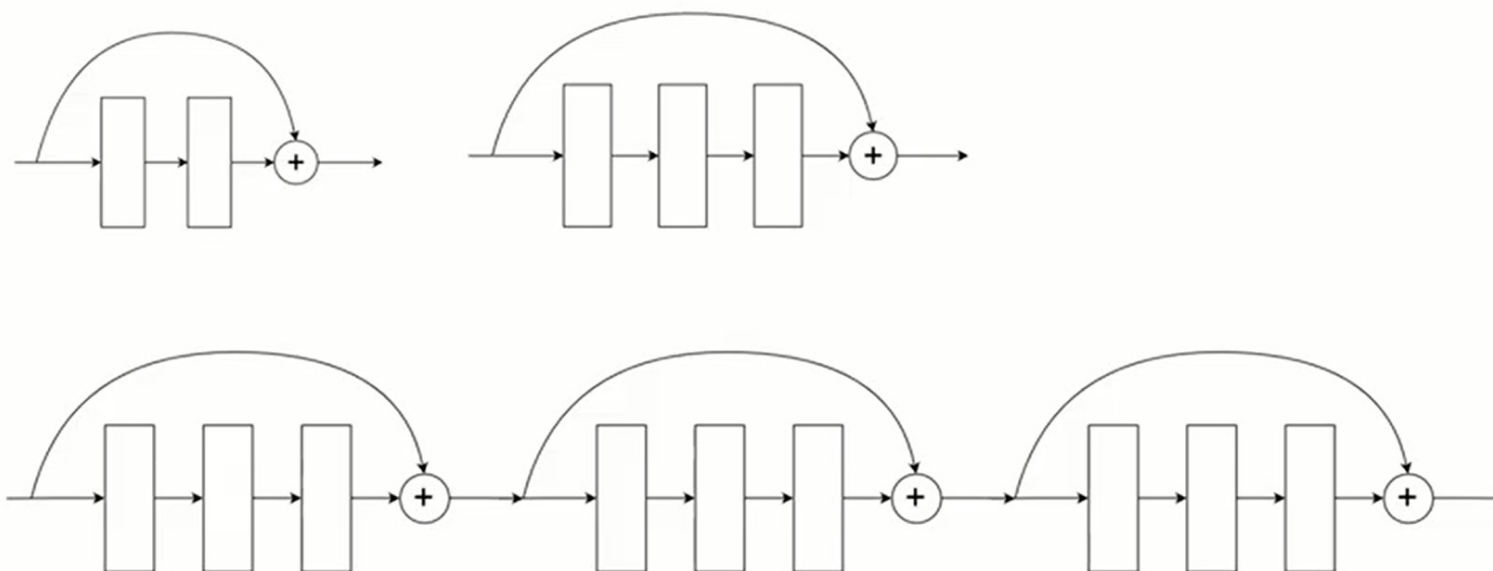
GoogLeNet: Winner of ILSVRC 2014



ImageNet, is a dataset of over 15 millions labeled high-resolution images with around 22,000 categories. ILSVRC uses a subset of ImageNet of around 1000 images in each of 1000 categories. In all, there are roughly 1.2 million training images, 50,000 validation images and 100,000 testing images. Fonte: <https://medium.com/coinmonks/paper-review-of-googlenet-inception-v1-winner-of-ilsvlc-2014-image-classification-c2b3565a64e7> >

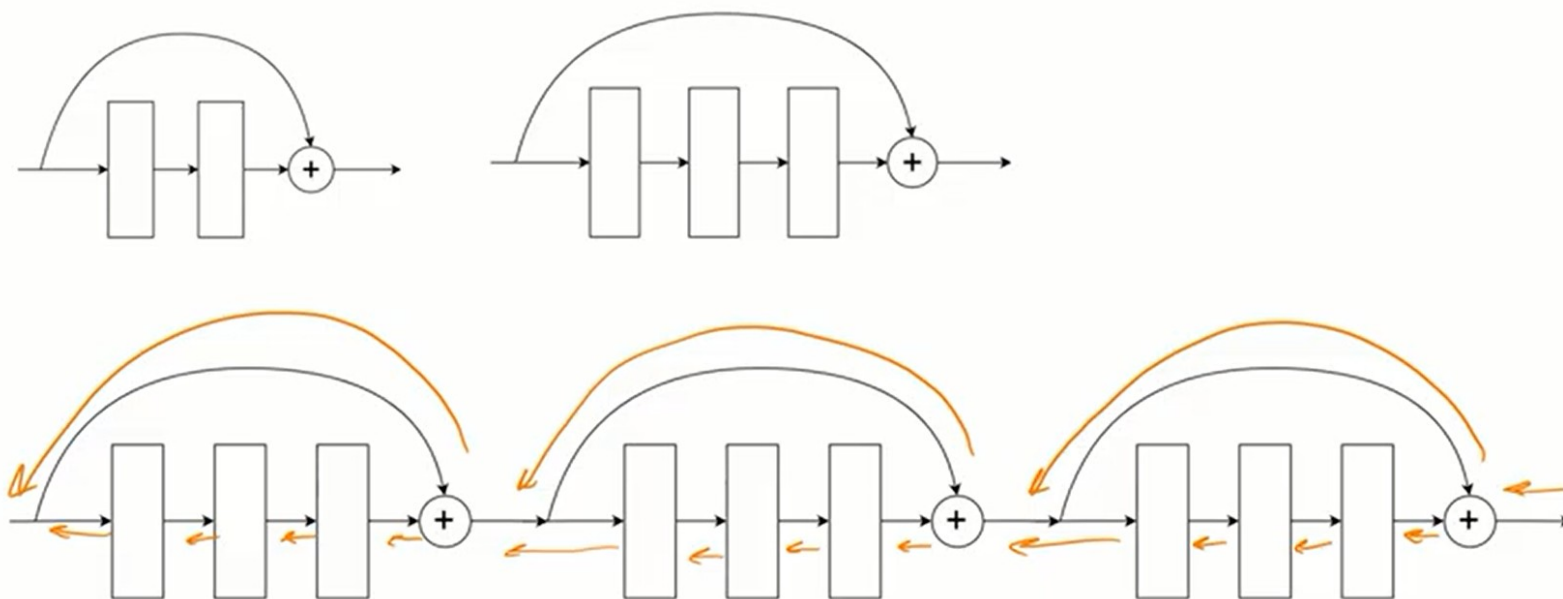
Skip connections

- building blocks of the ResNet architecture



Skip connections

- building blocks of the ResNet architecture



method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).

method	top-5 err. (test)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	3.57

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

HE, K., ZHANG, X., REN, S. & SUN, J. “Deep Residual Learning for Image Recognition”, arXiv:1512.03385v1, 2015.

How to choose the batch size?

- Image recognition tasks
 - batch size: 32
- Noisy gradient?
 - use larger batches
- Stuck in local minima? Out of memory?
 - use smaller batches

2 Convoluções separáveis

- As convoluções separáveis (*separable convolutions*) são técnicas alternativas à convolução tradicional, buscando reduzir significativamente o número de parâmetros livres dos filtros convolucionais e reduzir também o custo computacional associado às operações de convolução, mas sem comprometer demasiadamente o potencial de representação da máquina de aprendizado.
- Os dois principais tipos são a convolução separável no espaço (*spatial separable convolution*) e a convolução separável em profundidade (*depthwise separable convolution*), que serão descritos a seguir.

2.1 Convolução separável no espaço

- Para cada canal, a convolução separável no espaço (*spatial separable convolution*) substitui a matriz do filtro convolucional, que, em princípio, é arbitrária e tem dimensão $h \times h$, por uma matriz $h \times h$ de posto unitário, resultante do produto externo de dois filtros unidimensionais, um deles $h \times 1$ e o outro $1 \times h$.

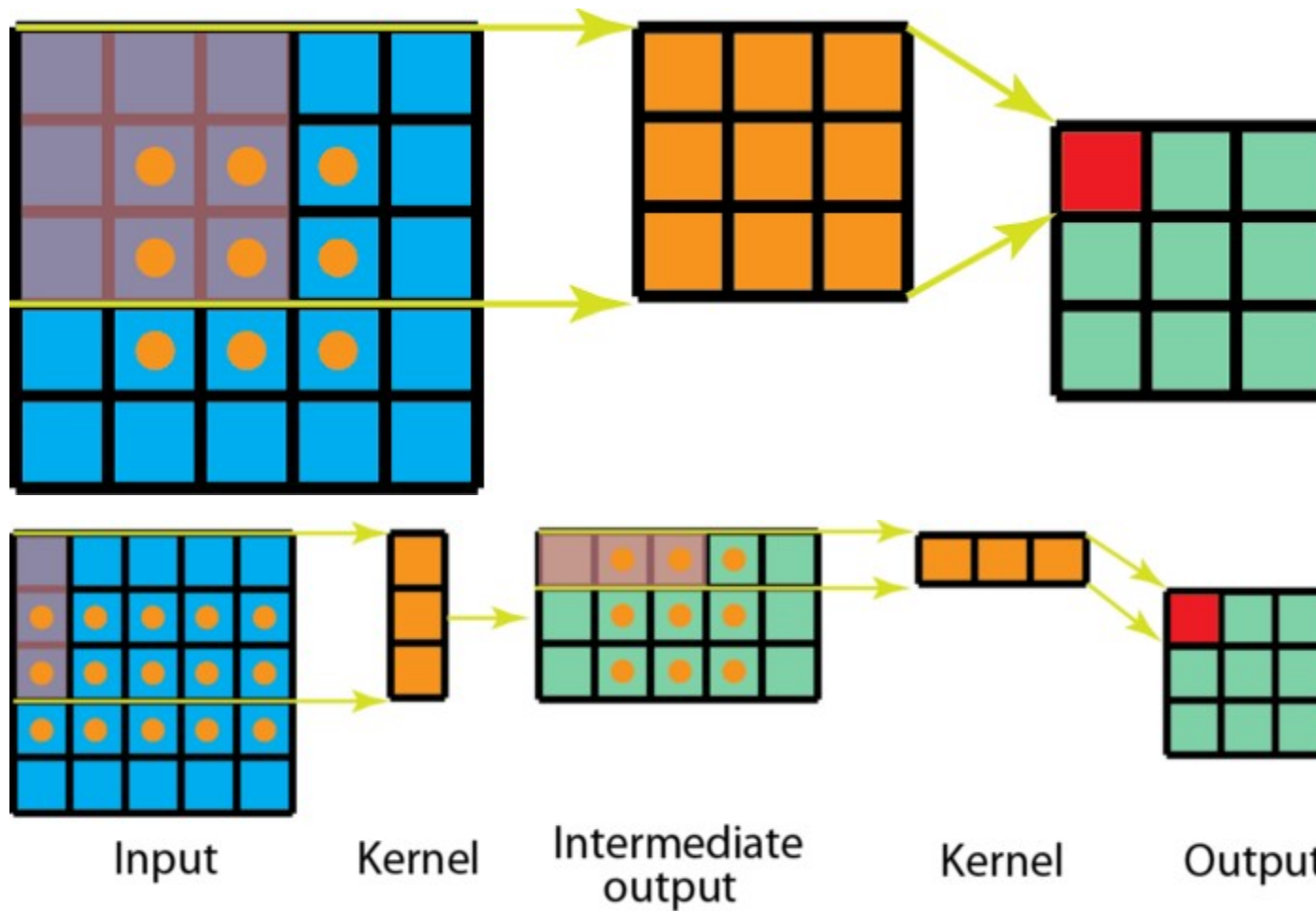
- A convolução, portanto, ocorre em duas etapas, uma para cada filtro unidimensional resultante da decomposição. Vale ressaltar que a decomposição ocorre somente nas dimensões de largura e altura de modo que a dimensão de profundidade permanece a mesma.
- Como um exemplo, o filtro de Sobel, para detecção de bordas, pode ser exatamente representado por este tipo de decomposição:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}.$$

- O *matching* com um filtro $h \times h$ tradicional numa imagem de resolução $H \times W$, *stride*=1 e *padding*=0, envolve $h \times h \times (H - h + 1) \times (W - h + 1)$ multiplicações. Já no caso da convolução separável no espaço, o número de multiplicações fica $W \cdot (H - h + 1) \cdot h + (W - h + 1) \cdot (H - h + 1) \cdot h = (2W - h + 1) \cdot (H - h + 1) \cdot h$.
- A razão entre esses dois números (último pelo primeiro) fica:

$$\frac{2}{h} + \frac{h-1}{h \cdot (W-h+1)}$$

- No caso em que $W \gg h$, a razão se aproxima de $\frac{2}{h}$, sendo $\frac{2}{3}$ para filtros 3×3 , $\frac{2}{5}$ para filtros 5×5 e $\frac{2}{7}$ para filtros 7×7 .
- O valor $\frac{2}{h}$ também dá a razão exata entre os parâmetros livres em cada caso.
- Mesmo com esta razoável economia de recursos computacionais, a convolução separável no espaço é raramente adotada em *deep learning*, pois a enorme maioria dos filtros convolucionais não é representável por esta decomposição, o que implicaria uma redução expressiva da capacidade de representação, a qual geralmente supera o ganho com a economia de recursos computacionais.
- A ideia de reduzir parâmetros livres e o número de multiplicações é válida, mas desde que não impacte demais na capacidade de representação do modelo de aprendizado, que aqui leva a filtros com posto unitário.

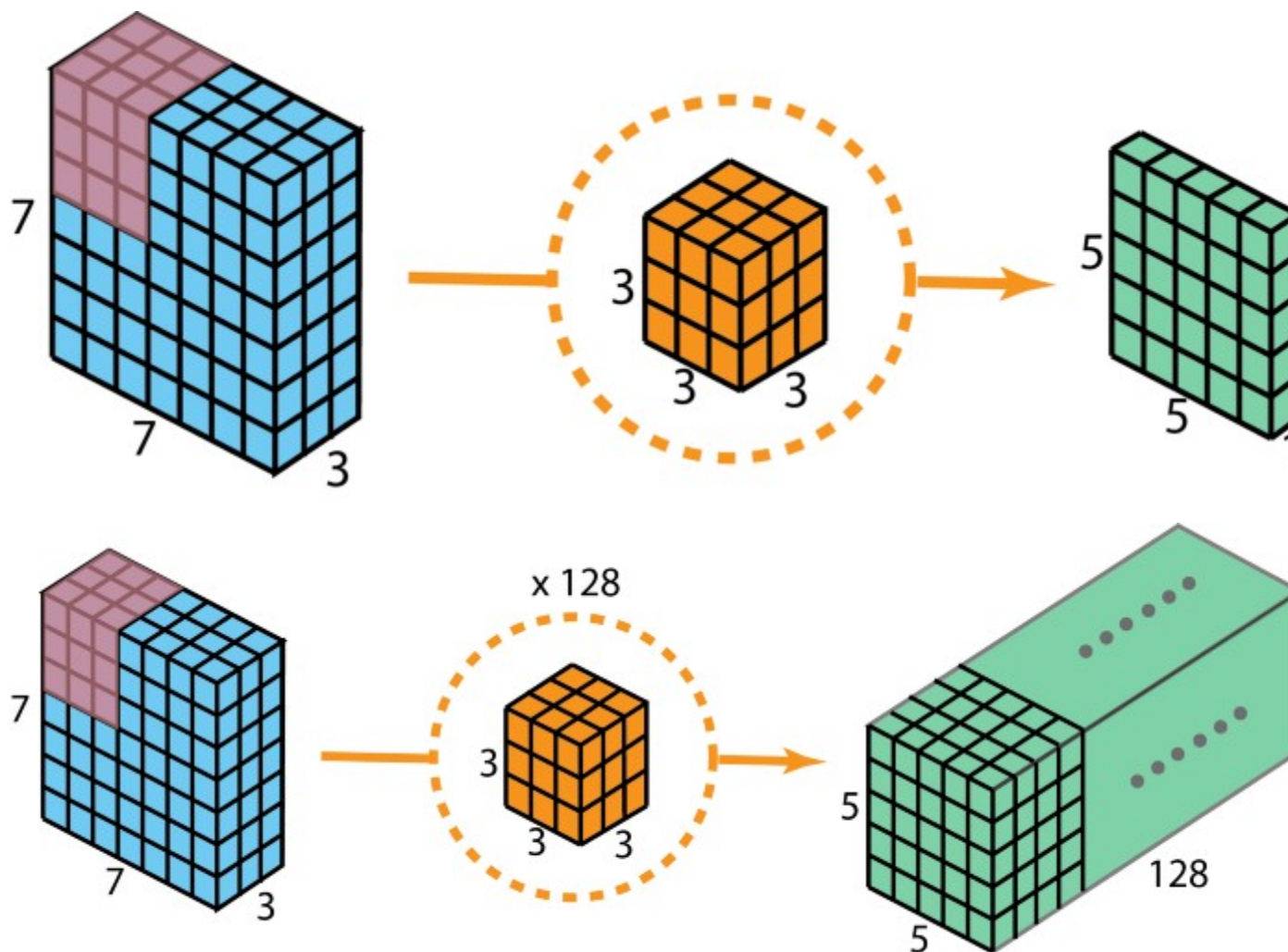


81 multiplicações e 9 parâmetros livres (acima) contra 72 multiplicações e 6 parâmetros livres (abaixo), isso sem considerar a profundidade (número de canais)

Fonte: <https://medium.com/data-science/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215>

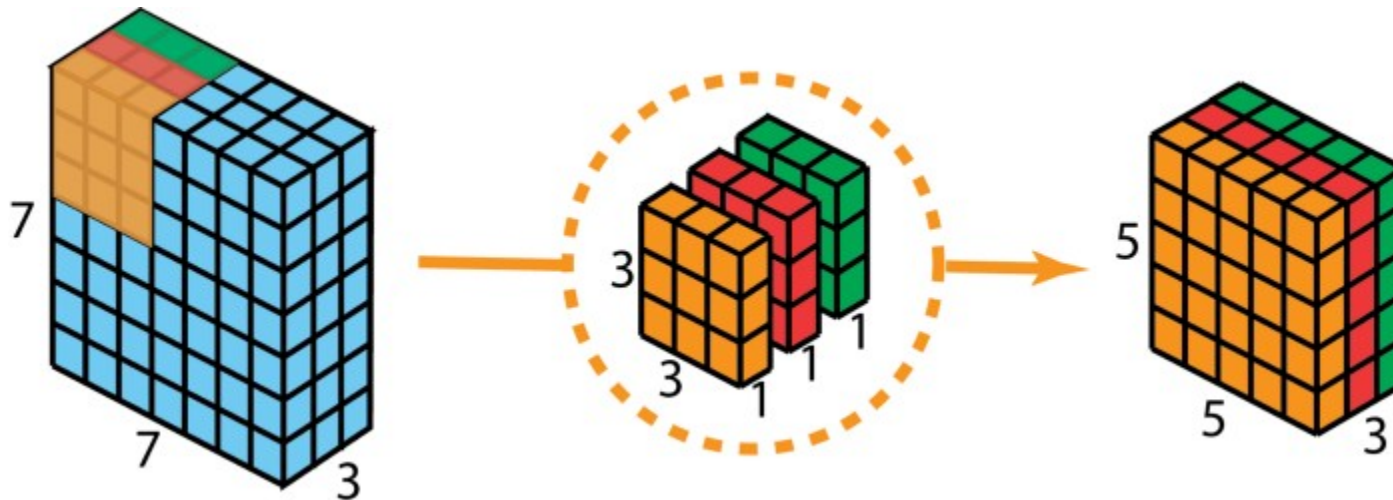
2.2 Convolução separável em profundidade

- A convolução separável em profundidade (*depthwise separable convolution*) também vai ser realizada em duas etapas, operando com um filtro por canal da imagem de entrada (*depthwise*), na etapa 1, e depois com tantos filtros $1 \times 1 \times c$, sendo c o número de canais de entrada, quanto canais de saída, na etapa 2.
- Esses filtros $1 \times 1 \times c$ realizam as chamadas convoluções ponto-a-ponto (do inglês *pointwise convolution*).
- Vamos tomar, a seguir, um caso de estudo $7 \times 7 \times 3$ (altura $\times$ largura $\times$ canais) e realizar uma convolução 2D tradicional usando 128 filtros de tamanho $3 \times 3 \times 3$. A saída de cada filtro (mapa de ativação) será $5 \times 5 \times 1$, resultando em uma saída empilhada de dimensão $5 \times 5 \times 128$.
- Com isso, a entrada $7 \times 7 \times 3$ desta camada convolucional tradicional foi convertida em uma saída $5 \times 5 \times 128$.

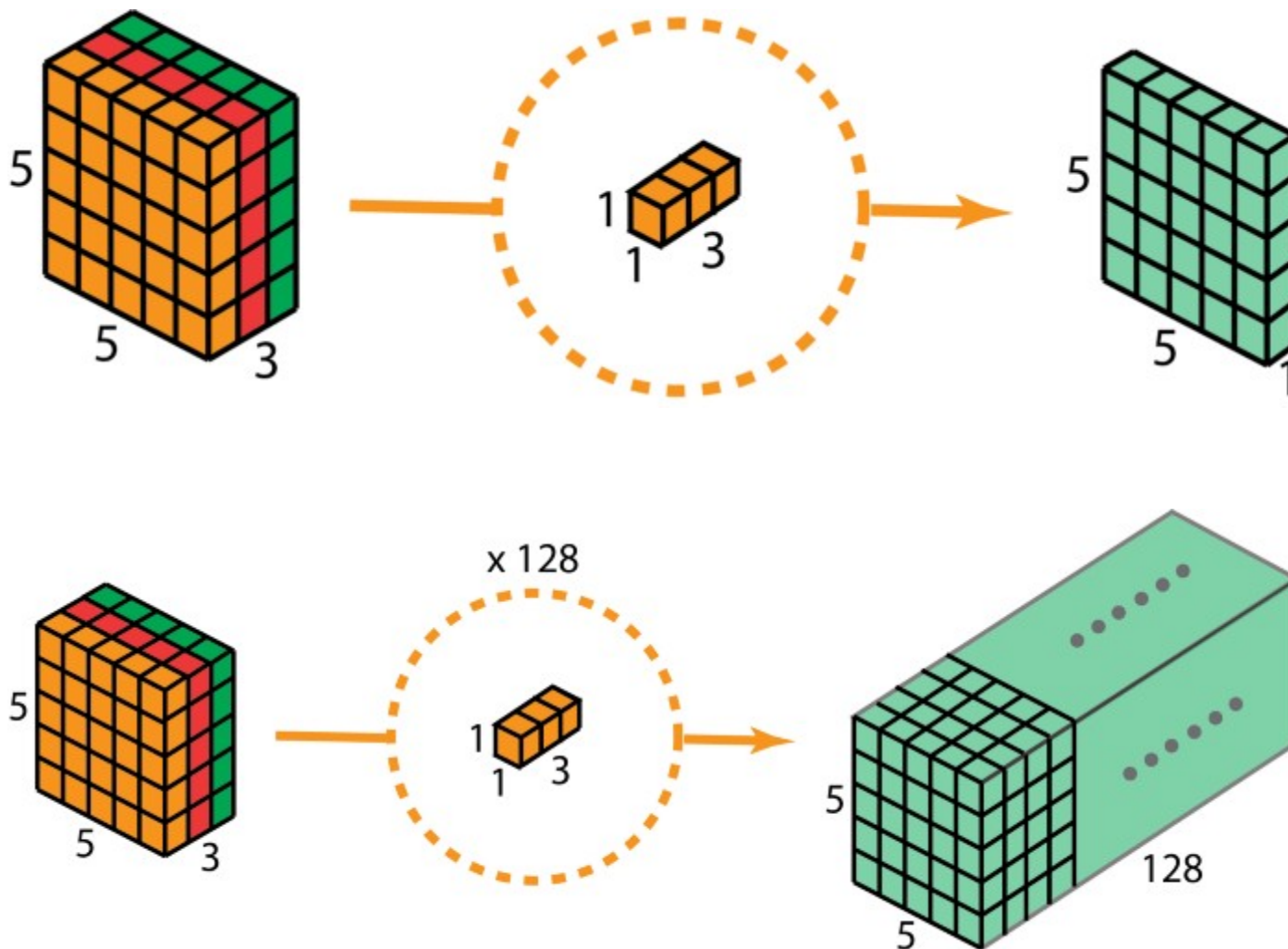


Fonte: <https://medium.com/data-science/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215>

- No caso da convolução separável em profundidade, a etapa 1 envolve 3 filtros $3 \times 3 \times 1$, cada um gerando um mapa de ativação $5 \times 5 \times 1$. Empilhando esses 3 mapas de ativação, resulta a saída intermediária de dimensão $5 \times 5 \times 3$.

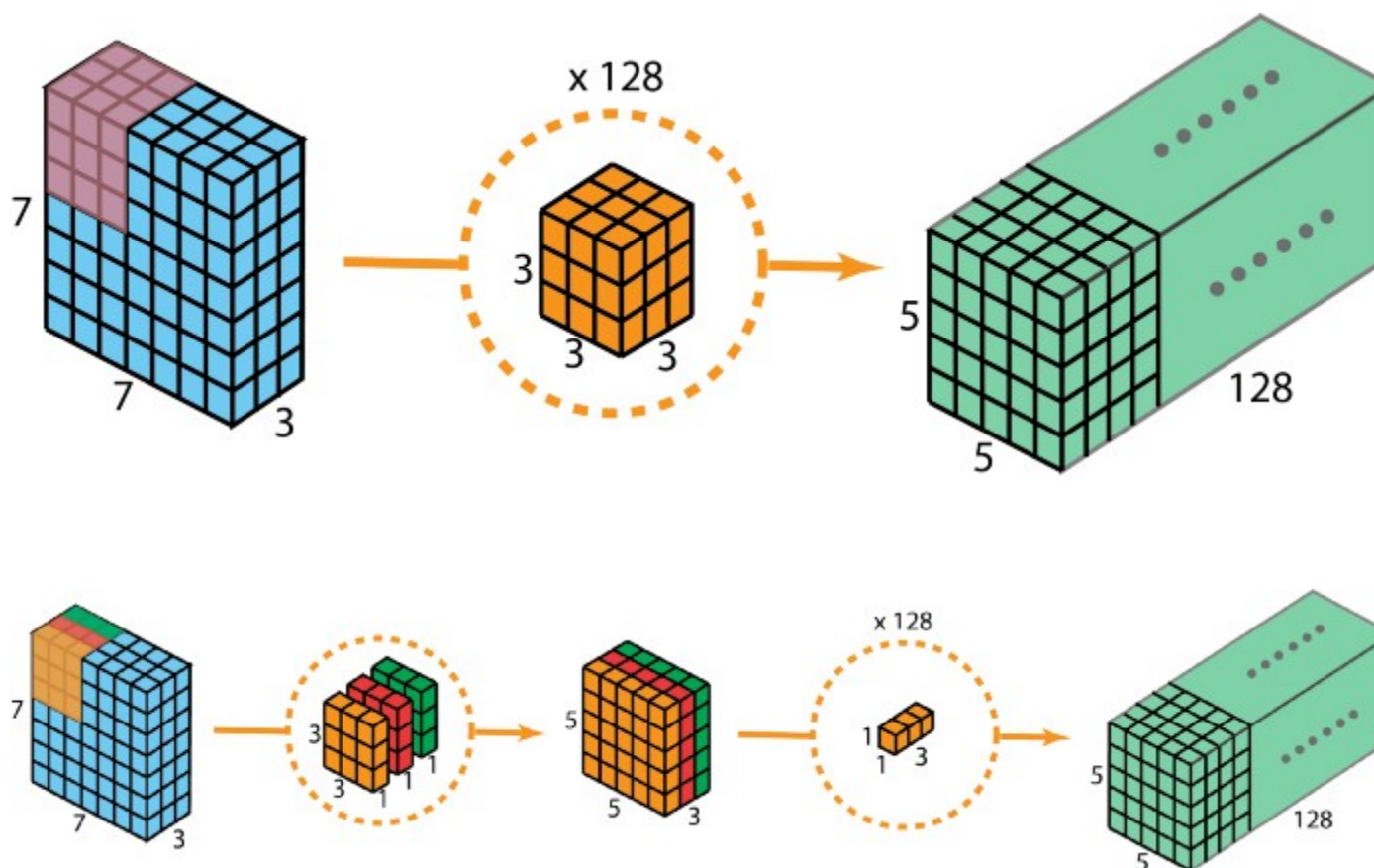


- Na etapa 2, 128 filtros de dimensão $1 \times 1 \times 3$ são aplicados, resultando em 128 mapas de ativação $5 \times 5 \times 1$ que, ao serem empilhados, produzem uma saída $5 \times 5 \times 128$.
- Novamente teremos a entrada $7 \times 7 \times 3$ convertida em uma saída $5 \times 5 \times 128$, mas com muito menos parâmetros e operações de multiplicação.



Fonte: <https://medium.com/data-science/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215>

- As duas convoluções podem ser comparadas nas figuras que seguem.



Fonte: <https://medium.com/data-science/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215>

- Na convolução tradicional, temos 128 filtros $3 \times 3 \times 3$ que se movem 5×5 vezes, resultando em $128 \times 3 \times 3 \times 3 \times 5 \times 5 = 86.400$ multiplicações.
- Na etapa 1 da convolução em profundidade, existem 3 filtros $3 \times 3 \times 1$ que se movem 5×5 vezes, resultando em $3 \times 3 \times 3 \times 1 \times 5 \times 5 = 675$ multiplicações. Na etapa 2, existem 128 filtros $1 \times 1 \times 3$ que se movem 5×5 vezes, resultando em $128 \times 1 \times 1 \times 3 \times 5 \times 5 = 9.600$ multiplicações. Assim, em geral, a convolução separável em profundidade executa $675 + 9600 = 10.275$ multiplicações. Isso é menos de 12% do custo da convolução tradicional.
- Em termos de parâmetros livres, as duas convoluções operam com:
 - ✓ Tradicional: $128 \times 3 \times 3 \times 3 = 3.456$ parâmetros livres
 - ✓ Separável em profundidade: $3 \times 3 \times 3 + 128 \times 3 = 411$ parâmetros livres
- É evidente que a convolução tradicional é mais flexível, mas, desta vez, a redução no uso de recursos computacionais tende a ser mais expressiva do que a queda na flexibilidade, o que gera uma boa relação custo-benefício.

- Já sabemos do poder representacional de modelos que realizam diferentes combinações lineares de “funções-base”. Em convolução separável em profundidade, temos tantas “funções-base” quanto mapas de ativação da etapa 1 e tantas combinações lineares de “funções-base” quanto filtros na etapa 2.
- Considerando dimensões genéricas, para uma imagem de entrada de tamanho $H \times W \times D$, queremos fazer convolução tradicional ($stride=1$, $padding=0$) com N filtros de tamanho $h \times h \times D$. Isso transforma a entrada de dimensão $H \times W \times D$ numa saída de dimensão $(H-h+1) \times (W-h+1) \times N$. As multiplicações necessárias são:

$$N \times h \times h \times D \times (H-h+1) \times (W-h+1)$$

- Já no caso da convolução separável em profundidade, o número total de multiplicações fica:

$$D \times h \times h \times 1 \times (H-h+1) \times (W-h+1) + N \times 1 \times 1 \times D \times (H-h+1) \times (W-h+1) = \\ (h \times h + N) \times D \times (H-h+1) \times (W-h+1)$$

- A razão entre esses dois números (último pelo primeiro) fica:

$$\frac{1}{N} + \frac{1}{h^2}$$

- A mesma razão vale para os parâmetros livres. Quando o número N de canais é elevado, a razão se aproxima de $1/h^2$, implicando em reduções aproximadas de 9, 25 e 49 vezes no número total de multiplicações e parâmetros livres quando se usa a convolução separável em profundidade, para filtros 3×3 , 5×5 e 7×7 .
- Evidentemente, essas reduções expressivas no número de multiplicações e no número de parâmetros livres tendem a ser benéficas em redes bem profundas (com muitas camadas convolucionais) e largas (com muitos canais por camada), requerendo menos dados de treinamento para se chegar a uma boa capacidade de generalização e requerendo também um menor custo computacional de treinamento (bem menos parâmetros ajustáveis, o que também pode levar a progressos mais expressivos) e de operação “de campo” (após o treinamento).

- As convoluções separáveis em profundidade viabilizaram muitos aplicativos para dispositivos móveis que operam em tempo real, mesmo sob fortes restrições de memória e de processamento.

MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

Andrew G. Howard Menglong Zhu Bo Chen Dmitry Kalenichenko
Weijun Wang Tobias Weyand Marco Andreetto Hartwig Adam

Google Inc.

{howarda, menglong, bochen, dkalenichenko, weijunw, weyand, anm, hadam}@google.com

Abstract

We present a class of efficient models called MobileNets for mobile and embedded vision applications. MobileNets are based on a streamlined architecture that uses depth-wise separable convolutions to build light weight deep neural networks. We introduce two simple global hyper-parameters that efficiently trade off between latency and accuracy. These hyper-parameters allow the model builder

models. Section 3 describes the MobileNet architecture and two hyper-parameters width multiplier and resolution multiplier to define smaller and more efficient MobileNets. Section 4 describes experiments on ImageNet as well a variety of different applications and use cases. Section 5 closes with a summary and conclusion.

2. Prior Work

There has been rising interest in building small and eff

17 Apr 2017

Xception: Deep Learning with Depthwise Separable Convolutions

François Chollet
Google, Inc.

fchollet@google.com

Abstract

We present an interpretation of Inception modules in convolutional neural networks as being an intermediate step in-between regular convolution and the depthwise separable convolution operation (a depthwise convolution followed by a pointwise convolution). In this light, a depthwise separable convolution can be understood as an Inception module with a maximally large number of towers. This observation leads us to propose a novel deep convolutional neural network architecture inspired by Inception, where Inception modules have been replaced with depthwise separable convolutions. We show that this architecture, dubbed Xception, slightly outperforms Inception V3 on the ImageNet dataset (which Inception V3 was designed for), and significantly outperforms Inception V3 on a larger image classification dataset comprising 350 million images and 17,000 classes. Since the Xception architecture has the same number of parameters as Inception V3, the performance gains are not due to increased capacity but rather to a more efficient use of model parameters.

as GoogLeNet (Inception V1), later refined as Inception V2 [7], Inception V3 [21], and most recently Inception-ResNet [19]. Inception itself was inspired by the earlier Network-In-Network architecture [11]. Since its first introduction, Inception has been one of the best performing family of models on the ImageNet dataset [14], as well as internal datasets in use at Google, in particular JFT [5].

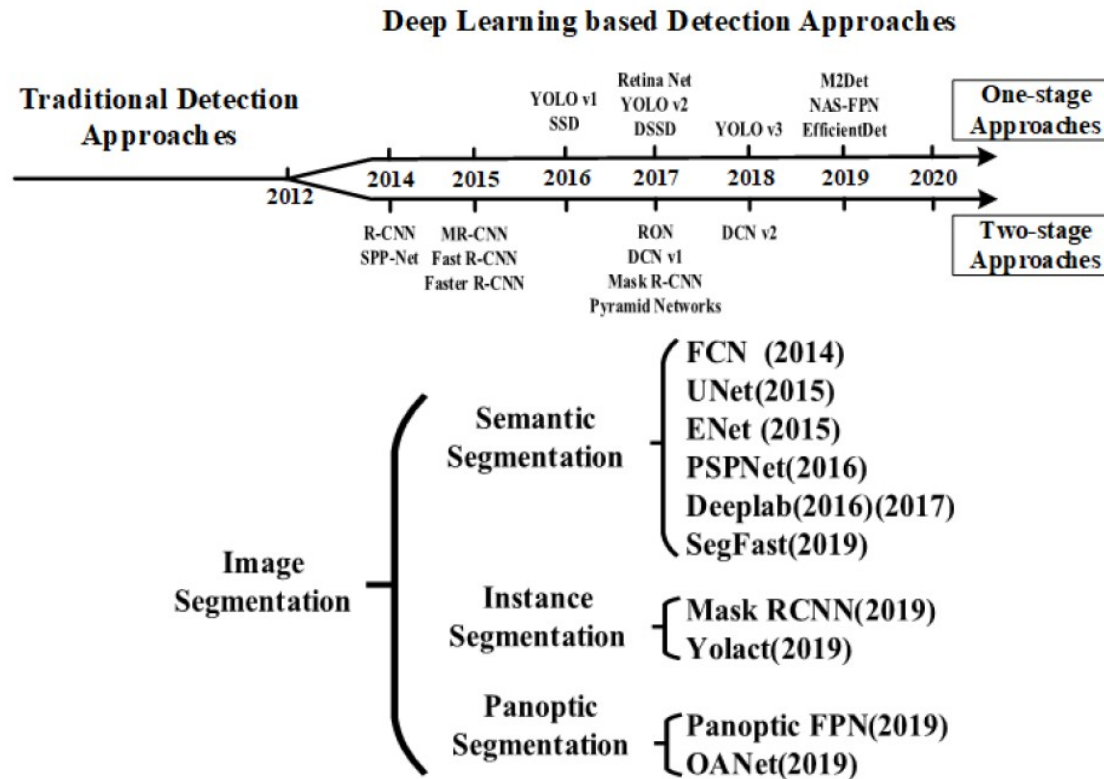
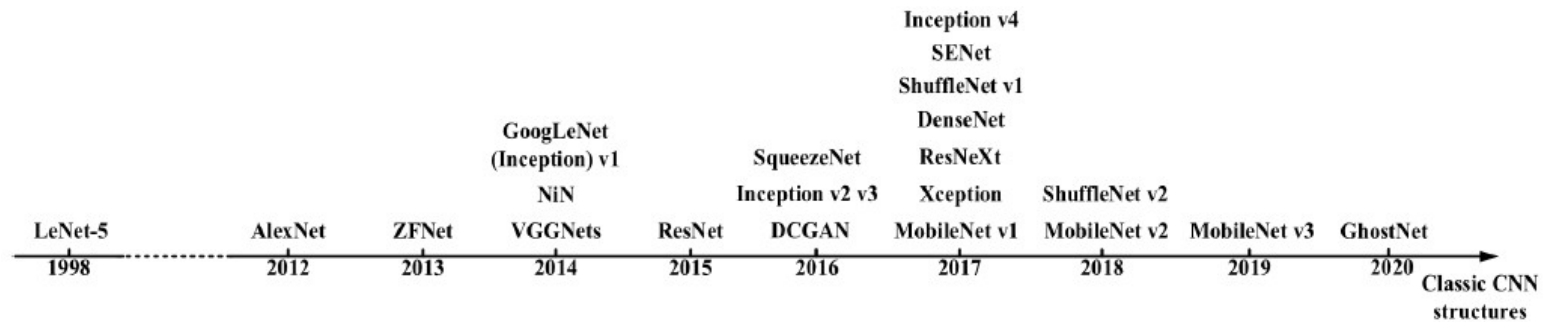
The fundamental building block of Inception-style models is the Inception module, of which several different versions exist. In figure 1 we show the canonical form of an Inception module, as found in the Inception V3 architecture. An Inception model can be understood as a stack of such modules. This is a departure from earlier VGG-style networks which were stacks of simple convolution layers.

While Inception modules are conceptually similar to convolutions (they are convolutional feature extractors), they empirically appear to be capable of learning richer representations with less parameters. How do they work, and how do they differ from regular convolutions? What design strategies come after Inception?

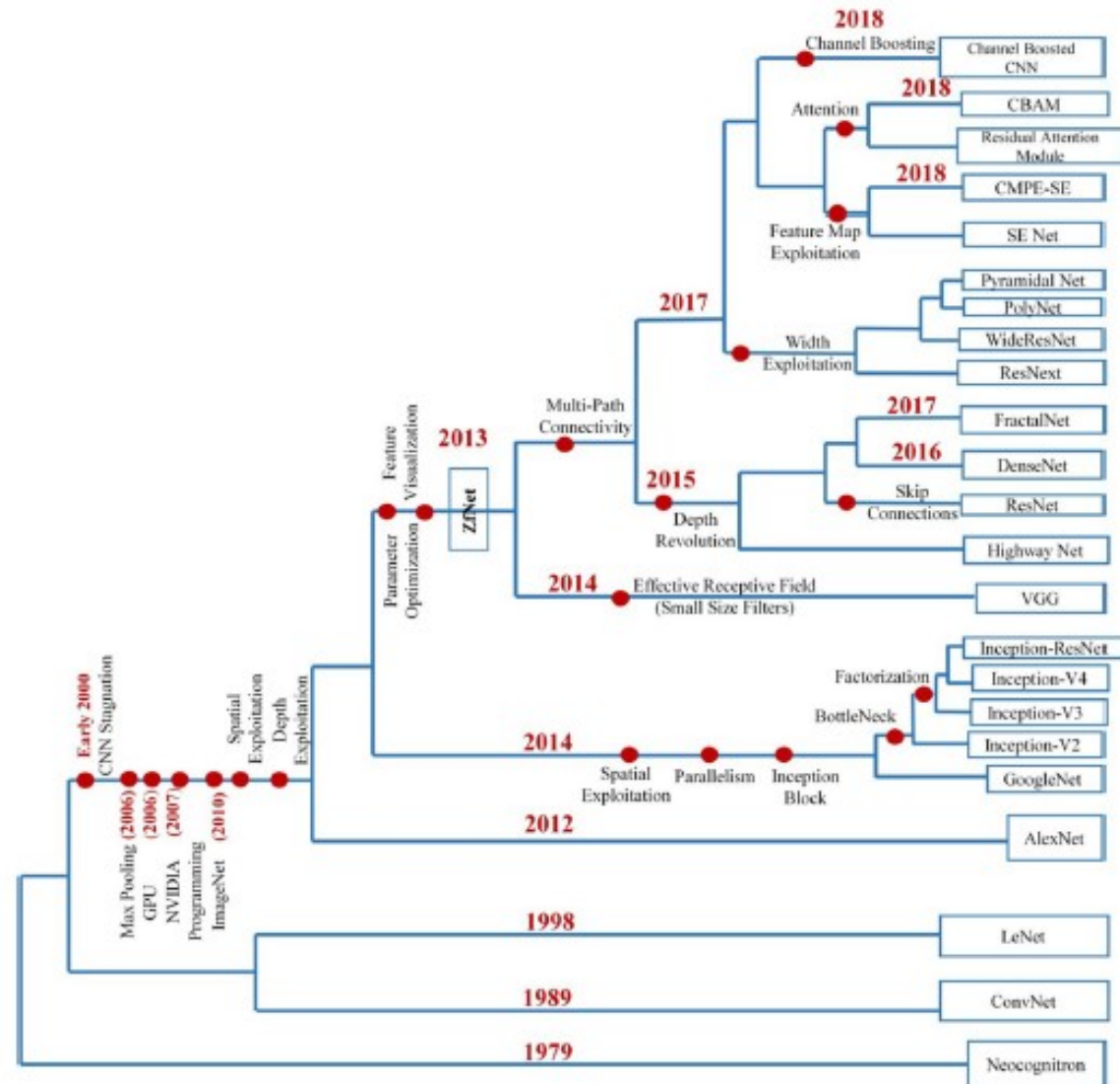
1.1. The Inception hypothesis

0.02357v3 [cs.CV] 4 Apr 2017

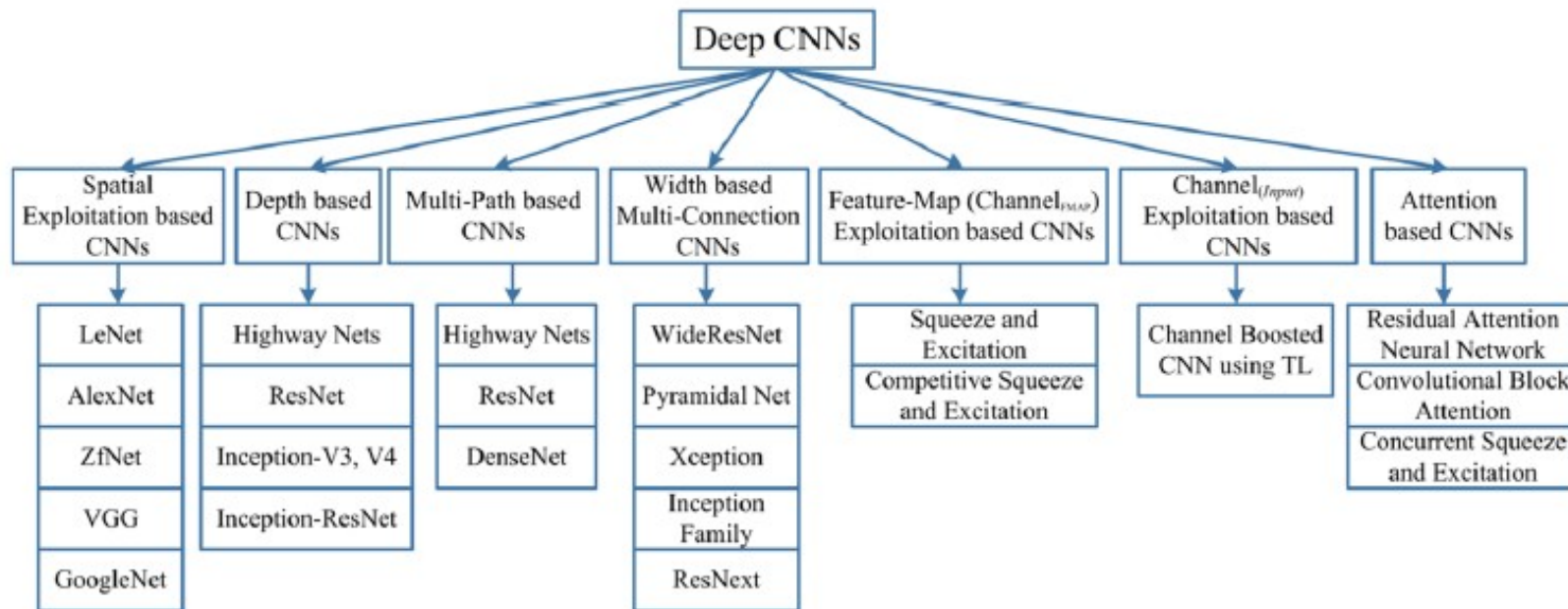
- Mais informações sobre convoluções separáveis podem ser obtidas em:
 - ✓ <https://medium.com/data-science/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215>
 - ✓ <https://eli.thegreenplace.net/2018/depthwise-separable-convolutions-for-machine-learning/>
 - ✓ <https://medium.com/data-science/a-basic-introduction-to-separable-convolutions-b99ec3102728>
 - ✓ <https://www.geeksforgeeks.org/depth-wise-separable-convolutional-neural-networks/>
 - ✓ <https://www.youtube.com/watch?v=T7o3xvJLuHk>
- Alguns reviews e surveys importantes para redes convolucionais em geral:
 - ✓ <https://arxiv.org/abs/1803.02129>
 - ✓ Li, Z.; Liu, F.; Yang, W.; Peng, S.; Zhou, J. “A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects”, IEEE Transactions on Neural Networks and Learning Systems, doi: 10.1109/TNNLS.2021.3084827, 2021.
 - ✓ Khan, A.; Sohail, A.; Zahoora, U.; Qureshi, A.S. “A survey of the recent architectures of deep convolutional neural networks”, Artificial Intelligence Review, vol. 53, pp. 5455-5516, 2020.



Fonte: Li et al. (2021).



Fonte: Khan et al. (2020).



Fonte: Khan et al. (2020).

- Um tema não abordado nessas notas de aula e que merece destaque:

✓ *Fixing the train-test resolution discrepancy:* <https://arxiv.org/abs/1906.06423>

3 Dropout

- Dada a enorme flexibilidade associada aos mapeamentos de entrada-saída que podem ser sintetizados em redes neurais com arquiteturas profundas (**não restritas a redes convolucionais**), o problema de *overfitting* é muito acentuado e precisa ser combatido com técnicas eficazes para a promoção de regularização.
- Sabe-se que a combinação de modelos, como realizado em comitê de máquinas, tende a melhorar o desempenho de máquinas de aprendizado, particularmente quando os modelos divergem no erro (PERRONE & COOPER, 1992). Duas formas alternativas de promover diferença de comportamento é propor modelos com arquiteturas distintas ou treinar com dados distintos.
- De fato, se houvesse recursos computacionais ilimitados, a melhor forma de regularizar um modelo de tamanho fixo é tirar a média de todas as possíveis atribuições de parâmetros, ponderando cada atribuição por sua probabilidade a posteriori, extraída dos dados de treinamento (BALDI & SADOWSKI, 2013).

- Particularmente no contexto de *deep learning*, a síntese independente de múltiplos modelos de aprendizado, fixos ou variáveis, a serem adequadamente combinados, é proibitiva. Além disso, se o modelo não for fixo, encontrar um conjunto adequado de hiperparâmetros para cada proposta de arquitetura é uma tarefa custosa e desafiadora. Se o modelo for fixo e a diversidade for promovida pelo treinamento com subconjuntos de dados distintos, pode não haver dado suficiente para se evitar o *overfitting* de cada modelo de aprendizado. E mesmo que se obtenha múltiplas redes profundas, colocar todas em operação simultaneamente, para se coletar uma saída de consenso, pode ser proibitivo em termos computacionais.
- Uma técnica capaz de superar todas essas limitações para se combinar vários modelos visando uma boa regularização se dá pela aplicação de *dropout* (SRIVASTAVA et al., 2014), que nada mais é do que a eliminação aleatória de neurônios da rede neural, juntamente com suas conexões sinápticas, durante o

treinamento da rede neural, visando a redução acentuada de um fenômeno denominado de co-adaptação.

- Uma outra interpretação para *dropout* é o treinamento de uma quantidade exponencial de redes neurais mais enxutas que compartilham pesos e que podem ser combinadas de forma bastante eficiente numa rede neural completa, com pesos de menor magnitude.
- A capacidade de regularização de *dropout* é superior ao que se consegue com outras técnicas de regularização já propostas na literatura, como: (1) parar o treinamento mais cedo, ao minimizar o erro junto a um conjunto de validação; (2) penalizar o aumento do módulo do vetor de pesos via norma L_1 (LASSO) ou norma L_2 (*ridge regression*); (3) promover o compartilhamento de pesos.
- A figura a seguir ilustra a aplicação de *dropout* numa rede neural do tipo perceptron de múltiplas camadas. Uma taxa de remoção de até 50% para as unidades escondidas é sugerida como uma escolha adequada (mas inicie com

menos e, se necessário, aumente), enquanto para as unidades de entrada ela deve ser mais baixa, preservando a grande maioria das entradas.

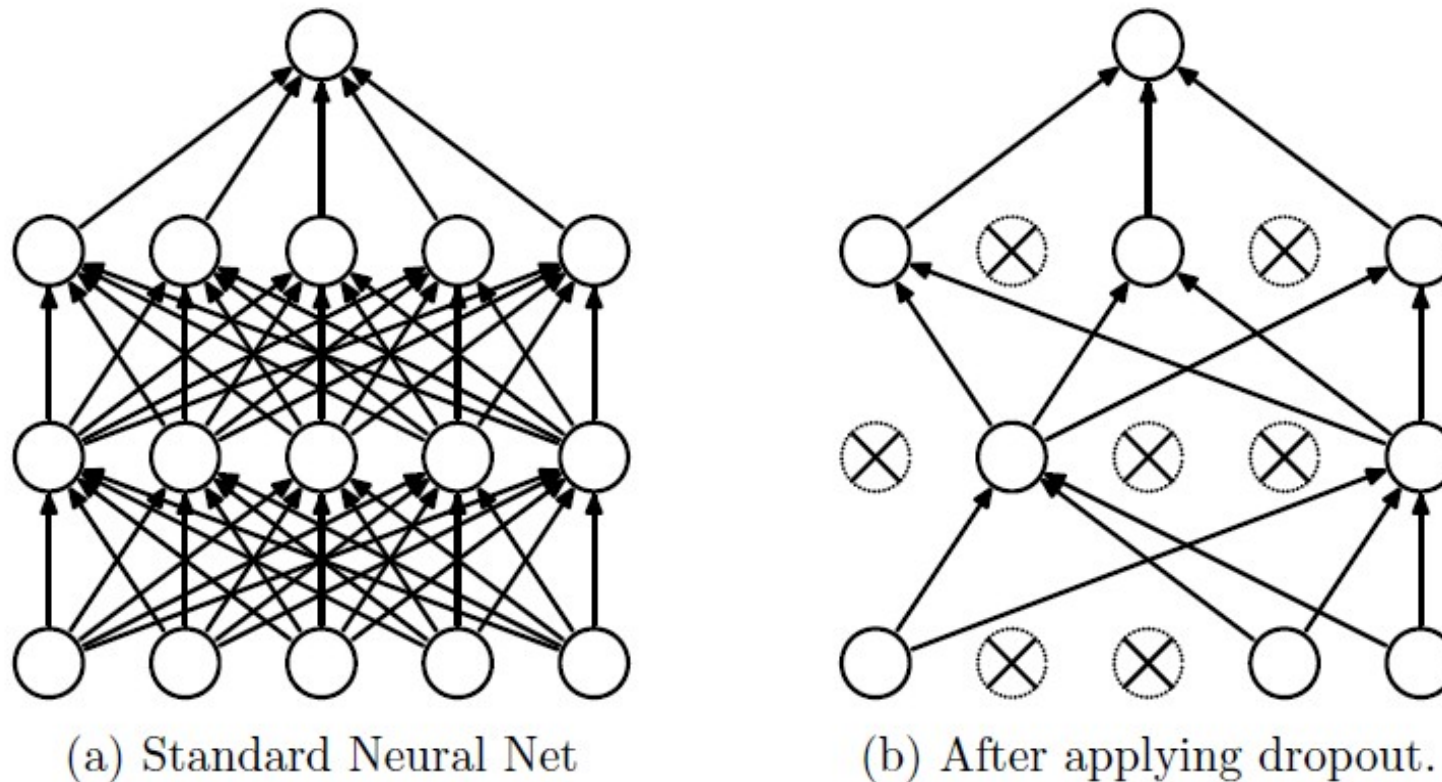


Figura 14 – Remoção temporária de neurônios de uma rede neural multicamadas do tipo *fully connected*. Fonte: SRIVASTAVA et al. (2014).

- Após o treinamento, os pesos produzidos passam a operar com os valores apresentados na figura a seguir.

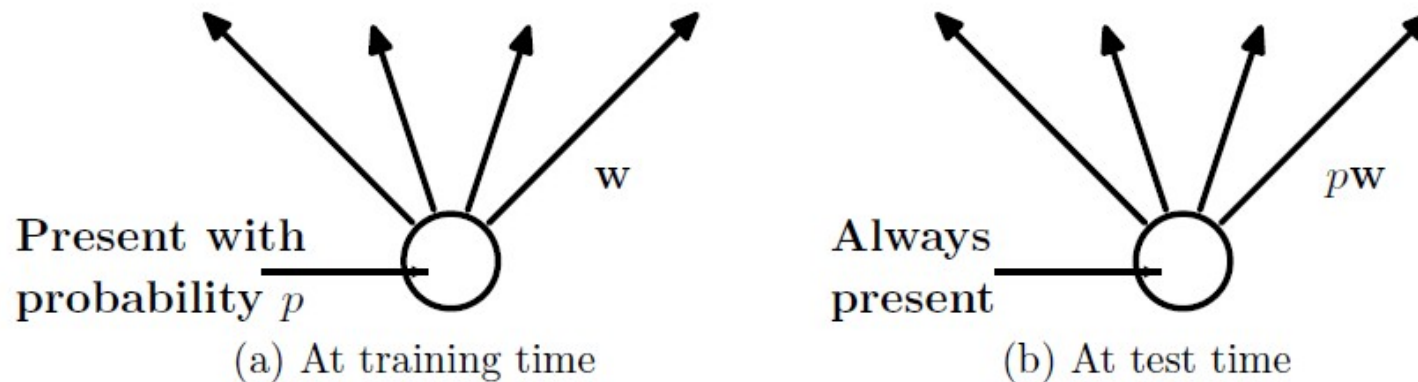


Figura 15 – Pesos durante o treinamento (a) e para a rede em operação, após o treinamento (b).
Fonte: SRIVASTAVA et al. (2014).

- Com esta política de uso, a saída da rede neural após o treinamento vai apresentar o valor médio produzido por um número exponencialmente elevado de redes neurais com remoção de unidades, que poderiam ser produzidas durante o treinamento. De fato, uma rede neural com n neurônios pode dar origem a 2^n diferentes arquiteturas que compartilham os mesmos pesos sinápticos (quando não removidos).

- A cada época ou *minibatch* de treinamento, uma rede neural enxuta é amostrada aleatoriamente da rede neural completa. Desse modo, treinar uma rede neural com n unidades aplicando *dropout* pode ser interpretado como 2^n redes neurais mais enxutas compartilhando pesos sinápticos, com cada uma das possíveis redes neurais mais enxutas sendo treinada muito raramente, se tanto.
- Em fase de operação, após terminado o treinamento, em lugar de se obter a média do comportamento de entrada-saída de 2^n redes neurais mais enxutas, o que seria inviável computacionalmente, aplica-se um método bem simples de aproximação da média, que opera bem em aplicações práticas. Usa-se uma única rede neural sem sofrer *dropout*, com cada peso obtido após o treinamento multiplicado pela probabilidade p de inclusão de sua correspondente unidade.
- FRAZIER-LOGUE & HANSON (2018) interpretaram *dropout* como um caso especial do algoritmo de treinamento *stochastic delta rule*, proposto em 1990.
- **Quanto maior a rede neural, maior tende a ser o efeito positivo de *dropout*.**

4 Comitê de máquinas – Ensemble e Mistura de Especialistas

- Método de aprendizado (supervisionado ou não-supervisionado) cujo objetivo é aumentar a *capacidade de generalização* de modelos de aprendizado.

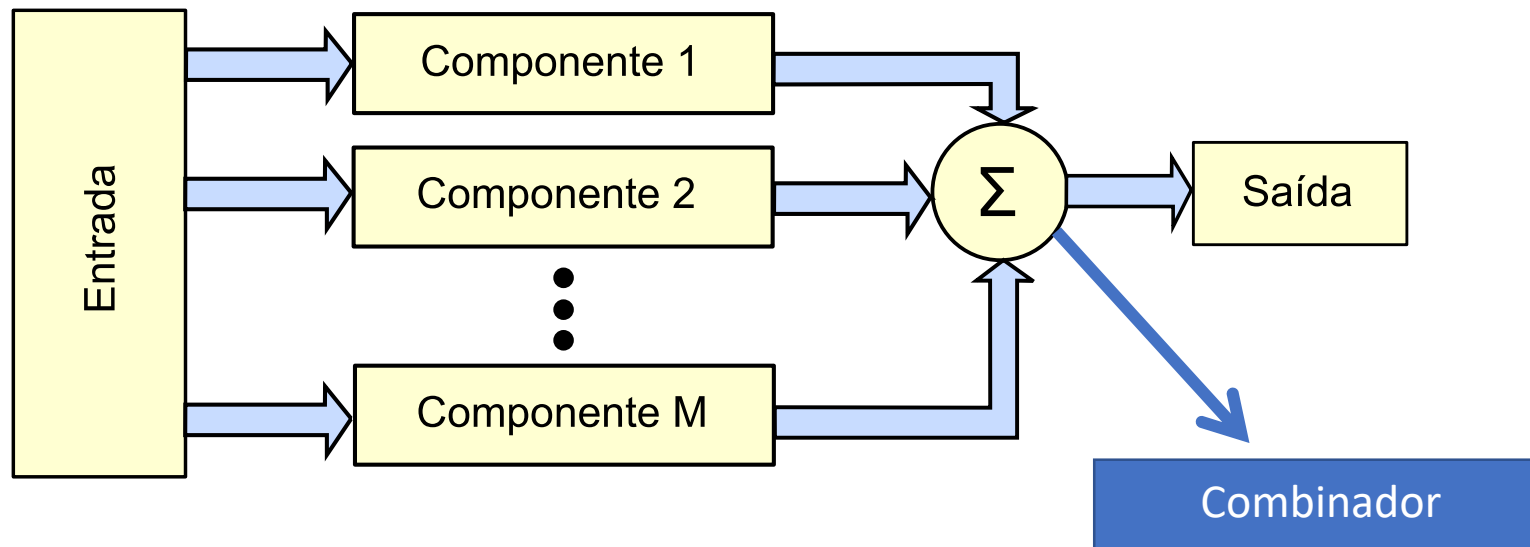


Figura 16 – Estrutura de um comitê de máquinas estático (*ensemble*)

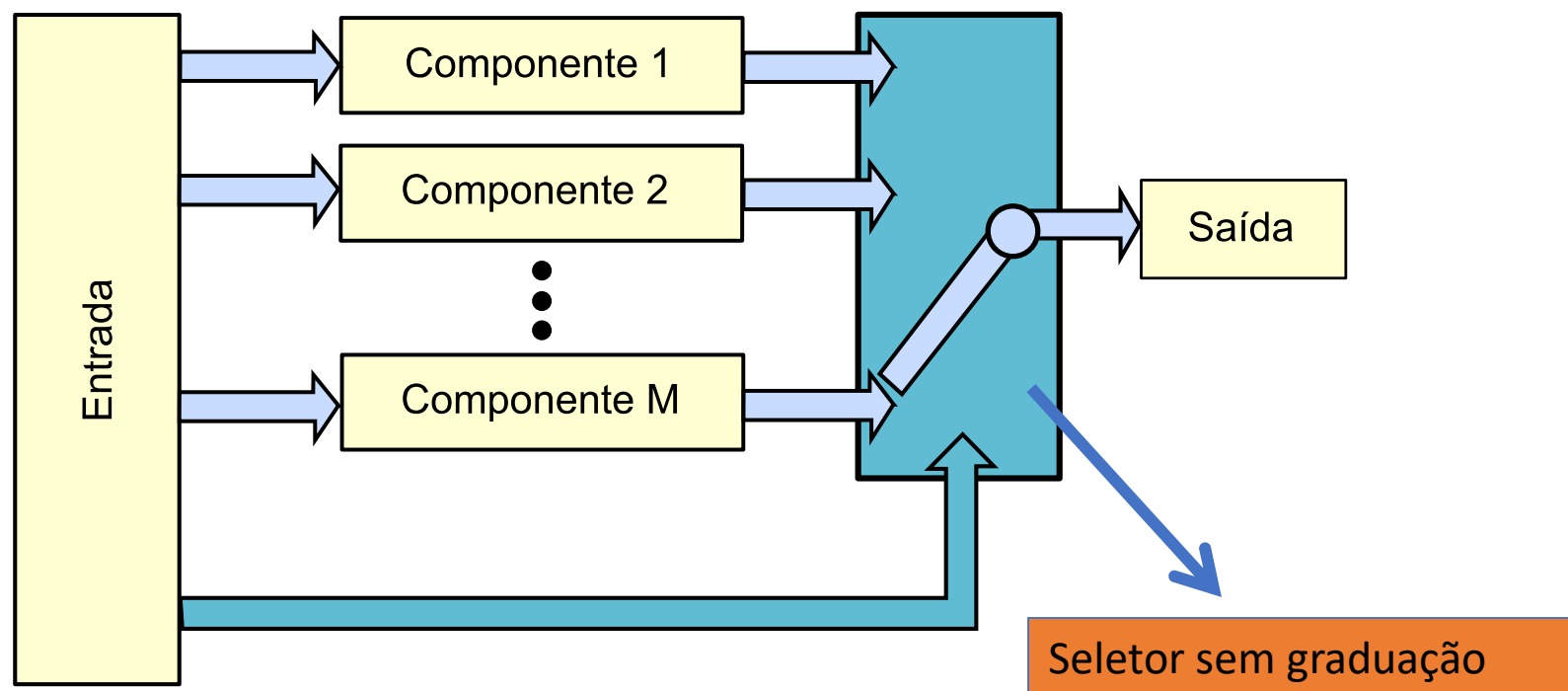


Figura 17 – Estrutura de um comitê de máquinas dinâmico, com seleção binária (*mixture of experts*)

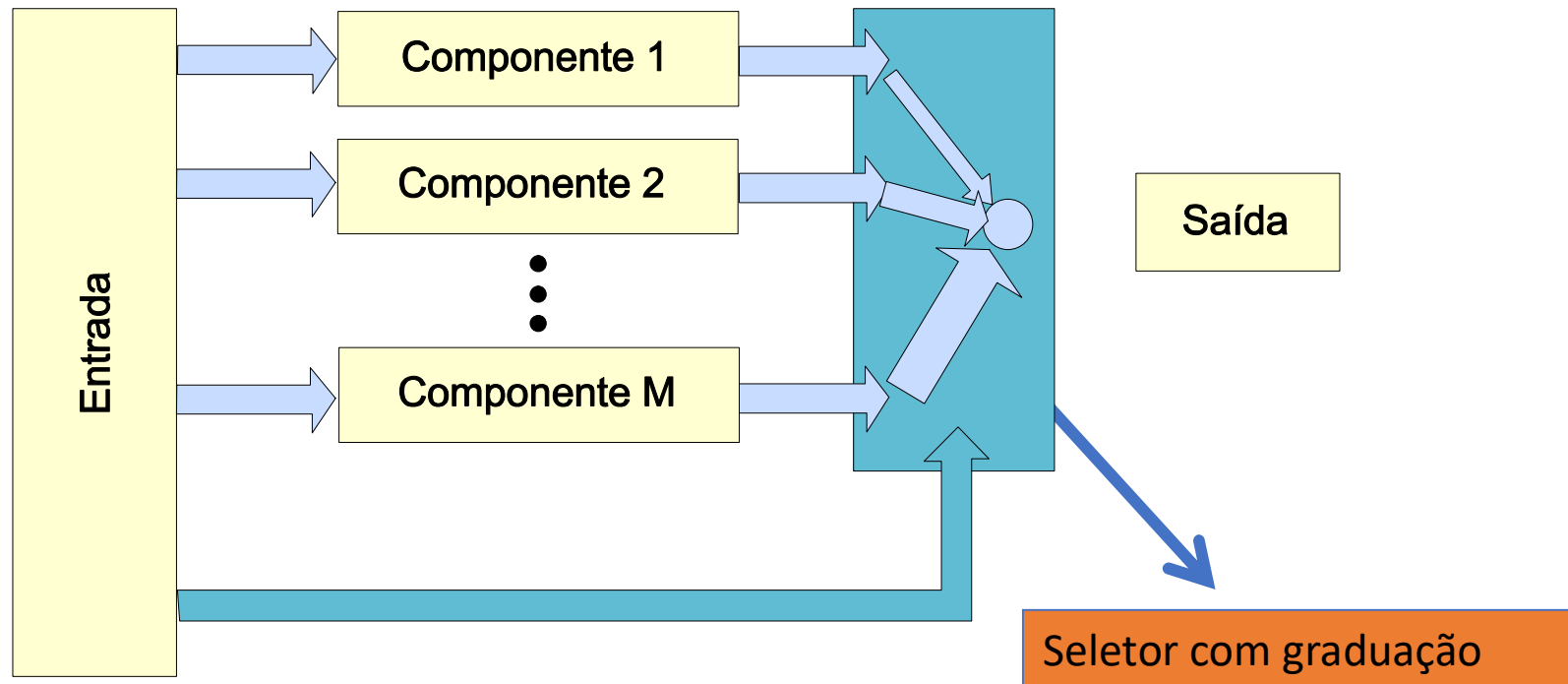


Figura 18 – Estrutura de um comitê de máquinas dinâmico, com seleção graduada (*mixture of experts*)

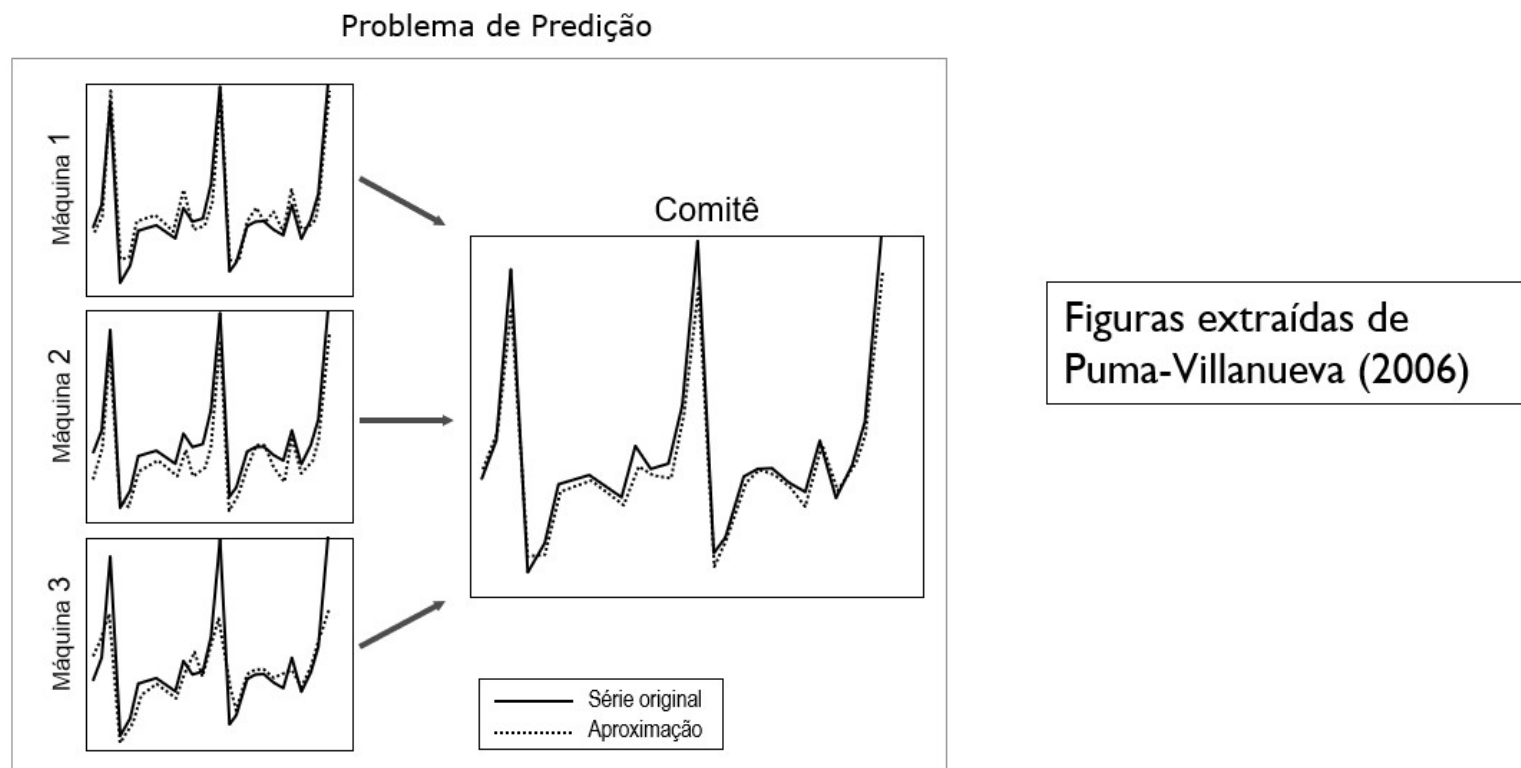
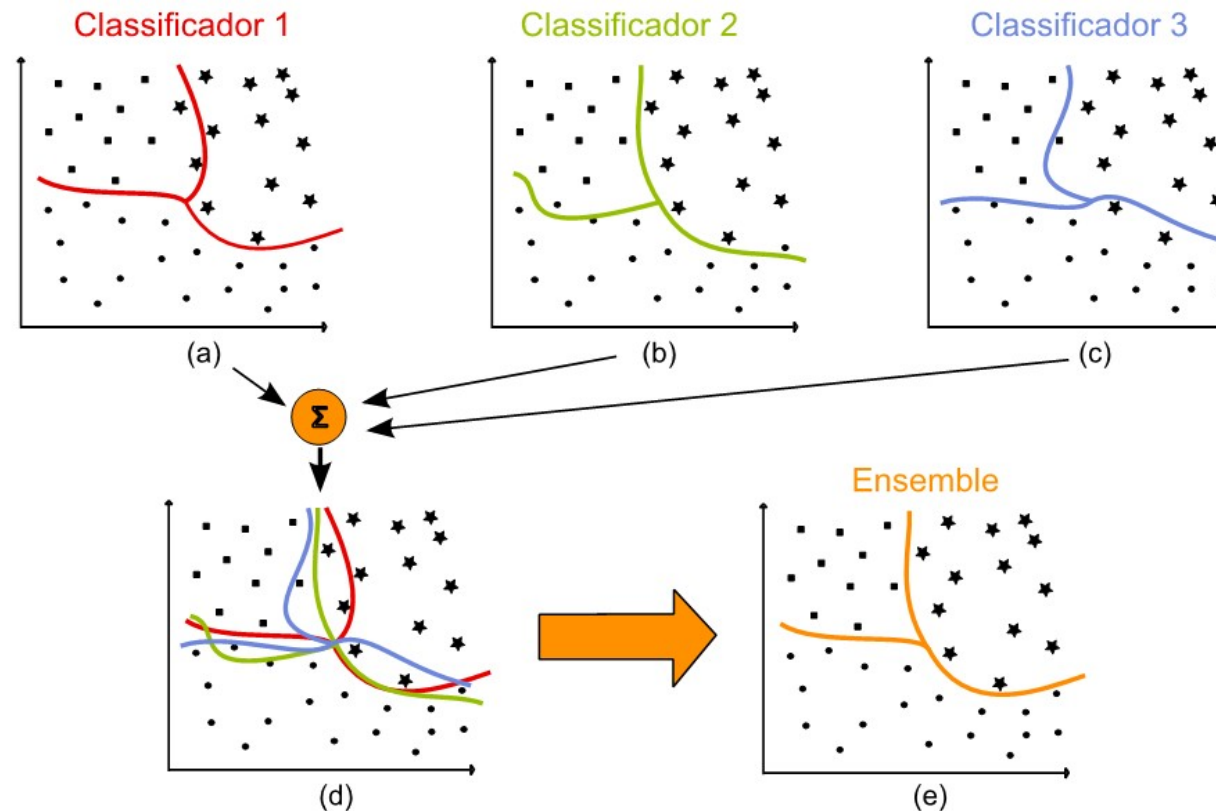


Figura 19 – Exemplo de um comitê de máquinas em predição (ensemble)



Combinação por Voto Majoritário

Figura 20 – Exemplo de um comitê de máquinas em classificação (ensemble).

Fonte: PUMA-VILLANUEVA (2006).

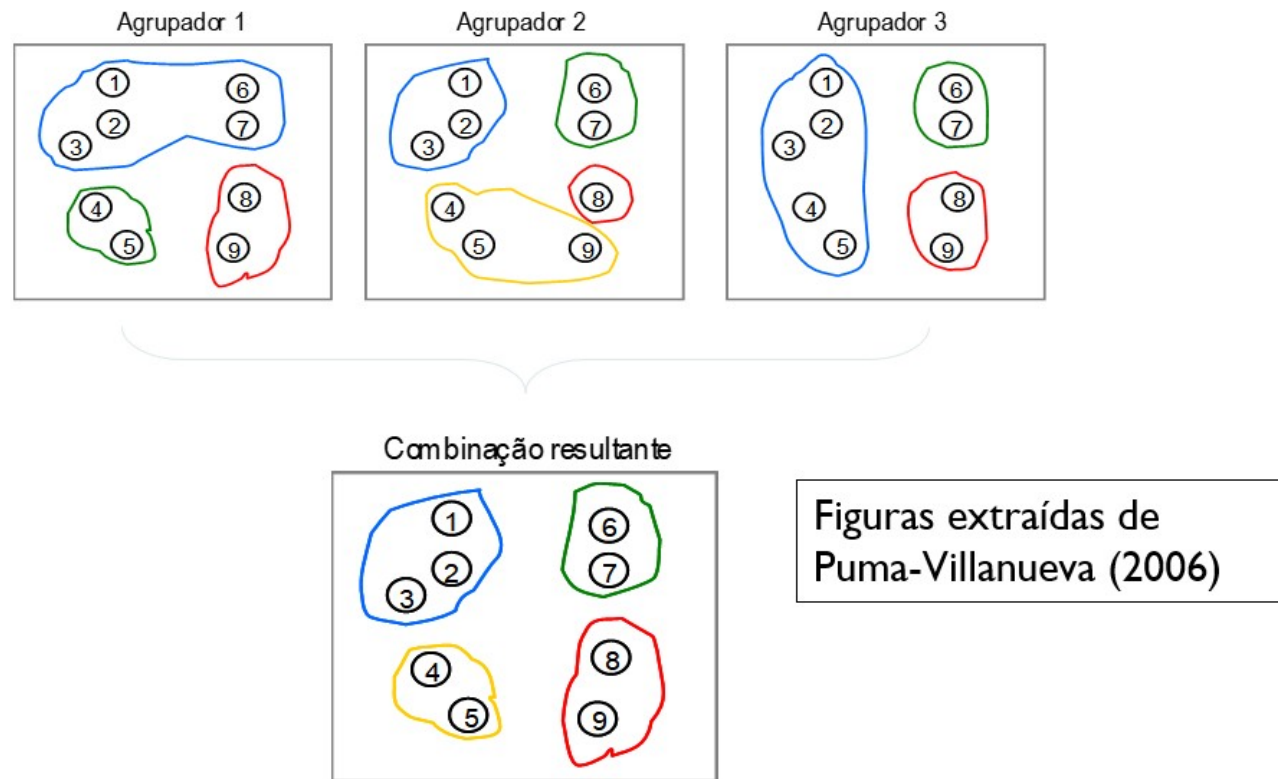


Figura 21 – Exemplo de um comitê de máquinas em agrupamento de dados (ensemble)

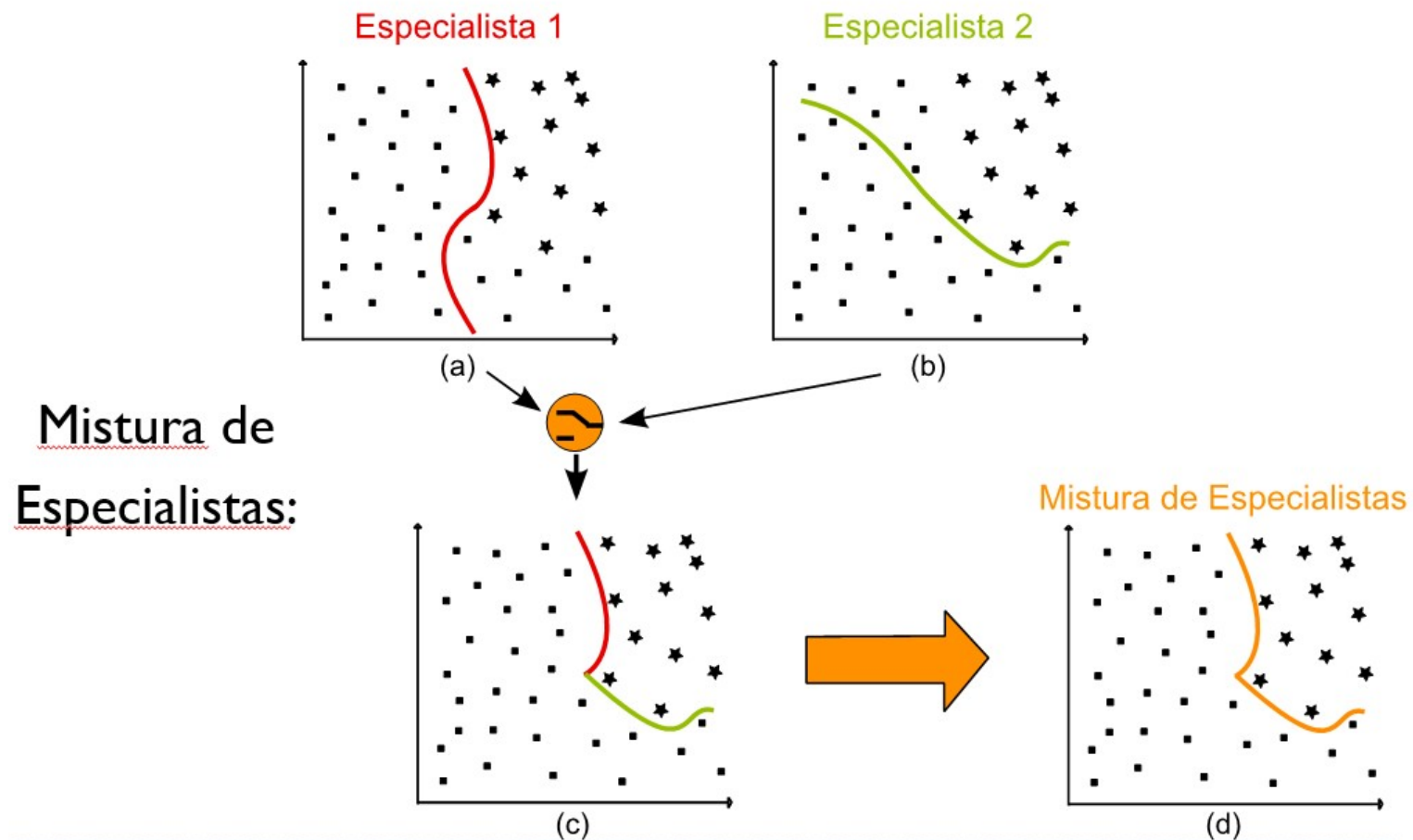


Figura 22 – Exemplo de um comitê de máquinas em classificação (mistura de especialistas).

Fonte: PUMA-VILLANUEVA (2006).

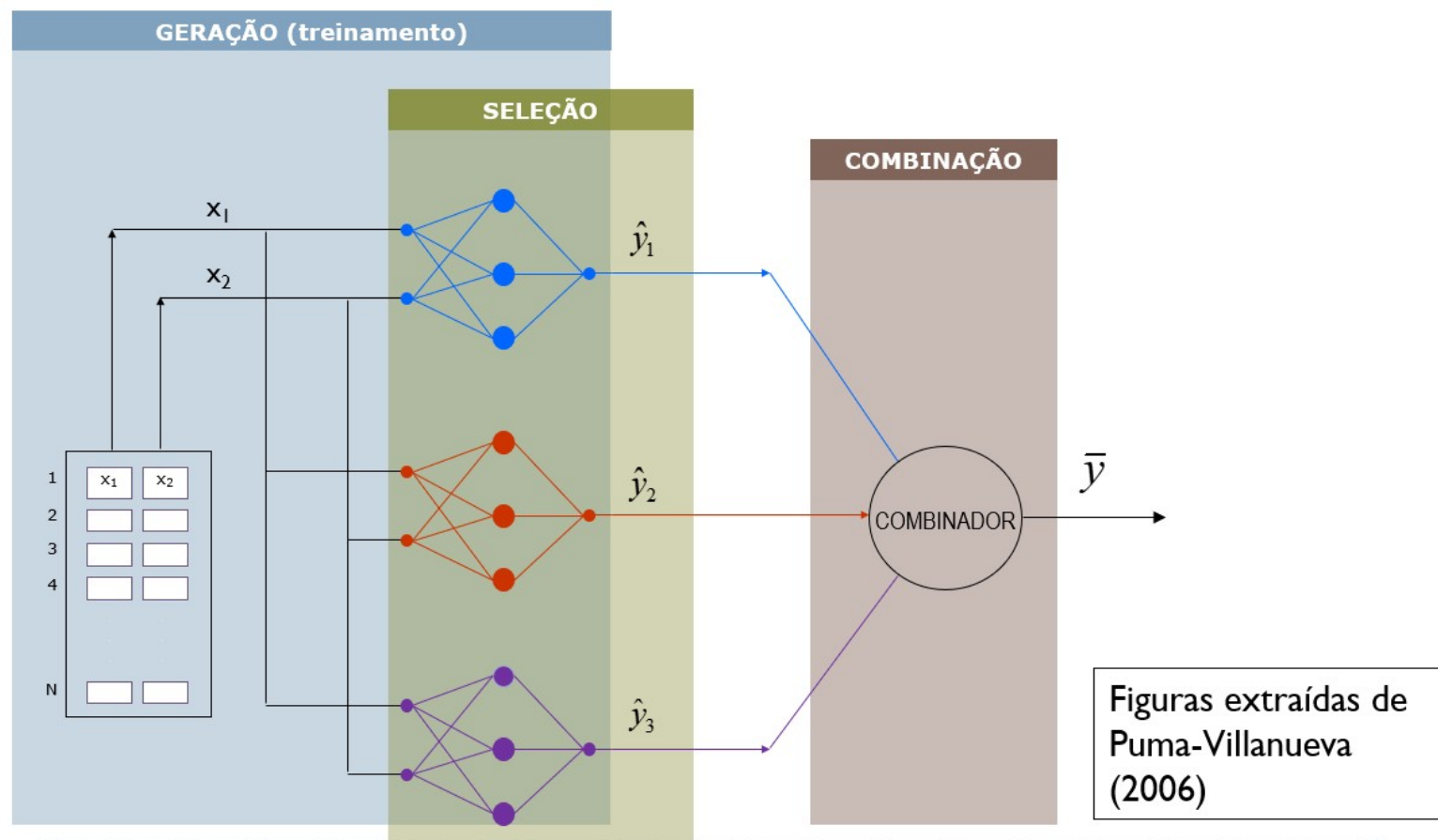


Figura 23 – Etapas na construção de um ensemble

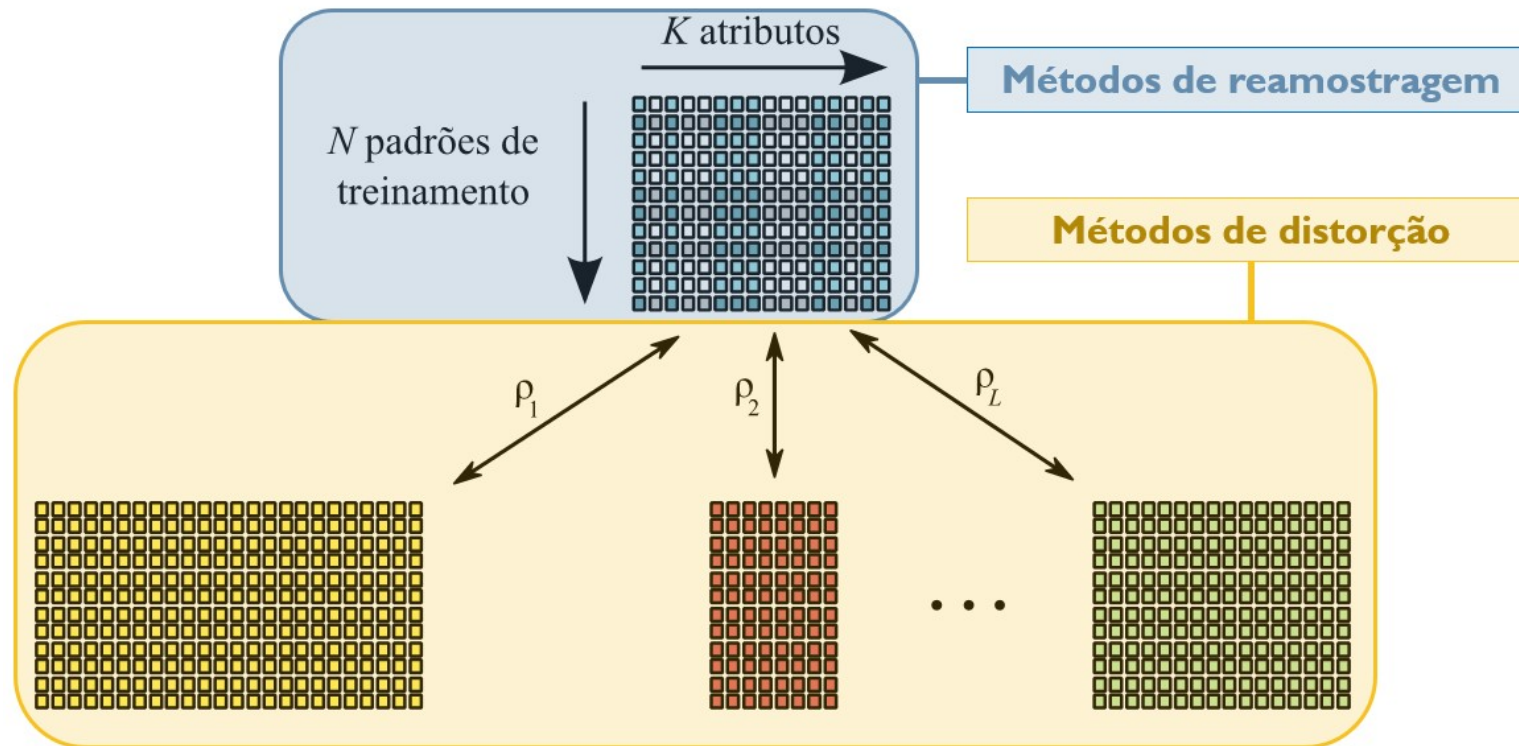


Figura 24 – Manipulação de dados para a geração de diversidade nos modelos de aprendizado.

Fonte: PUMA-VILLANUEVA (2006).

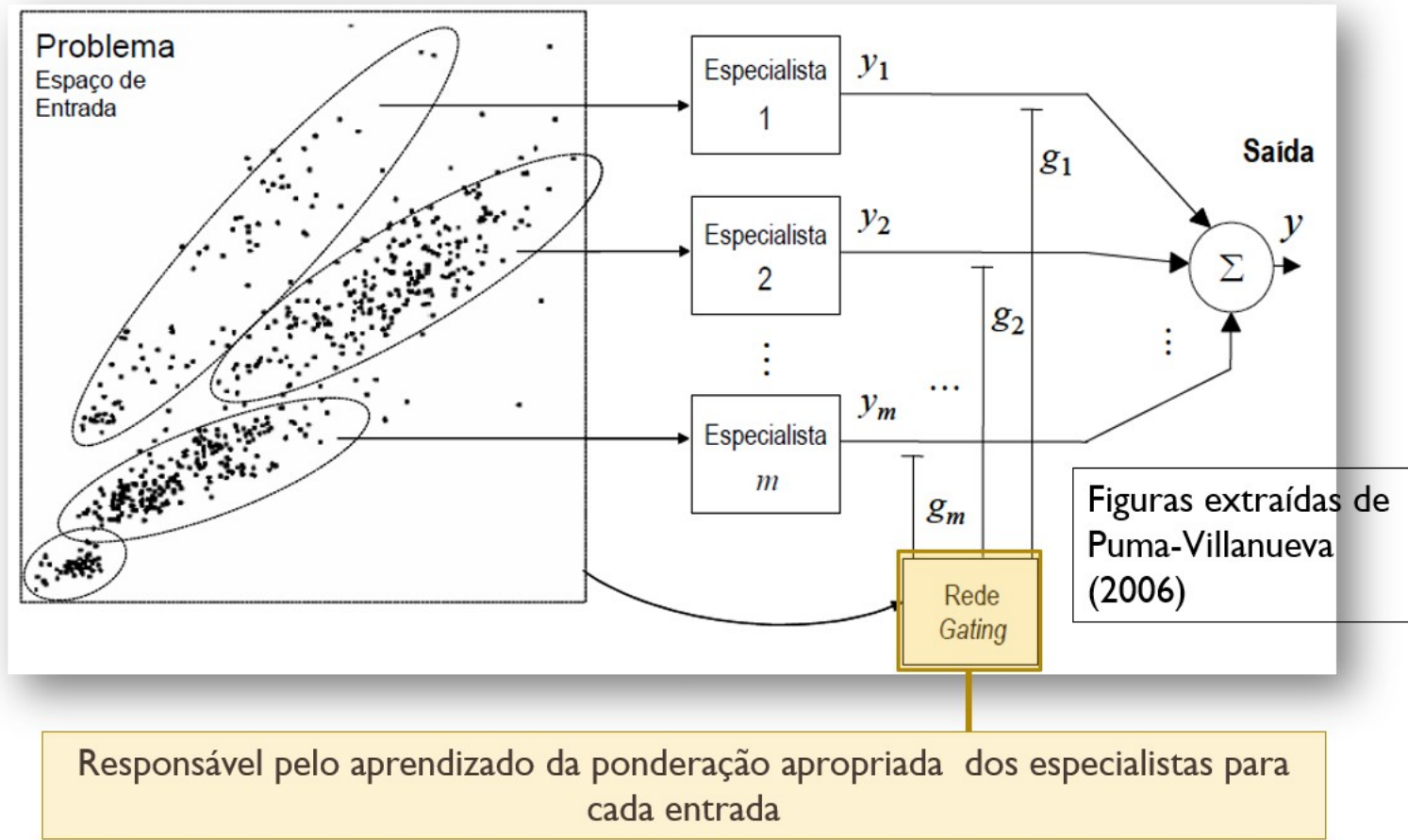
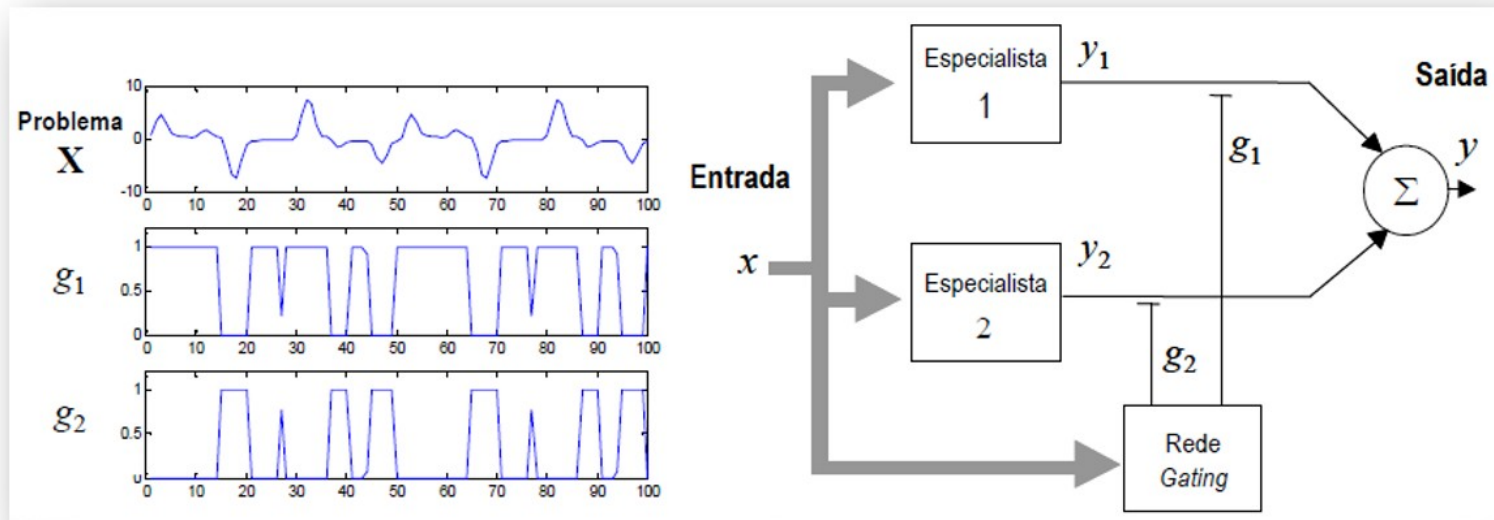


Figura 25 – Princípio de operação da rede *gating* em mistura de especialistas



Figuras extraídas de
Lima (2004)

Figura 26 – Dois especialistas em um problema de predição de série temporal, com ênfase no papel da rede *gating*

Figuras
extraídas de
Lima (2004)

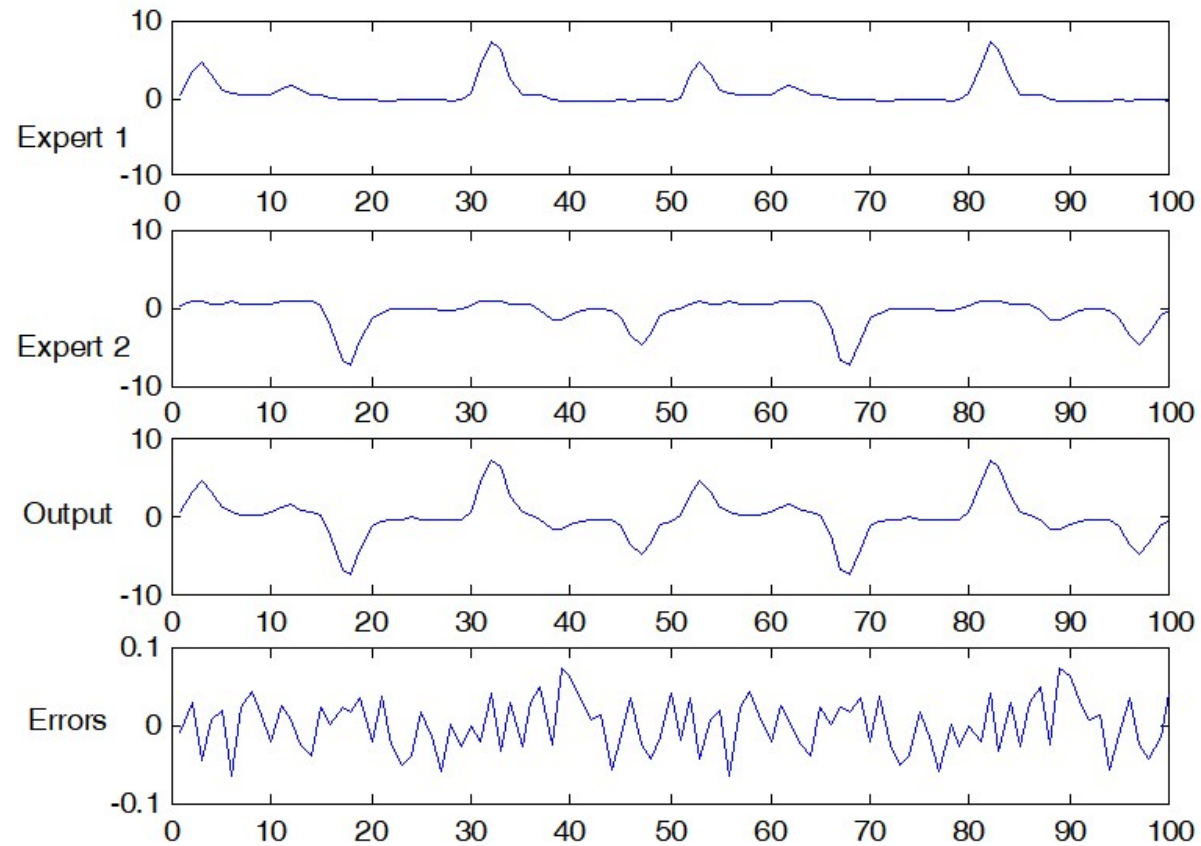


Figura 27 – Dois especialistas em um problema de predição de série temporal, com ênfase no papel de cada especialista

5 Referências bibliográficas

- BALDI, P. & SADOWSKI, P. “Understanding Dropout”, Advances in Neural Information Processing Systems (NIPS), vol. 26, 2013.
- FRAZIER-LOGUE, N. & HANSON, S.J. “Dropout is a special case of the stochastic delta rule: faster and more accurate deep learning”, arXiv:1808.03578v1, 2018.
- GANAIE, M.A.; HU, M.; MALIK, A.K.; TANVEER, M.; SUGANTHAN, P.N. “Ensemble deep learning: A review”, arXiv:2104.02395v2, 2022.
- GOODFELLOW, I.; BENGIO, Y. & COURVILLE, A. “Deep Learning”, The MIT Press, 2016.
(<http://www.deeplearningbook.org/>)
- LIMA, C.A.M. “Comitê de Máquinas: Uma Abordagem Unificada Empregando Máquinas de Vetores-Suporte”, Tese de Doutorado, FEEC, Unicamp, 2004.
- MASOUDNIA, S.; EBRAHIMPOUR, R. “Mixture of experts: a literature survey”, Artificial Intelligence Review, vol. 42, no. 2, pp. 275-293, 2014.
- PERRONE, M.P.; COOPER, L.N. “When Networks Disagree: Ensemble Methods for Hybrid Neural Networks”, in R.J. Mammone (ed.) Neural Networks for Speech and Image Processing, Chapman-Hall, 1992.
- PUMA-VILLANUEVA, W.J. “Comitê de Máquinas em Predição de Séries Temporais”, Dissertação de Mestrado, Fac. de Engenharia Elétrica e de Computação, Unicamp, 2006.
- SRIVASTAVA, N.; HINTON, G.; KRIZHEVSKY, A.; SUTSKEVER, I.; SALAKHUTDINOV, R. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”, Journal of Machine Learning Research, vol. 15, pp. 1929-1958, 2014.
- YUKSEL, S.E.; WILSON, J.N.; GADER, P.D. “Twenty Years of Mixture of Experts”, IEEE Trans. on Neural Networks and Learning Systems, vol. 23, no. 8, pp. 1177-1193, 2012.
- ZHOU, Z.-H. “Ensemble Methods: Foundations and Algorithms”, Chapman & Hall/CRC Press, 2012.